Namespace SixLabors.Fonts

Classes

Font
: Defines a particular format for text, including font face, size, and style attributes. This class cannot be inherited.

FontCollection
: Represents a collection of font families.

FontCollectionExtensions
: Extension methods for IFontCollection.

FontDescription
: Provides basic descriptive metadata for the font.

FontException
: Base class for exceptions thrown by this library.

FontFamilyNotFoundException
: Exception for detailing missing font families.

FontGlyphMetrics
: Represents a glyph metric from a particular font face.

FontMetrics
: Represents a font face with metrics, which is a set of glyphs with a specific style (regular, italic, bold etc).

FontPalette
: Selects the CPAL color palette used to resolve COLR glyph colors and optionally replaces individual palette entries.

  A palette index outside the range defined by the font selects the default palette (index 0), and overrides still apply, matching the CSS `font-palette` behavior. Overrides apply in order, so a later override of the same entry replaces an earlier one, and overrides whose entry index lies outside the font's palette entry range are ignored.

GlyphMissingException
: Exception for detailing missing font families.

GlyphOptions

Provides configuration options for rendering a glyph by id.

HorizontalMetrics
Represent the metrics of a font face specific to horizontal text.

InvalidFontFileException
Exception font loading can throw if it encounters invalid data during font loading.

InvalidFontTableException
Exception font loading can throw if it encounters invalid data during font loading.

KnownVariationAxes
Defines the registered design-variation axis tags for variable fonts. These tags are used with FontVariation to control font design axes. https://learn.microsoft.com/en-us/typography/opentype/spec/dvaraxisreg

LayoutModeExtensions
Extensions to LayoutMode.

LineLayout
Represents one laid-out line from a TextBlock.

LineLayoutEnumerator
Walks a TextBlock one laid-out line at a time.

MissingFontTableException
Exception font loading can throw if it finds a required table is missing during font loading.

SystemFontFallbackResolver
Resolves fallback font families from the fonts installed on the current machine using the operating system's character-to-font matching service. Results depend on the machine's installed fonts: the same input can resolve different families, or none, on different machines.

SystemFonts
Provides a collection of fonts.

TextBlock
Represents text prepared for repeated line layout, measurement, and rendering.

TextDecorationOptions
Represents overrides for the geometry of a single text decoration line, allowing a TextRun to replace the values that would otherwise be derived from the font metrics.

TextMeasurer
Encapsulates logic for laying out and then measuring text properties.

TextMetrics
: Encapsulates the full set of measurement results for laid-out text.

TextOptions
: Provides configuration options for rendering and shaping text.

TextRun
: Represents a run of text spanning a series of graphemes within a string.

TextShaper
: Encapsulates logic for shaping text into a positioned glyph stream.

TextShapingBuffer
: A reusable, caller-owned buffer holding text, the properties it is shaped under, and the glyphs shaping produces. Text and properties are set on the buffer, the buffer is shaped, and the glyphs are read back from it. Each shaping call replaces the glyphs.

VerticalMetrics
: Represent the metrics of a font face specific to vertical text.

Structs

CaretPosition
: Represents a caret line in laid-out text.

ClipBounds
: Represents a rectangular clipping region as axis-aligned bounds in the design space of the glyph source together with the transform that maps that space to the rendering surface.

FontFamily
: Defines a group of type faces having a similar basic design and certain variations in styles.

FontMatch
: Represents a font family and style matched for a character.

FontPaletteOverride
: Replaces the color of a single palette entry when a FontPalette is applied. An override whose Index lies outside the font's palette entry range is ignored.

FontRectangle
: Stores a set of four single precision floating points that represent the location and size of a rectangle.

FontVariation

Represents a single variation axis setting for a variable font, consisting of a four-character tag and a value.

Glyph
Represents a font-specific glyph at the point size used for layout and rendering.

GlyphColor
Provides access to the color details for the current glyph.

GlyphMetrics
Represents one laid-out glyph entry in final layout order.

GraphemeMetrics
Represents one coalesced grapheme in final layout order.

LineMetrics
Encapsulates measured metrics for a single laid-out text line.

ShapedGlyph
Represents a single shaped glyph: the result of substitution and positioning, before line breaking.

TextHit
Represents a hit-tested grapheme position in laid-out text.

TextPlaceholder
Represents an atomic inline placeholder with caller-supplied dimensions.

WordMetrics
Represents the positioned metrics for one Unicode word-boundary segment.

Interfaces

IFontCollection
A readable and writable collection of fonts.

IFontFallbackResolver
Resolves a font family for code points that no configured font can shape. The shaping pipeline consults the resolver only after Font and every FallbackFontFamilies entry have attempted the text, and at most once per distinct unresolved code point per shaping operation.

IMetricsHeader
Defines the contract for the metrics header of a font face.

IReadOnlyFontCollection

Represents a readonly collection of fonts.

IReadOnlySystemFontCollection
Represents a readonly collection of Operating System fonts.

Enums

CaretMovement
Specifies a caret movement operation within laid-out text.

CaretPlacement
Specifies an absolute caret placement within a laid-out text scope.

ColorFontSupport
Specifies which color font formats are enabled for layout and rendering.

DecorationPositioningMode
Defines how text decorations (underline, overline, strikethrough) are positioned relative to font metrics.

FontStyle
The font styles

FontWeight
Identifies the visual weight of a font on the OpenType and CSS weight scale.

GlyphLayoutMode
Provides enumeration for the various layout mode of an individual glyph within a body of text.

GlyphType
Represents the various versions of a glyph records.

HintingMode
Defines modes to determine how to apply hinting. The use of mathematical instructions to adjust the display of an outline font so that it lines up with a rasterized grid.

HorizontalAlignment
Horizontal alignment modes.

KerningMode
Kerning is the contextual adjustment of inter-glyph spacing. This property controls metric kerning, kerning that utilizes adjustment data contained in the font.

LayoutMode
Defines modes to determine the layout direction of text.

TextAlignment
: Text alignment modes.

TextAttributes
: Provides enumeration of various text attributes.

TextBaseline
: Specifies which reference line of laid-out text is placed at Origin along the block flow axis, and equivalently which line rides the path when text follows one. Values match the CSS `dominant-baseline` and HTML canvas `textBaseline` vocabulary.

TextBidiMode
: Specifies how bidirectional text is resolved.

TextDecorationSkipInk
: Identifies whether underline and overline decorations are interrupted where they would otherwise cross glyph ink, mirroring the CSS `text-decoration-skip-ink` property. Strikethrough decorations always cross the ink regardless of this setting.

TextDecorations
: Provides enumeration of various text decorations.

TextDirection
: Specifies the writing direction for text.

TextEllipsis
: Specifies ellipsis behavior when laid-out text is limited to a maximum number of lines.

TextHyphenation
: Specifies hyphenation marker behavior when text breaks at hyphenation opportunities.

TextInteractionMode
: Specifies how text interaction positions are modeled for laid-out text.

TextJustification
: Text justification modes.

TextPlaceholderAlignment
: Specifies how an inline placeholder is aligned against surrounding text.

VerticalAlignment
: Vertical alignment modes.

WordBreaking

Defines modes to determine when line breaks should appear when words overflow their content box.

Enum CaretMovement

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies a caret movement operation within laid-out text.

```
public enum CaretMovement
```

Fields

`LineDown = 9`

Move to the next visual line.

`LineEnd = 5`

Move to the end of the current line.

`LineStart = 4`

Move to the start of the current line.

`LineUp = 8`

Move to the previous visual line.

`Next = 1`

Move to the next grapheme insertion position.

`NextWord = 3`

Move to the next Unicode word boundary.

`Previous = 0`

Move to the previous grapheme insertion position.

`PreviousWord = 2`

Move to the previous Unicode word boundary.

`TextEnd = 7`

Move to the end of the laid-out text.

`TextStart = 6`

Move to the start of the laid-out text.

Enum CaretPlacement

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies an absolute caret placement within a laid-out text scope.

```
public enum CaretPlacement
```

Fields

`End = 1`

Place the caret at the end of the laid-out text scope.

`Start = 0`

Place the caret at the start of the laid-out text scope.

Struct CaretPosition

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a caret line in laid-out text.

```
public readonly struct CaretPosition
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

End

Gets the caret end point in pixel units.

```
public Vector2 End { get; }
```

Property Value

Vector2

GraphemeIndex

Gets the zero-based grapheme index in the original text.

```
public int GraphemeIndex { get; }
```

Property Value

int

HasSecondary

Gets a value indicating whether a second visual caret position is available.

```
public bool HasSecondary { get; }
```

Property Value

bool

LineIndex

Gets the zero-based line index.

```
public int LineIndex { get; }
```

Property Value

int

SecondaryEnd

Gets the secondary caret end point in pixel units.

```
public Vector2 SecondaryEnd { get; }
```

Property Value

Vector2

SecondaryStart

Gets the secondary caret start point in pixel units.

```
public Vector2 SecondaryStart { get; }
```

Property Value

Vector2

Start

Gets the caret start point in pixel units.

```
public Vector2 Start { get; }
```

Property Value

Vector2

StringIndex

Gets the zero-based UTF-16 code unit index in the original text.

```
public int StringIndex { get; }
```

Property Value

int

Struct ClipBounds

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a rectangular clipping region as axis-aligned bounds in the design space of the glyph source together with the transform that maps that space to the rendering surface.

```
public readonly struct ClipBounds
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Remarks

A renderer clips against Bounds in design space first, then applies Transform to the result. The bounds are never pre-transformed: a transformed region loses its axis-aligned rectangle and with it the ability to intersect exactly in design space.

Constructors

ClipBounds(FontRectangle, Matrix3×2)

Initializes a new instance of the ClipBounds struct.

```
public ClipBounds(FontRectangle bounds, Matrix3x2 transform)
```

Parameters

`bounds` FontRectangle

The axis-aligned bounds of the clipping region in design space.

`transform` Matrix3×2

The transform from the design space of `bounds` to the rendering surface.

Properties

Bounds

Gets the axis-aligned bounds of the clipping region in design space.

```
public FontRectangle Bounds { get; }
```

Property Value

FontRectangle

Transform

Gets the transform from the design space of Bounds to the rendering surface.

```
public Matrix3x2 Transform { get; }
```

Property Value

Matrix3×2

Enum ColorFontSupport

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies which color font formats are enabled for layout and rendering.

```
[Flags]
public enum ColorFontSupport
```

Fields

`ColrV0 = 1`

Enable rendering of COLR version 0 color glyphs (layered solid colors defined by COLR/CPAL tables).

`ColrV1 = 2`

Enable rendering of COLR version 1 color glyphs (paint graph-based color glyphs with gradients and transforms).

`None = 0`

Disable color font rendering entirely. All glyphs will be drawn as monochrome outlines.

`Svg = 4`

Enable rendering of color glyphs stored as SVG documents in the OpenType SVG table.

Remarks

This enumeration allows a renderer to select which OpenType color font technologies to honor when processing glyph runs. Multiple formats may be enabled simultaneously.

Enum DecorationPositioningMode

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Defines how text decorations (underline, overline, strikethrough) are positioned relative to font metrics.

```
public enum DecorationPositioningMode
```

Fields

`GlyphFont = 1`

Uses each glyph's own font metrics to position its decoration. Decoration positions may vary between glyphs and fallback fonts within the same line. Matches typical Microsoft Word behavior.

`PrimaryFont = 0`

Uses the primary (base) font's metrics for the entire run or line, ensuring a consistent decoration position across mixed fonts and scripts. Matches typical browser behavior.

Class Font

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Defines a particular format for text, including font face, size, and style attributes. This class cannot be inherited.

```
public sealed class Font
```

Inheritance

object ← Font

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

Font(Font, FontStyle)

Initializes a new instance of the Font class.

```
public Font(Font prototype, FontStyle style)
```

Parameters

`prototype` Font

The prototype.

`style` FontStyle

The font style.

Font(Font, params FontVariation[])

Initializes a new instance of the Font class with the specified variation axis settings.

```
public Font(Font prototype, params FontVariation[] variations)
```

Parameters

`prototype` Font

The prototype font providing family, size, and style.

`variations` FontVariation[]

The variation axis settings to apply.

Font(Font, float)

Initializes a new instance of the Font class.

```
public Font(Font prototype, float size)
```

Parameters

`prototype` Font

The prototype.

`size` float

The size of the font in PT units.

Font(Font, float, FontStyle)

Initializes a new instance of the Font class.

```
public Font(Font prototype, float size, FontStyle style)
```

Parameters

`prototype` Font

The prototype.

`size` float

The size of the font in PT units.

`style` FontStyle

The font style.

Font(FontFamily, float)

Initializes a new instance of the Font class.

```
public Font(FontFamily family, float size)
```

Parameters

`family` FontFamily

The font family.

`size` float

The size of the font in PT units.

Font(FontFamily, float, FontStyle)

Initializes a new instance of the Font class.

```
public Font(FontFamily family, float size, FontStyle style)
```

Parameters

`family` FontFamily

The font family.

`size` float

The size of the font in PT units.

`style` FontStyle

The font style.

Properties

Family

Gets the family.

```
public FontFamily Family { get; }
```

Property Value

FontFamily

FontMetrics

Gets the font metrics.

```
public FontMetrics FontMetrics { get; }
```

Property Value

FontMetrics

Exceptions

FontException

Font instance not found.

IsBold

Gets a value indicating whether this Font is bold.

```
public bool IsBold { get; }
```

Property Value

bool

IsItalic

Gets a value indicating whether this Font is italic.

```
public bool IsItalic { get; }
```

Property Value

bool

Name

Gets the name.

```
public string Name { get; }
```

Property Value

string

Size

Gets the size of the font in PT units.

```
public float Size { get; }
```

Property Value

float

Variations

Gets the variation axis settings applied to this font.

```
public ReadOnlySpan<FontVariation> Variations { get; }
```

Property Value

ReadOnlySpan<FontVariation>

Methods

TryGetGlyph(CodePoint, TextAttributes, LayoutMode, ColorFontSupport, out Glyph?)

Gets the glyph for the given codepoint.

```
public bool TryGetGlyph(CodePoint codePoint, TextAttributes textAttributes, LayoutMode
layoutMode, ColorFontSupport support, out Glyph? glyph)
```

Parameters

`codePoint` CodePoint

The code point of the character.

`textAttributes` TextAttributes

The text attributes to apply to the glyphs.

`layoutMode` LayoutMode

The layout mode to apply to the glyphs.

`support` ColorFontSupport

Options for enabling color font support during layout and rendering.

`glyph` Glyph?

When this method returns, contains the glyph for the given codepoint, attributes, and color support if the glyph is found; otherwise the default value. This parameter is passed uninitialized.

Returns

bool

true if the face contains glyphs for the specified codepoint; otherwise, false.

TryGetGlyph(CodePoint, TextAttributes, TextDecorations, LayoutMode, ColorFontSupport, FontPalette?, out Glyph?)

Gets the glyph for the given codepoint.

```
public bool TryGetGlyph(CodePoint codePoint, TextAttributes textAttributes, TextDecorations textDecorations, LayoutMode layoutMode, ColorFontSupport support, FontPalette? palette, out Glyph? glyph)
```

Parameters

`codePoint` CodePoint

The code point of the character.

`textAttributes` TextAttributes

The text attributes to apply to the glyphs.

`textDecorations` TextDecorations

The text decorations to apply to the glyphs.

`layoutMode` LayoutMode

The layout mode to apply to the glyphs.

`support` ColorFontSupport

Options for enabling color font support during layout and rendering.

`palette` FontPalette

The color palette selection for color fonts, or null for the font's default palette.

`glyph` Glyph?

When this method returns, contains the glyph for the given codepoint, attributes, and color support if the glyph is found; otherwise the default value. This parameter is passed uninitialized.

Returns

bool

true if the face contains glyphs for the specified codepoint; otherwise, false.

TryGetGlyph(CodePoint, TextAttributes, TextDecorations, LayoutMode, ColorFontSupport, out Glyph?)

Gets the glyph for the given codepoint.

```
public bool TryGetGlyph(CodePoint codePoint, TextAttributes textAttributes, TextDecorations
textDecorations, LayoutMode layoutMode, ColorFontSupport support, out Glyph? glyph)
```

Parameters

`codePoint` CodePoint

The code point of the character.

`textAttributes` TextAttributes

The text attributes to apply to the glyphs.

`textDecorations` TextDecorations

The text decorations to apply to the glyphs.

`layoutMode` LayoutMode

The layout mode to apply to the glyphs.

`support` ColorFontSupport

Options for enabling color font support during layout and rendering.

`glyph` Glyph?

When this method returns, contains the glyph for the given codepoint, attributes, and color support if the glyph is found; otherwise the default value. This parameter is passed uninitialized.

Returns

bool

true if the face contains glyphs for the specified codepoint; otherwise, false.

TryGetGlyphId(CodePoint, out ushort)

Gets the glyph identifier for the given code point.

```
public bool TryGetGlyphId(CodePoint codePoint, out ushort glyphId)
```

Parameters

`codePoint` CodePoint

The code point of the character.

`glyphId` ushort

When this method returns, contains the glyph identifier for the given code point if the glyph is found; otherwise 0. This parameter is passed uninitialized.

Returns

bool

true if the face contains a glyph for the specified code point; otherwise, false.

TryGetGlyphs(CodePoint, ColorFontSupport, out Glyph?)

Gets the glyph for the given codepoint.

```
public bool TryGetGlyphs(CodePoint codePoint, ColorFontSupport support, out Glyph? glyph)
```

Parameters

`codePoint` CodePoint

The code point of the character.

`support` ColorFontSupport

Options for enabling color font support during layout and rendering.

`glyph` Glyph?

When this method returns, contains the glyphs for the given codepoint and color support if the glyph is found; otherwise the default value. This parameter is passed uninitialized.

Returns

bool

true if the face contains glyphs for the specified codepoint; otherwise, false.

TryGetGlyphs(CodePoint, TextAttributes, ColorFontSupport, out Glyph?)

Gets the glyph for the given codepoint.

```
public bool TryGetGlyphs(CodePoint codePoint, TextAttributes textAttributes,
ColorFontSupport support, out Glyph? glyph)
```

Parameters

`codePoint` CodePoint

The code point of the character.

`textAttributes` TextAttributes

The text attributes to apply to the glyphs.

`support` ColorFontSupport

Options for enabling color font support during layout and rendering.

`glyph` Glyph?

When this method returns, contains the glyph for the given codepoint, attributes, and color support if the glyph is found; otherwise the default value. This parameter is passed uninitialized.

Returns

bool

true if the face contains glyphs for the specified codepoint; otherwise, false.

TryGetGlyphs(CodePoint, out Glyph?)

Gets the glyph for the given codepoint.

```
public bool TryGetGlyphs(CodePoint codePoint, out Glyph? glyph)
```

Parameters

`codePoint` CodePoint

The code point of the character.

`glyph` Glyph?

When this method returns, contains the glyph for the given codepoint if the glyph is found; otherwise the default value. This parameter is passed uninitialized.

Returns

bool

true if the face contains glyphs for the specified codepoint; otherwise, false.

TryGetKerningOffset(Glyph, Glyph, float, out Vector2)

Gets the amount, in px units, the `current` glyph should be offset if it is followed by the `next` glyph.

```
public bool TryGetKerningOffset(Glyph current, Glyph next, float dpi, out Vector2 vector)
```

Parameters

`current` Glyph

The current glyph.

`next` Glyph

The next glyph.

`dpi` float

The DPI (Dots Per Inch) to render/measure the kerning offset at.

`vector` Vector2

When this method returns, contains the offset, in font units, that should be applied to the `current` glyph, if the offset is found; otherwise the default vector value. This parameter is passed uninitialized.

Returns

bool

true if the face contains and offset for the glyph combination; otherwise, false.

TryGetPath(out string?)

Gets the filesystem path to the font family source.

```
public bool TryGetPath(out string? path)
```

Parameters

`path` string

When this method returns, contains the filesystem path to the font family source, if the path exists; otherwise, the default value for the type of the path parameter. This parameter is passed uninitialized.

Returns

bool

true if the Font was created via a filesystem path; otherwise, false.

Class FontCollection

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a collection of font families.

```
public sealed class FontCollection : IFontCollection, IReadOnlyFontCollection
```

Inheritance

object ← FontCollection

Implements

IFontCollection, IReadOnlyFontCollection

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Extension Methods

FontCollectionExtensions.AddSystemFonts(FontCollection) , FontCollectionExtensions.AddSystemFonts(FontCollection, Predicate<FontMetrics>)

Constructors

FontCollection()

Initializes a new instance of the FontCollection class.

```
public FontCollection()
```

Properties

Families

Gets the collection of FontFamily in this IReadOnlyFontCollection using the invariant culture.

```
public IEnumerable<FontFamily> Families { get; }
```

Property Value

IEnumerable<FontFamily>

Methods

Add(Stream)

Adds a font to the collection.

```
public FontFamily Add(Stream stream)
```

Parameters

`stream` Stream

The font stream.

Returns

FontFamily

The new FontFamily.

Add(Stream, out FontDescription)

Adds a font to the collection.

```
public FontFamily Add(Stream stream, out FontDescription description)
```

Parameters

`stream` Stream

The font stream.

`description` FontDescription

The description of the added font.

Returns

FontFamily

The new FontFamily.

Add(string)

Adds a font to the collection.

```
public FontFamily Add(string path)
```

Parameters

`path` string

The filesystem path to the font file.

Returns

FontFamily

The new FontFamily.

Add(string, out FontDescription)

Adds a font to the collection.

```
public FontFamily Add(string path, out FontDescription description)
```

Parameters

`path` string

The filesystem path to the font file.

`description` FontDescription

The description of the added font.

Returns

FontFamily

The new FontFamily.

AddCollection(Stream)

Adds a true type font collection (.ttc).

```
public ReadOnlyMemory<FontFamily> AddCollection(Stream stream)
```

Parameters

`stream` Stream

The font stream.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(Stream, CultureInfo)

Adds a true type font collection (.ttc).

```
public ReadOnlyMemory<FontFamily> AddCollection(Stream stream, CultureInfo culture)
```

Parameters

`stream` Stream

The font stream.

`culture` CultureInfo

The culture of the fonts to add.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(Stream, CultureInfo, out ReadOnlyMemory<FontDescription>)

Adds a true type font collection (.ttc).

```
public ReadOnlyMemory<FontFamily> AddCollection(Stream stream, CultureInfo culture, out
ReadOnlyMemory<FontDescription> descriptions)
```

Parameters

`stream` Stream

The font stream.

`culture` CultureInfo

The culture of the fonts to add.

`descriptions` ReadOnlyMemory<FontDescription>

The descriptions of the added fonts.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(Stream, out ReadOnlyMemory<FontDescription>)

Adds a true type font collection (.ttc).

```
public ReadOnlyMemory<FontFamily> AddCollection(Stream stream, out
ReadOnlyMemory<FontDescription> descriptions)
```

Parameters

`stream` Stream

The font stream.

`descriptions` ReadOnlyMemory<FontDescription>

The descriptions of the added fonts.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(string)

Adds a true type font collection (.ttc).

```
public ReadOnlyMemory<FontFamily> AddCollection(string path)
```

Parameters

`path` string

The font collection path.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(string, CultureInfo)

Adds a true type font collection (.ttc).

```
public ReadOnlyMemory<FontFamily> AddCollection(string path, CultureInfo culture)
```

Parameters

`path` string

The font collection path.

`culture` CultureInfo

The culture of the fonts to add.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(string, CultureInfo, out ReadOnlyMemory<FontDescription>)

Adds a true type font collection (.ttc).

```
public ReadOnlyMemory<FontFamily> AddCollection(string path, CultureInfo culture, out
ReadOnlyMemory<FontDescription> descriptions)
```

Parameters

`path` string

The font collection path.

`culture` CultureInfo

The culture of the fonts to add.

`descriptions` ReadOnlyMemory<FontDescription>

The descriptions of the added fonts.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(string, out ReadOnlyMemory<FontDescription>)

Adds a true type font collection (.ttc).

```
public ReadOnlyMemory<FontFamily> AddCollection(string path, out
ReadOnlyMemory<FontDescription> descriptions)
```

Parameters

`path` string

The font collection path.

`descriptions` ReadOnlyMemory<FontDescription>

The descriptions of the added fonts.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddWithCulture(Stream, CultureInfo)

Adds a font to the collection.

```
public FontFamily AddWithCulture(Stream stream, CultureInfo culture)
```

Parameters

`stream` Stream

The font stream.

`culture` CultureInfo

The culture of the font to add.

Returns

FontFamily

The new FontFamily.

AddWithCulture(Stream, CultureInfo, out FontDescription)

Adds a font to the collection.

```
public FontFamily AddWithCulture(Stream stream, CultureInfo culture, out
FontDescription description)
```

Parameters

`stream` Stream

The font stream.

`culture` CultureInfo

The culture of the font to add.

`description` FontDescription

The description of the added font.

Returns

FontFamily

The new FontFamily.

AddWithCulture(string, CultureInfo)

Adds a font to the collection.

```
public FontFamily AddWithCulture(string path, CultureInfo culture)
```

Parameters

`path` string

The filesystem path to the font file.

`culture` CultureInfo

The culture of the font to add.

Returns

FontFamily

The new FontFamily.

AddWithCulture(string, CultureInfo, out FontDescription)

Adds a font to the collection.

```
public FontFamily AddWithCulture(string path, CultureInfo culture, out
FontDescription description)
```

Parameters

`path` string

The filesystem path to the font file.

`culture` CultureInfo

The culture of the font to add.

`description` FontDescription

The description of the added font.

Returns

FontFamily

The new FontFamily.

Get(string)

Gets the specified font family matching the invariant culture and font family name.

```
public FontFamily Get(string name)
```

Parameters

`name` string

The font family name.

Returns

FontFamily

The first FontFamily matching the given name.

Exceptions

ArgumentNullException

`name` is null

FontFamilyNotFoundException

The collection contains no matches.

GetByCulture(CultureInfo)

Gets the collection of FontFamily in this FontCollection using the given culture.

```
public IEnumerable<FontFamily> GetByCulture(CultureInfo culture)
```

Parameters

`culture` CultureInfo

The culture of the families to return.

Returns

IEnumerable<FontFamily>

The IEnumerable<T>.

GetByCulture(string, CultureInfo)

Gets the specified font family matching the given culture and font family name.

```
public FontFamily GetByCulture(string name, CultureInfo culture)
```

Parameters

`name` string

The font family name.

`culture` CultureInfo

The culture to use when searching for a match.

Returns

FontFamily

The first FontFamily matching the given name.

Exceptions

ArgumentNullException

`name` is null

FontFamilyNotFoundException

The collection contains no matches.

TryGet(string, out FontFamily)

Gets the specified font family matching the invariant culture and font family name.

```
public bool TryGet(string name, out FontFamily family)
```

Parameters

`name` string

The font family name.

`family` FontFamily

When this method returns, contains the family associated with the specified name, if the name is found; otherwise, the default value for the type of the family parameter. This parameter is passed uninitialized.

Returns

bool

true if the IReadOnlyFontCollection contains a family with the specified name; otherwise, false.

Exceptions

ArgumentNullException

`name` is null

TryGetByCulture(string, CultureInfo, out FontFamily)

Gets the specified font family matching the given culture and font family name.

```
public bool TryGetByCulture(string name, CultureInfo culture, out FontFamily family)
```

Parameters

`name` string

The font family name.

`culture` CultureInfo

The culture to use when searching for a match.

`family` FontFamily

When this method returns, contains the family associated with the specified name, if the name is found; otherwise, the default value for the type of the family parameter. This parameter is passed uninitialized.

Returns

bool

true if the IReadOnlyFontCollection contains a family with the specified name; otherwise, false.

Exceptions

ArgumentNullException

`name` is null

Class FontCollectionExtensions

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Extension methods for IFontCollection.

```
public static class FontCollectionExtensions
```

Inheritance

object ← FontCollectionExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

AddSystemFonts(FontCollection)

Adds the fonts from the SystemFonts collection to this FontCollection.

```
public static FontCollection AddSystemFonts(this FontCollection collection)
```

Parameters

`collection` FontCollection

The font collection.

Returns

FontCollection

The FontCollection containing the system fonts.

AddSystemFonts(FontCollection, Predicate<FontMetrics>)

Adds the fonts from the SystemFonts collection to this FontCollection.

```
public static FontCollection AddSystemFonts(this FontCollection collection,
Predicate<FontMetrics> match)
```

Parameters

`collection` FontCollection

The font collection.

`match` Predicate<FontMetrics>

The Predicate<T> delegate that defines the conditions of FontMetrics to add into the font collection.

Returns

FontCollection

The FontCollection containing the system fonts.

Class FontDescription

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Provides basic descriptive metadata for the font.

```
public class FontDescription
```

Inheritance

object ← FontDescription

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

FontFamilyInvariantCulture

Gets the name of the font family in the invariant culture.

```
public string FontFamilyInvariantCulture { get; }
```

Property Value

string

FontNameInvariantCulture

Gets the name of the font in the invariant culture.

```
public string FontNameInvariantCulture { get; }
```

Property Value

string

FontNameUpperInvariantCulture

Gets the invariant-culture font name upper-cased once at construction, shared by every per-glyph renderer parameter instance.

```
public string FontNameUpperInvariantCulture { get; }
```

Property Value

string

FontSubFamilyNameInvariantCulture

Gets the font sub family in the invariant culture.

```
public string FontSubFamilyNameInvariantCulture { get; }
```

Property Value

string

Style

Gets the style.

```
public FontStyle Style { get; }
```

Property Value

FontStyle

Weight

Gets the visual weight declared by the font.

```
public FontWeight Weight { get; }
```

Property Value

FontWeight

Methods

FontFamily(CultureInfo)

Gets the name of the font family.

```
public string FontFamily(CultureInfo culture)
```

Parameters

`culture` CultureInfo

The culture to load metadata in.

Returns

string

The font family name.

FontName(CultureInfo)

Gets the name of the font.

```
public string FontName(CultureInfo culture)
```

Parameters

`culture` CultureInfo

The culture to load metadata in.

Returns

string

The font name.

FontSubFamilyName(CultureInfo)

Gets the font sub family.

```
public string FontSubFamilyName(CultureInfo culture)
```

Parameters

`culture` CultureInfo

The culture to load metadata in.

Returns

string

The font sub family name.

GetNameById(CultureInfo, KnownNameIds)

Gets the name matching the given culture and id. If InvariantCulture is passed this method will return the first name matching the id.

```
public string GetNameById(CultureInfo culture, KnownNameIds nameId)
```

Parameters

`culture` CultureInfo

The culture to load metadata in.

`nameId` KnownNameIds

The name id to match.

Returns

string

The string name.

LoadDescription(Stream)

Reads a FontDescription from the specified stream.

```
public static FontDescription LoadDescription(Stream stream)
```

Parameters

`stream` Stream

The stream.

Returns

FontDescription

a FontDescription.

LoadDescription(string)

Reads a FontDescription from the specified stream.

```
public static FontDescription LoadDescription(string path)
```

Parameters

`path` string

The file path.

Returns

FontDescription

a FontDescription.

LoadFontCollectionDescriptions(Stream)

Reads all the FontDescriptions from the specified stream (typically a .ttc file like simsun.ttc).

```
public static ReadOnlyMemory<FontDescription> LoadFontCollectionDescriptions(Stream stream)
```

Parameters

`stream` Stream

The stream to read the font collection from.

Returns

ReadOnlyMemory<FontDescription>

A read-only memory region containing the font descriptions.

LoadFontCollectionDescriptions(string)

Reads all the FontDescriptions from the file at the specified path (typically a .ttc file like simsun.ttc).

```
public static ReadOnlyMemory<FontDescription> LoadFontCollectionDescriptions(string path)
```

Parameters

`path` string

The file path.

Returns

ReadOnlyMemory<FontDescription>

A read-only memory region containing the font descriptions.

Class FontException

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Base class for exceptions thrown by this library.

```
public class FontException : Exception, ISerializable
```

Inheritance
object ← Exception ← FontException

Implements
ISerializable

Derived
FontFamilyNotFoundException, GlyphMissingException

Inherited Members
Exception.GetBaseException() , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

FontException(string)

Initializes a new instance of the FontException class.

```
public FontException(string message)
```

Parameters

`message` string

The message that describes the error.

See Also

Exception

Struct FontFamily

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Defines a group of type faces having a similar basic design and certain variations in styles.

```
public struct FontFamily : IEquatable<FontFamily>
```

Implements
IEquatable<FontFamily>

Inherited Members
object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

Culture

Gets the culture this instance was extracted against.

```
public readonly CultureInfo Culture { get; }
```

Property Value

CultureInfo

Name

Gets the name.

```
public readonly string Name { get; }
```

Property Value

string

Methods

CreateFont(float)

Create a new instance of the Font for the named font family with regular styling.

```
public readonly Font CreateFont(float size)
```

Parameters

`size` float

The size of the font in PT units.

Returns

Font

The new Font.

CreateFont(float, FontStyle)

Create a new instance of the Font for the named font family.

```
public readonly Font CreateFont(float size, FontStyle style)
```

Parameters

`size` float

The size of the font in PT units.

`style` FontStyle

The font style.

Returns

Font

The new Font.

CreateFont(float, FontStyle, params FontVariation[])

Create a new instance of the Font for the named font family with the specified style and variation axis settings.

```
public readonly Font CreateFont(float size, FontStyle style, params
FontVariation[] variations)
```

Parameters

`size` float

The size of the font in PT units.

`style` FontStyle

The font style.

`variations` FontVariation[]

The variation axis settings to apply.

Returns

Font

The new Font.

CreateFont(float, params FontVariation[])

Create a new instance of the Font for the named font family with regular styling and the specified variation axis settings.

```
public readonly Font CreateFont(float size, params FontVariation[] variations)
```

Parameters

`size` float

The size of the font in PT units.

`variations` FontVariation[]

The variation axis settings to apply.

Returns

Font

The new Font.

Equals(FontFamily)

Indicates whether the current object is equal to another object of the same type.

```
public readonly bool Equals(FontFamily other)
```

Parameters

`other` FontFamily

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

obj object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetAvailableStyles()

Gets the collection of FontStyle that are currently available.

```
public readonly ReadOnlyMemory<FontStyle> GetAvailableStyles()
```

Returns

ReadOnlyMemory<FontStyle>

A read-only memory region containing the available font styles.

GetHashCode()

Returns the hash code for this instance.

```
public override readonly int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

ToString()

Returns the fully qualified type name of this instance.

```
public override readonly string ToString()
```

Returns

string

The fully qualified type name.

TryGetMetrics(FontStyle, out FontMetrics?)

Gets the specified font metrics matching the given font style.

```
public readonly bool TryGetMetrics(FontStyle style, out FontMetrics? metrics)
```

Parameters

`style` FontStyle

The font style to use when searching for a match.

`metrics` FontMetrics

When this method returns, contains the metrics associated with the specified name, if the name is found; otherwise, the default value for the type of the metrics parameter. This parameter is passed uninitialized.

Returns

bool

true if the FontFamily contains font metrics with the specified name; otherwise, false.

TryGetMetrics(FontStyle, FontWeight, out FontMetrics?)

Attempts to get font metrics matching the specified style and numeric weight.

```
public readonly bool TryGetMetrics(FontStyle style, FontWeight weight, out
FontMetrics? metrics)
```

Parameters

`style` FontStyle

The requested font style.

`weight` FontWeight

The requested font weight.

`metrics` FontMetrics

The matching font metrics, if found.

Returns

bool

true when the family contains a matching face; otherwise, false.

TryGetPaths(out ReadOnlyMemory<string>)

Gets the collection of filesystem paths to the font family sources.

```
public bool TryGetPaths(out ReadOnlyMemory<string> paths)
```

Parameters

`paths` ReadOnlyMemory<string>

When this method returns, contains the filesystem paths to the font family sources, if the path exists; otherwise, an empty memory region. This parameter is passed uninitialized.

Returns

bool

true if the FontFamily was created via filesystem paths; otherwise, false.

Operators

operator ==(FontFamily, FontFamily)

Compares two FontFamily objects for equality.

```
public static bool operator ==(FontFamily left, FontFamily right)
```

Parameters

`left` FontFamily

The FontFamily on the left side of the operand.

`right` FontFamily

The FontFamily on the right side of the operand.

Returns

bool

true if the current left is equal to the `right` parameter; otherwise, false.

operator !=(FontFamily, FontFamily)

Compares two FontFamily objects for inequality.

```
public static bool operator !=(FontFamily left, FontFamily right)
```

Parameters

`left` FontFamily

The FontFamily on the left side of the operand.

`right` FontFamily

The FontFamily on the right side of the operand.

Returns

bool

true if the current left is unequal to the `right` parameter; otherwise, false.

Class FontFamilyNotFoundException

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Exception for detailing missing font families.

```
public class FontFamilyNotFoundException : FontException, ISerializable
```

Inheritance

object ← Exception ← FontException ← FontFamilyNotFoundException

Implements

ISerializable

Inherited Members

Exception.GetBaseException() , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

FontFamilyNotFoundException(string)

Initializes a new instance of the FontFamilyNotFoundException class.

```
public FontFamilyNotFoundException(string family)
```

Parameters

`family` string

The name of the missing font family.

FontFamilyNotFoundException(string, IReadOnlyCollection<string>)

Initializes a new instance of the FontFamilyNotFoundException class.

```
public FontFamilyNotFoundException(string family, IReadOnlyCollection<string>
searchDirectories)
```

Parameters

`family` string

The name of the missing font family.

`searchDirectories` IReadOnlyCollection<string>

The collection of directories that were searched for the font family. Pass an empty collection if font families were not searched in directories.

Properties

FontFamily

Gets the name of the font family that was not found.

```
public string FontFamily { get; }
```

Property Value

string

SearchDirectories

Gets the collection of directories that were unsuccessfully searched for the font family.

```
public IReadOnlyCollection<string> SearchDirectories { get; }
```

Property Value

IReadOnlyCollection<string>

Remarks

If the exception did not originate from the Collection then this property will be empty.

See Also

FontException

Class FontGlyphMetrics

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a glyph metric from a particular font face.

```
public abstract class FontGlyphMetrics
```

Inheritance

object ← FontGlyphMetrics

Derived

PaintedGlyphMetrics, TrueTypeGlyphMetrics

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AdvanceHeight

Gets the advance height for vertical layout, expressed in font units.

```
public ushort AdvanceHeight { get; }
```

Property Value

ushort

AdvanceWidth

Gets the advance width for horizontal layout, expressed in font units.

```
public ushort AdvanceWidth { get; }
```

Property Value

ushort

BottomSideBearing

Gets the bottom side bearing for vertical layout, expressed in font units.

```
public short BottomSideBearing { get; }
```

Property Value

short

CodePoint

Gets the Unicode codepoint of the glyph.

```
public CodePoint CodePoint { get; }
```

Property Value

CodePoint

GlyphId

Gets the id of the glyph within the font tables.

```
public ushort GlyphId { get; }
```

Property Value

ushort

GlyphType

Gets the glyph type.

```
public GlyphType GlyphType { get; }
```

Property Value

GlyphType

Height

Gets the height, expressed in font units.

```
public float Height { get; }
```

Property Value

float

LeftSideBearing

Gets the left side bearing for horizontal layout, expressed in font units.

```
public short LeftSideBearing { get; }
```

Property Value

short

RightSideBearing

Gets the right side bearing for horizontal layout, expressed in font units.

```
public short RightSideBearing { get; }
```

Property Value

short

ScaleFactor

Gets the scale factor that is applied to all glyphs in this face. Normally calculated as 72 * UnitsPer Em so that 1pt = 1px unless the glyph has TextAttributes that apply scaling adjustment.

```
public Vector2 ScaleFactor { get; }
```

Property Value

Vector2

TextAttributes

Gets the text attributes applied to the glyph.

```
public TextAttributes TextAttributes { get; }
```

Property Value

TextAttributes

TextDecorations

Gets the text decorations applied to the glyph.

```
public TextDecorations TextDecorations { get; }
```

Property Value

TextDecorations

TopSideBearing

Gets the top side bearing for vertical layout, expressed in font units.

```
public short TopSideBearing { get; }
```

Property Value

short

UnitsPerEm

Gets the number of font units per EM square for this face.

```
public ushort UnitsPerEm { get; }
```

Property Value

ushort

Width

Gets the width, expressed in font units.

```
public float Width { get; }
```

Property Value

float

Methods

ShouldSkipGlyphRendering(CodePoint)

Gets a value indicating whether the specified code point should be skipped when rendering: line terminators participate in layout but have no glyph to draw.

```
protected static bool ShouldSkipGlyphRendering(CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The code point.

Returns

bool

The bool.

TryGetHintedAdvanceWidth(float, float, HintingMode, out float)

Attempts to compute the whole pixel advance width for the glyph under full hinting. Outline formats that grid fit under Full override this so layout can accumulate whole pixel advances matching the fitted outlines. The base implementation never applies: glyphs without fitted outlines keep their shaped fractional advances. Implementations must not execute hinting or touch outline caches, as this runs on the layout hot path.

```
public virtual bool TryGetHintedAdvanceWidth(float pointSize, float dpi, HintingMode
hintingMode, out float advancePx)
```

Parameters

`pointSize` float

The font size in pt units.

`dpi` float

The DPI (Dots Per Inch) to render/measure the glyph at

`hintingMode` HintingMode

The requested hinting mode.

`advancePx` float

The advance width in whole device pixels.

Returns

bool

true if a hinted advance applies; otherwise, false.

Struct FontMatch

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a font family and style matched for a character.

```
public readonly struct FontMatch
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

Family

Gets the matched font family.

```
public FontFamily Family { get; }
```

Property Value

FontFamily

Style

Gets the matched font style.

```
public FontStyle Style { get; }
```

Property Value

FontStyle

Class FontMetrics

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a font face with metrics, which is a set of glyphs with a specific style (regular, italic, bold etc).

```
public abstract class FontMetrics
```

Inheritance

object ← FontMetrics

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

CapHeight

Gets the cap height in font design units, or 0 when the font does not provide it.

```
public abstract short CapHeight { get; }
```

Property Value

short

Description

Gets the basic description of the face.

```
public abstract FontDescription Description { get; }
```

Property Value

FontDescription

HorizontalMetrics

Gets the metrics specific to horizontal text.

```
public abstract HorizontalMetrics HorizontalMetrics { get; }
```

Property Value

HorizontalMetrics

ItalicAngle

Gets the italic angle in counter-clockwise degrees from the vertical. Zero for upright text, negative for text that leans to the right (forward).

```
public abstract float ItalicAngle { get; }
```

Property Value

float

ScaleFactor

Gets the scale factor that is applied to all glyphs in this face. Calculated as 72 * UnitsPerEm so that 1pt = 1px.

```
public abstract float ScaleFactor { get; }
```

Property Value

float

StrikeoutPosition

Gets the position of the top of the strikeout stroke relative to the baseline in font design units.

```
public abstract short StrikeoutPosition { get; }
```

Property Value

short

StrikeoutSize

Gets thickness of the strikeout stroke in font design units.

```
public abstract short StrikeoutSize { get; }
```

Property Value

short

SubscriptXOffset

Gets the recommended horizontal offset in font design units for subscripts for this font.

```
public abstract short SubscriptXOffset { get; }
```

Property Value

short

SubscriptXSize

Gets the recommended horizontal size in font design units for subscripts for this font.

```
public abstract short SubscriptXSize { get; }
```

Property Value

short

SubscriptYOffset

Gets the recommended vertical offset in font design units for subscripts for this font.

```
public abstract short SubscriptYOffset { get; }
```

Property Value

short

SubscriptYSize

Gets the recommended vertical size in font design units for subscripts for this font.

```
public abstract short SubscriptYSize { get; }
```

Property Value

short

SuperscriptXOffset

Gets the recommended horizontal offset in font design units for superscripts for this font.

```
public abstract short SuperscriptXOffset { get; }
```

Property Value

short

SuperscriptXSize

Gets the recommended horizontal size in font design units for superscripts for this font.

```
public abstract short SuperscriptXSize { get; }
```

Property Value

short

SuperscriptYOffset

Gets the recommended vertical offset in font design units for superscripts for this font.

```
public abstract short SuperscriptYOffset { get; }
```

Property Value

short

SuperscriptYSize

Gets the recommended vertical size in font design units for superscripts for this font.

```
public abstract short SuperscriptYSize { get; }
```

Property Value

short

UnderlinePosition

Gets the suggested distance of the top of the underline from the baseline (negative values indicate below baseline).

```
public abstract short UnderlinePosition { get; }
```

Property Value

short

UnderlineThickness

Gets the suggested values for the underline thickness. In general, the underline thickness should match the thickness of the underscore character (U+005F LOW LINE), and should also match the strikeout thickness, which is specified in the OS/2 table.

```
public abstract short UnderlineThickness { get; }
```

Property Value

short

UnitsPerEm

Gets the number of font units per EM square for this face.

```
public abstract ushort UnitsPerEm { get; }
```

Property Value

ushort

VerticalMetrics

Gets the metrics specific to vertical text.

```
public abstract VerticalMetrics VerticalMetrics { get; }
```

Property Value

VerticalMetrics

XHeight

Gets the x-height in font design units, or 0 when the font does not provide it.

```
public abstract short XHeight { get; }
```

Property Value

short

Methods

GetAvailableCodePoints()

Gets the unicode codepoints for which a glyph exists in the font.

```
public abstract ReadOnlyMemory<CodePoint> GetAvailableCodePoints()
```

Returns

ReadOnlyMemory<CodePoint>

A read-only memory region containing the available codepoints.

OpenStream()

Opens a readable stream positioned at the font face data.

```
public abstract Stream OpenStream()
```

Returns

Stream

A readable stream positioned at the font face data.

TryGetGlyphMetrics(CodePoint, TextAttributes, TextDecorations, LayoutMode, ColorFontSupport, FontPalette?, out FontGlyphMetrics?)

Gets the glyph metrics for a given code point.

```
public abstract bool TryGetGlyphMetrics(CodePoint codePoint, TextAttributes textAttributes,
TextDecorations textDecorations, LayoutMode layoutMode, ColorFontSupport support,
FontPalette? palette, out FontGlyphMetrics? metrics)
```

Parameters

`codePoint` CodePoint

The Unicode code point to get the glyph for.

`textAttributes` TextAttributes

The text attributes applied to the glyph.

`textDecorations` TextDecorations

The text decorations applied to the glyph.

`layoutMode` LayoutMode

The layout mode applied to the glyph.

`support` ColorFontSupport

Options for enabling color font support during layout and rendering.

`palette` FontPalette

The color palette selection for color fonts, or null for the font's default palette.

`metrics` FontGlyphMetrics

When this method returns, contains the metrics for the given codepoint and color support if the metrics are found; otherwise the default value. This parameter is passed uninitialized.

Returns

bool

true if the face contains glyph metrics for the specified codepoint; otherwise, false.

TryGetGlyphMetrics(ushort, TextAttributes, TextDecorations, LayoutMode, ColorFontSupport, FontPalette?, out FontGlyphMetrics?)

Gets the glyph metrics for a given glyph id.

```
public abstract bool TryGetGlyphMetrics(ushort glyphId, TextAttributes textAttributes,
TextDecorations textDecorations, LayoutMode layoutMode, ColorFontSupport support,
FontPalette? palette, out FontGlyphMetrics? metrics)
```

Parameters

`glyphId` ushort

The glyph identifier.

`textAttributes` TextAttributes

The text attributes applied to the glyph.

`textDecorations` TextDecorations

The text decorations applied to the glyph.

`layoutMode` LayoutMode

The layout mode applied to the glyph.

`support` ColorFontSupport

Options for enabling color font support during layout and rendering.

`palette` FontPalette

The color palette selection for color fonts, or null for the font's default palette.

`metrics` FontGlyphMetrics

When this method returns, contains the metrics for the given glyph id and color support if the metrics are found; otherwise the default value. This parameter is passed uninitialized.

Returns

bool

true if the face contains glyph metrics for the specified glyph id; otherwise, false.

TryGetTableData(Tag, out ReadOnlyMemory<byte>)

Attempts to get the OpenType table data for the specified tag.

```
public abstract bool TryGetTableData(Tag tag, out ReadOnlyMemory<byte> table)
```

Parameters

`tag` Tag

The table tag.

`table` ReadOnlyMemory<byte>

When this method returns, contains the table data if the table exists; otherwise, the default value. This parameter is passed uninitialized.

Returns

bool

true if the table exists; otherwise, false.

TryGetVariationAxes(out ReadOnlyMemory<VariationAxis>)

Tries to get the variation axes that this font supports. The font needs to have a fvar table.

```
public abstract bool TryGetVariationAxes(out ReadOnlyMemory<VariationAxis> variationAxes)
```

Parameters

`variationAxes` ReadOnlyMemory<VariationAxis>

A read-only memory region containing the variation axes.

Returns

bool

True, if fvar table is present.

Class FontPalette

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Selects the CPAL color palette used to resolve COLR glyph colors and optionally replaces individual palette entries.

A palette index outside the range defined by the font selects the default palette (index 0), and overrides still apply, matching the CSS `font-palette` behavior. Overrides apply in order, so a later override of the same entry replaces an earlier one, and overrides whose entry index lies outside the font's palette entry range are ignored.

```
public sealed class FontPalette : IEquatable<FontPalette>
```

Inheritance

object ← FontPalette

Implements

IEquatable<FontPalette>

Inherited Members

object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

FontPalette(int)

Initializes a new instance of the FontPalette class.

```
public FontPalette(int index)
```

Parameters

`index` int

The zero-based palette index within the font's CPAL table.

FontPalette(int, IReadOnlyList<FontPaletteOverride>)

Initializes a new instance of the FontPalette class.

```
public FontPalette(int index, IReadOnlyList<FontPaletteOverride> overrides)
```

Parameters

`index` int

The zero-based palette index within the font's CPAL table.

`overrides` IReadOnlyList<FontPaletteOverride>

The override colors to apply, in order, on top of the selected palette.

Properties

Index

Gets the zero-based palette index within the font's CPAL table.

```
public int Index { get; }
```

Property Value

int

Overrides

Gets the override colors applied, in order, on top of the selected palette.

```
public IReadOnlyList<FontPaletteOverride> Overrides { get; }
```

Property Value

IReadOnlyList<FontPaletteOverride>

Methods

Equals(FontPalette?)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(FontPalette? other)
```

Parameters

`other` FontPalette

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Determines whether the specified object is equal to the current object.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current object.

Returns

bool

true if the specified object is equal to the current object; otherwise, false.

GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

Returns

int

A hash code for the current object.

Struct FontPaletteOverride

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Replaces the color of a single palette entry when a FontPalette is applied. An override whose Index lies outside the font's palette entry range is ignored.

```
public readonly struct FontPaletteOverride : IEquatable<FontPaletteOverride>
```

Implements
IEquatable<FontPaletteOverride>

Inherited Members
ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

FontPaletteOverride(int, GlyphColor)

Initializes a new instance of the FontPaletteOverride struct.

```
public FontPaletteOverride(int index, GlyphColor color)
```

Parameters

`index` int

The zero-based palette entry index to replace.

`color` GlyphColor

The replacement color.

Properties

Color

Gets the replacement color.

```
public GlyphColor Color { get; }
```

Property Value

GlyphColor

Index

Gets the zero-based palette entry index to replace.

```
public int Index { get; }
```

Property Value

int

Methods

Equals(FontPaletteOverride)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(FontPaletteOverride other)
```

Parameters

`other` FontPaletteOverride

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

Operators

operator ==(FontPaletteOverride, FontPaletteOverride)

Compares two FontPaletteOverride objects for equality.

```
public static bool operator ==(FontPaletteOverride left, FontPaletteOverride right)
```

Parameters

`left` FontPaletteOverride

The FontPaletteOverride on the left side of the operand.

`right` FontPaletteOverride

The FontPaletteOverride on the right side of the operand.

Returns

bool

True if the current left is equal to the `right` parameter; otherwise, false.

operator !=(FontPaletteOverride, FontPaletteOverride)

Compares two FontPaletteOverride objects for inequality.

```
public static bool operator !=(FontPaletteOverride left, FontPaletteOverride right)
```

Parameters

`left` FontPaletteOverride

The FontPaletteOverride on the left side of the operand.

`right` FontPaletteOverride

The FontPaletteOverride on the right side of the operand.

Returns

bool

True if the current left is unequal to the `right` parameter; otherwise, false.

Struct FontRectangle

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Stores a set of four single precision floating points that represent the location and size of a rectangle.

```
public readonly struct FontRectangle : IEquatable<FontRectangle>
```

Implements
IEquatable<FontRectangle>

Inherited Members
object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

FontRectangle(Vector2, Vector2)

Initializes a new instance of the FontRectangle struct.

```
public FontRectangle(Vector2 point, Vector2 size)
```

Parameters

`point` Vector2

The Vector2 which specifies the rectangles point in a two-dimensional plane.

`size` Vector2

The Vector2 which specifies the rectangles height and width.

FontRectangle(float, float, float, float)

Initializes a new instance of the FontRectangle struct.

```
public FontRectangle(float x, float y, float width, float height)
```

Parameters

`x` float

The horizontal position of the rectangle.

`y` float

The vertical position of the rectangle.

`width` float

The width of the rectangle.

`height` float

The height of the rectangle.

Fields

Empty

Represents a FontRectangle that has X, Y, Width, and Height values set to zero.

```
public static readonly FontRectangle Empty
```

Field Value

FontRectangle

Properties

Bottom

Gets the y-coordinate of the bottom edge of this FontRectangle.

```
public float Bottom { get; }
```

Property Value

float

Height

Gets the height of this FontRectangle.

```
public float Height { get; }
```

Property Value

float

Left

Gets the x-coordinate of the left edge of this FontRectangle.

```
public float Left { get; }
```

Property Value

float

Right

Gets the x-coordinate of the right edge of this FontRectangle.

```
public float Right { get; }
```

Property Value

float

Top

Gets the y-coordinate of the top edge of this FontRectangle.

```
public float Top { get; }
```

Property Value

float

Width

Gets the width of this FontRectangle.

```
public float Width { get; }
```

Property Value

float

X

Gets the x-coordinate of this FontRectangle.

```
public float X { get; }
```

Property Value

float

Y

Gets the y-coordinate of this FontRectangle.

```
public float Y { get; }
```

Property Value

float

Methods

Center(in FontRectangle)

Returns the center point of the given FontRectangle.

```
public static Vector2 Center(in FontRectangle rectangle)
```

Parameters

`rectangle` FontRectangle

The rectangle.

Returns

Vector2

The Vector2.

Contains(in FontRectangle)

Determines if the rectangular region represented by `rectangle` is entirely contained within the rectangular region represented by this FontRectangle .

```
public bool Contains(in FontRectangle rectangle)
```

Parameters

`rectangle` FontRectangle

The rectangle.

Returns

bool

The bool.

Contains(Vector2)

Determines if the specified point is contained within the rectangular region defined by this Font Rectangle .

```
public bool Contains(Vector2 point)
```

Parameters

`point` Vector2

The point.

Returns

bool

The bool.

Contains(float, float)

Determines if the specified point is contained within the rectangular region defined by this Font Rectangle.

```
public bool Contains(float x, float y)
```

Parameters

`x` float

The x-coordinate of the given point.

`y` float

The y-coordinate of the given point.

Returns

bool

The bool.

Deconstruct(out float, out float, out float, out float)

Deconstructs this rectangle into four floats.

```
public void Deconstruct(out float x, out float y, out float width, out float height)
```

Parameters

`x` float

The out value for X.

`y` float

The out value for Y.

`width` float

The out value for the width.

`height` float

The out value for the height.

Equals(FontRectangle)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(FontRectangle other)
```

Parameters

`other` FontRectangle

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

FromLTRB(float, float, float, float)

Creates a new FontRectangle with the specified location and size.

```
public static FontRectangle FromLTRB(float left, float top, float right, float bottom)
```

Parameters

`left` float

The left coordinate of the rectangle.

`top` float

The top coordinate of the rectangle.

`right` float

The right coordinate of the rectangle.

`bottom` float

The bottom coordinate of the rectangle.

Returns

FontRectangle

The FontRectangle.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

Inflate(in FontRectangle, float, float)

Creates a new FontRectangle from the given `rectangle` that is inflated by the specified amount.

```
public static FontRectangle Inflate(in FontRectangle rectangle, float x, float y)
```

Parameters

`rectangle` FontRectangle

The rectangle.

`x` float

The amount to inflate the width by.

`y` float

The amount to inflate the height by.

Returns

FontRectangle

A new FontRectangle.

Inflate(Vector2)

Creates a new FontRectangle inflated by the specified amount.

```
public FontRectangle Inflate(Vector2 size)
```

Parameters

`size` Vector2

The size.

Returns

FontRectangle

New FontRectangle representing the inflated rectangle

Inflate(float, float)

Creates a new FontRectangle inflated by the specified amount.

```
public FontRectangle Inflate(float width, float height)
```

Parameters

`width` float

The width.

`height` float

The height.

Returns

FontRectangle

New FontRectangle representing the inflated rectangle

Intersect(in FontRectangle)

Creates a FontRectangle that represents the intersection between this FontRectangle and the `rectangle`.

```
public FontRectangle Intersect(in FontRectangle rectangle)
```

Parameters

`rectangle` FontRectangle

The rectangle.

Returns

FontRectangle

New FontRectangle representing the intersections between the two rectangles.

Intersect(in FontRectangle, in FontRectangle)

Creates a rectangle that represents the intersection between `a` and `b`. If there is no intersection, an empty rectangle is returned.

```
public static FontRectangle Intersect(in FontRectangle a, in FontRectangle b)
```

Parameters

`a` FontRectangle

The first rectangle.

`b` FontRectangle

The second rectangle.

Returns

FontRectangle

The FontRectangle.

IntersectsWith(in FontRectangle)

Determines if the specified FontRectangle intersects the rectangular region defined by this FontRectangle.

```
public bool IntersectsWith(in FontRectangle rectangle)
```

Parameters

`rectangle` FontRectangle

The other rectangle.

Returns

bool

The bool.

Offset(Vector2)

Adjusts the location of this rectangle by the specified amount.

```
public FontRectangle Offset(Vector2 point)
```

Parameters

`point` Vector2

The point.

Returns

FontRectangle

New FontRectangle representing the offset rectangle.

Offset(float, float)

Adjusts the location of this rectangle by the specified amount.

```
public FontRectangle Offset(float dx, float dy)
```

Parameters

`dx` float

The amount to offset the x-coordinate.

`dy` float

The amount to offset the y-coordinate.

Returns

FontRectangle

New FontRectangle representing the inflated rectangle.

ToString()

Returns the fully qualified type name of this instance.

```
public override string ToString()
```

Returns

string

The fully qualified type name.

Transform(in FontRectangle, Matrix3×2)

Creates a new FontRectangle by transforming the given rectangle by the given matrix.

```
public static FontRectangle Transform(in FontRectangle rectangle, Matrix3x2 matrix)
```

Parameters

`rectangle` FontRectangle

The source rectangle.

`matrix` Matrix3×2

The transformation matrix.

Returns

FontRectangle

A transformed FontRectangle.

Union(in FontRectangle, in FontRectangle)

Creates a rectangle that represents the union between `a` and `b`.

```
public static FontRectangle Union(in FontRectangle a, in FontRectangle b)
```

Parameters

`a` FontRectangle

The first rectangle.

`b` FontRectangle

The second rectangle.

Returns

FontRectangle

The FontRectangle.

Operators

operator ==(in FontRectangle, in FontRectangle)

Compares two FontRectangle objects for equality.

```
public static bool operator ==(in FontRectangle left, in FontRectangle right)
```

Parameters

`left` FontRectangle

The FontRectangle on the left side of the operand.

`right` FontRectangle

The FontRectangle on the right side of the operand.

Returns

bool

True if the current left is equal to the `right` parameter; otherwise, false.

operator !=(in FontRectangle, in FontRectangle)

Compares two FontRectangle objects for inequality.

```
public static bool operator !=(in FontRectangle left, in FontRectangle right)
```

Parameters

`left` FontRectangle

The FontRectangle on the left side of the operand.

`right` FontRectangle

The FontRectangle on the right side of the operand.

Returns

bool

True if the current left is unequal to the `right` parameter; otherwise, false.

Enum FontStyle

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

The font styles

```
[Flags]
public enum FontStyle
```

Fields

`Bold = 1`

Bold

`BoldItalic = Bold | Italic`

Bold and Italic

`Italic = 2`

Italic

`Regular = 0`

Regular

Struct FontVariation

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a single variation axis setting for a variable font, consisting of a four-character tag and a value.

```
public readonly struct FontVariation
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Remarks

Follows CSS `font-variation-settings` semantics. Values are clamped to the axis range defined in the font's `fvar` table.

Constructors

FontVariation(Tag, float)

Initializes a new instance of the FontVariation struct.

```
public FontVariation(Tag tag, float value)
```

Parameters

`tag` Tag

The four-byte OpenType axis tag.

`value` float

The axis value in design-space units.

FontVariation(string, float)

Initializes a new instance of the FontVariation struct.

```
public FontVariation(string tag, float value)
```

Parameters

`tag` string

The four-character axis tag (e.g. "wght", "wdth", "opsz").

`value` float

The axis value in design-space units.

Properties

Tag

Gets the four-character axis tag identifying the design variation.

```
public Tag Tag { get; }
```

Property Value

Tag

Value

Gets the axis value in design-space units.

```
public float Value { get; }
```

Property Value

float

Enum FontWeight

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Identifies the visual weight of a font on the OpenType and CSS weight scale.

```
public enum FontWeight
```

Fields

`Black = 900`

Black weight (900).

`Bold = 700`

Bold weight (700).

`ExtraBold = 800`

Extra-bold weight (800).

`ExtraLight = 200`

Extra-light weight (200).

`Light = 300`

Light weight (300).

`Medium = 500`

Medium weight (500).

`Normal = 400`

Normal weight (400).

`SemiBold = 600`

Semi-bold weight (600).

```
Thin = 100
```

Thin weight (100).

Struct Glyph

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a font-specific glyph at the point size used for layout and rendering.

```
public readonly struct Glyph
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

GlyphMetrics

Gets the font metrics for this glyph.

```
public FontGlyphMetrics GlyphMetrics { get; }
```

Property Value

FontGlyphMetrics

Methods

BoundingBox(GlyphLayoutMode, Vector2, float)

Calculates the rendered glyph bounds for the specified layout mode and origin.

```
public FontRectangle BoundingBox(GlyphLayoutMode mode, Vector2 glyphOrigin, float dpi)
```

Parameters

`mode` GlyphLayoutMode

The glyph layout mode to measure with.

`glyphOrigin` Vector2

The glyph origin to calculate the bounds from.

`dpi` float

The DPI to measure the glyph at.

Returns

FontRectangle

The rendered glyph bounds.

Struct GlyphColor

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Provides access to the color details for the current glyph.

```
public readonly struct GlyphColor : IEquatable<GlyphColor>
```

Implements
IEquatable<GlyphColor>

Inherited Members
ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

GlyphColor(byte, byte, byte, byte)

Initializes a new instance of the GlyphColor struct.

```
public GlyphColor(byte red, byte green, byte blue, byte alpha)
```

Parameters

`red` byte

The red component.

`green` byte

The green component.

`blue` byte

The blue component.

`alpha` byte

The alpha component.

Fields

AliceBlue

Represents a matching the W3C definition that has an hex value of #F0F8FF.

```
public static readonly GlyphColor AliceBlue
```

Field Value

GlyphColor

AntiqueWhite

Represents a matching the W3C definition that has an hex value of #FAEBD7.

```
public static readonly GlyphColor AntiqueWhite
```

Field Value

GlyphColor

Aqua

Represents a matching the W3C definition that has an hex value of #00FFFF.

```
public static readonly GlyphColor Aqua
```

Field Value

GlyphColor

Aquamarine

Represents a matching the W3C definition that has an hex value of #7FFFD4.

```
public static readonly GlyphColor Aquamarine
```

Field Value

GlyphColor

Azure

Represents a matching the W3C definition that has an hex value of #F0FFFF.

```
public static readonly GlyphColor Azure
```

Field Value

GlyphColor

Beige

Represents a matching the W3C definition that has an hex value of #F5F5DC.

```
public static readonly GlyphColor Beige
```

Field Value

GlyphColor

Bisque

Represents a matching the W3C definition that has an hex value of #FFE4C4.

```
public static readonly GlyphColor Bisque
```

Field Value

GlyphColor

Black

Represents a matching the W3C definition that has an hex value of #000000.

```
public static readonly GlyphColor Black
```

Field Value

GlyphColor

BlanchedAlmond

Represents a matching the W3C definition that has an hex value of #FFEBCD.

```
public static readonly GlyphColor BlanchedAlmond
```

Field Value

GlyphColor

Blue

Represents a matching the W3C definition that has an hex value of #0000FF.

```
public static readonly GlyphColor Blue
```

Field Value

GlyphColor

BlueViolet

Represents a matching the W3C definition that has an hex value of #8A2BE2.

```
public static readonly GlyphColor BlueViolet
```

Field Value

GlyphColor

Brown

Represents a matching the W3C definition that has an hex value of #A52A2A.

```
public static readonly GlyphColor Brown
```

Field Value

GlyphColor

BurlyWood

Represents a matching the W3C definition that has an hex value of #DEB887.

```
public static readonly GlyphColor BurlyWood
```

Field Value

GlyphColor

CadetBlue

Represents a matching the W3C definition that has an hex value of #5F9EA0.

```
public static readonly GlyphColor CadetBlue
```

Field Value

GlyphColor

Chartreuse

Represents a matching the W3C definition that has an hex value of #7FFF00.

```
public static readonly GlyphColor Chartreuse
```

Field Value

GlyphColor

Chocolate

Represents a matching the W3C definition that has an hex value of #D2691E.

```
public static readonly GlyphColor Chocolate
```

Field Value

GlyphColor

Coral

Represents a matching the W3C definition that has an hex value of #FF7F50.

```
public static readonly GlyphColor Coral
```

Field Value

GlyphColor

CornflowerBlue

Represents a matching the W3C definition that has an hex value of #6495ED.

```
public static readonly GlyphColor CornflowerBlue
```

Field Value

GlyphColor

Cornsilk

Represents a matching the W3C definition that has an hex value of #FFF8DC.

```
public static readonly GlyphColor Cornsilk
```

Field Value

GlyphColor

Crimson

Represents a matching the W3C definition that has an hex value of #DC143C.

```
public static readonly GlyphColor Crimson
```

Field Value

GlyphColor

Cyan

Represents a matching the W3C definition that has an hex value of #00FFFF.

```
public static readonly GlyphColor Cyan
```

Field Value

GlyphColor

DarkBlue

Represents a matching the W3C definition that has an hex value of #00008B.

```
public static readonly GlyphColor DarkBlue
```

Field Value

GlyphColor

DarkCyan

Represents a matching the W3C definition that has an hex value of #008B8B.

```
public static readonly GlyphColor DarkCyan
```

Field Value

GlyphColor

DarkGoldenrod

Represents a matching the W3C definition that has an hex value of #B8860B.

```
public static readonly GlyphColor DarkGoldenrod
```

Field Value

GlyphColor

DarkGray

Represents a matching the W3C definition that has an hex value of #A9A9A9.

```
public static readonly GlyphColor DarkGray
```

Field Value

GlyphColor

DarkGreen

Represents a matching the W3C definition that has an hex value of #006400.

```
public static readonly GlyphColor DarkGreen
```

Field Value

GlyphColor

DarkGrey

Represents a matching the W3C definition that has an hex value of #A9A9A9.

```
public static readonly GlyphColor DarkGrey
```

Field Value

GlyphColor

DarkKhaki

Represents a matching the W3C definition that has an hex value of #BDB76B.

```
public static readonly GlyphColor DarkKhaki
```

Field Value

GlyphColor

DarkMagenta

Represents a matching the W3C definition that has an hex value of #8B008B.

```
public static readonly GlyphColor DarkMagenta
```

Field Value

GlyphColor

DarkOliveGreen

Represents a matching the W3C definition that has an hex value of #556B2F.

```
public static readonly GlyphColor DarkOliveGreen
```

Field Value

GlyphColor

DarkOrange

Represents a matching the W3C definition that has an hex value of #FF8C00.

```
public static readonly GlyphColor DarkOrange
```

Field Value

GlyphColor

DarkOrchid

Represents a matching the W3C definition that has an hex value of #9932CC.

```
public static readonly GlyphColor DarkOrchid
```

Field Value

GlyphColor

DarkRed

Represents a matching the W3C definition that has an hex value of #8B0000.

```
public static readonly GlyphColor DarkRed
```

Field Value

GlyphColor

DarkSalmon

Represents a matching the W3C definition that has an hex value of #E9967A.

```
public static readonly GlyphColor DarkSalmon
```

Field Value

GlyphColor

DarkSeaGreen

Represents a matching the W3C definition that has an hex value of #8FBC8F.

```
public static readonly GlyphColor DarkSeaGreen
```

Field Value

GlyphColor

DarkSlateBlue

Represents a matching the W3C definition that has an hex value of #483D8B.

```
public static readonly GlyphColor DarkSlateBlue
```

Field Value

GlyphColor

DarkSlateGray

Represents a matching the W3C definition that has an hex value of #2F4F4F.

```
public static readonly GlyphColor DarkSlateGray
```

Field Value

GlyphColor

DarkSlateGrey

Represents a matching the W3C definition that has an hex value of #2F4F4F.

```
public static readonly GlyphColor DarkSlateGrey
```

Field Value

GlyphColor

DarkTurquoise

Represents a matching the W3C definition that has an hex value of #00CED1.

```
public static readonly GlyphColor DarkTurquoise
```

Field Value

GlyphColor

DarkViolet

Represents a matching the W3C definition that has an hex value of #9400D3.

```
public static readonly GlyphColor DarkViolet
```

Field Value

GlyphColor

DeepPink

Represents a matching the W3C definition that has an hex value of #FF1493.

```
public static readonly GlyphColor DeepPink
```

Field Value

GlyphColor

DeepSkyBlue

Represents a matching the W3C definition that has an hex value of #00BFFF.

```
public static readonly GlyphColor DeepSkyBlue
```

Field Value

GlyphColor

DimGray

Represents a matching the W3C definition that has an hex value of #696969.

```
public static readonly GlyphColor DimGray
```

Field Value

GlyphColor

DimGrey

Represents a matching the W3C definition that has an hex value of #696969.

```
public static readonly GlyphColor DimGrey
```

Field Value

GlyphColor

DodgerBlue

Represents a matching the W3C definition that has an hex value of #1E90FF.

```
public static readonly GlyphColor DodgerBlue
```

Field Value

GlyphColor

Firebrick

Represents a matching the W3C definition that has an hex value of #B22222.

```
public static readonly GlyphColor Firebrick
```

Field Value

GlyphColor

FloralWhite

Represents a matching the W3C definition that has an hex value of #FFFAF0.

```
public static readonly GlyphColor FloralWhite
```

Field Value

GlyphColor

ForestGreen

Represents a matching the W3C definition that has an hex value of #228B22.

```
public static readonly GlyphColor ForestGreen
```

Field Value

GlyphColor

Fuchsia

Represents a matching the W3C definition that has an hex value of #FF00FF.

```
public static readonly GlyphColor Fuchsia
```

Field Value

GlyphColor

Gainsboro

Represents a matching the W3C definition that has an hex value of #DCDCDC.

```
public static readonly GlyphColor Gainsboro
```

Field Value

GlyphColor

GhostWhite

Represents a matching the W3C definition that has an hex value of #F8F8FF.

```
public static readonly GlyphColor GhostWhite
```

Field Value

GlyphColor

Gold

Represents a matching the W3C definition that has an hex value of #FFD700.

```
public static readonly GlyphColor Gold
```

Field Value

GlyphColor

Goldenrod

Represents a matching the W3C definition that has an hex value of #DAA520.

```
public static readonly GlyphColor Goldenrod
```

Field Value

GlyphColor

Gray

Represents a matching the W3C definition that has an hex value of #808080.

```
public static readonly GlyphColor Gray
```

Field Value

GlyphColor

Green

Represents a matching the W3C definition that has an hex value of #008000.

```
public static readonly GlyphColor Green
```

Field Value

GlyphColor

GreenYellow

Represents a matching the W3C definition that has an hex value of #ADFF2F.

```
public static readonly GlyphColor GreenYellow
```

Field Value

GlyphColor

Grey

Represents a matching the W3C definition that has an hex value of #808080.

```
public static readonly GlyphColor Grey
```

Field Value

GlyphColor

Honeydew

Represents a matching the W3C definition that has an hex value of #F0FFF0.

```
public static readonly GlyphColor Honeydew
```

Field Value

GlyphColor

HotPink

Represents a matching the W3C definition that has an hex value of #FF69B4.

```
public static readonly GlyphColor HotPink
```

Field Value

GlyphColor

IndianRed

Represents a matching the W3C definition that has an hex value of #CD5C5C.

```
public static readonly GlyphColor IndianRed
```

Field Value

GlyphColor

Indigo

Represents a matching the W3C definition that has an hex value of #4B0082.

```
public static readonly GlyphColor Indigo
```

Field Value

GlyphColor

Ivory

Represents a matching the W3C definition that has an hex value of #FFFFF0.

```
public static readonly GlyphColor Ivory
```

Field Value

GlyphColor

Khaki

Represents a matching the W3C definition that has an hex value of #F0E68C.

```
public static readonly GlyphColor Khaki
```

Field Value

GlyphColor

Lavender

Represents a matching the W3C definition that has an hex value of #E6E6FA.

```
public static readonly GlyphColor Lavender
```

Field Value

GlyphColor

LavenderBlush

Represents a matching the W3C definition that has an hex value of #FFF0F5.

```
public static readonly GlyphColor LavenderBlush
```

Field Value

GlyphColor

LawnGreen

Represents a matching the W3C definition that has an hex value of #7CFC00.

```
public static readonly GlyphColor LawnGreen
```

Field Value

GlyphColor

LemonChiffon

Represents a matching the W3C definition that has an hex value of #FFFACD.

```
public static readonly GlyphColor LemonChiffon
```

Field Value

GlyphColor

LightBlue

Represents a matching the W3C definition that has an hex value of #ADD8E6.

```
public static readonly GlyphColor LightBlue
```

Field Value

GlyphColor

LightCoral

Represents a matching the W3C definition that has an hex value of #F08080.

```
public static readonly GlyphColor LightCoral
```

Field Value

GlyphColor

LightCyan

Represents a matching the W3C definition that has an hex value of #E0FFFF.

```
public static readonly GlyphColor LightCyan
```

Field Value

GlyphColor

LightGoldenrodYellow

Represents a matching the W3C definition that has an hex value of #FAFAD2.

```
public static readonly GlyphColor LightGoldenrodYellow
```

Field Value

GlyphColor

LightGray

Represents a matching the W3C definition that has an hex value of #D3D3D3.

```
public static readonly GlyphColor LightGray
```

Field Value

GlyphColor

LightGreen

Represents a matching the W3C definition that has an hex value of #90EE90.

```
public static readonly GlyphColor LightGreen
```

Field Value

GlyphColor

LightGrey

Represents a matching the W3C definition that has an hex value of #D3D3D3.

```
public static readonly GlyphColor LightGrey
```

Field Value

GlyphColor

LightPink

Represents a matching the W3C definition that has an hex value of #FFB6C1.

```
public static readonly GlyphColor LightPink
```

Field Value

GlyphColor

LightSalmon

Represents a matching the W3C definition that has an hex value of #FFA07A.

```
public static readonly GlyphColor LightSalmon
```

Field Value

GlyphColor

LightSeaGreen

Represents a matching the W3C definition that has an hex value of #20B2AA.

```
public static readonly GlyphColor LightSeaGreen
```

Field Value

GlyphColor

LightSkyBlue

Represents a matching the W3C definition that has an hex value of #87CEFA.

```
public static readonly GlyphColor LightSkyBlue
```

Field Value

GlyphColor

LightSlateGray

Represents a matching the W3C definition that has an hex value of #778899.

```
public static readonly GlyphColor LightSlateGray
```

Field Value

GlyphColor

LightSlateGrey

Represents a matching the W3C definition that has an hex value of #778899.

```
public static readonly GlyphColor LightSlateGrey
```

Field Value

GlyphColor

LightSteelBlue

Represents a matching the W3C definition that has an hex value of #B0C4DE.

```
public static readonly GlyphColor LightSteelBlue
```

Field Value

GlyphColor

LightYellow

Represents a matching the W3C definition that has an hex value of #FFFFE0.

```
public static readonly GlyphColor LightYellow
```

Field Value

GlyphColor

Lime

Represents a matching the W3C definition that has an hex value of #00FF00.

```
public static readonly GlyphColor Lime
```

Field Value

GlyphColor

LimeGreen

Represents a matching the W3C definition that has an hex value of #32CD32.

```
public static readonly GlyphColor LimeGreen
```

Field Value

GlyphColor

Linen

Represents a matching the W3C definition that has an hex value of #FAF0E6.

```
public static readonly GlyphColor Linen
```

Field Value

GlyphColor

Magenta

Represents a matching the W3C definition that has an hex value of #FF00FF.

```
public static readonly GlyphColor Magenta
```

Field Value

GlyphColor

Maroon

Represents a matching the W3C definition that has an hex value of #800000.

```
public static readonly GlyphColor Maroon
```

Field Value

GlyphColor

MediumAquamarine

Represents a matching the W3C definition that has an hex value of #66CDAA.

```
public static readonly GlyphColor MediumAquamarine
```

Field Value

GlyphColor

MediumBlue

Represents a matching the W3C definition that has an hex value of #0000CD.

```
public static readonly GlyphColor MediumBlue
```

Field Value

GlyphColor

MediumOrchid

Represents a matching the W3C definition that has an hex value of #BA55D3.

```
public static readonly GlyphColor MediumOrchid
```

Field Value

GlyphColor

MediumPurple

Represents a matching the W3C definition that has an hex value of #9370DB.

```
public static readonly GlyphColor MediumPurple
```

Field Value

GlyphColor

MediumSeaGreen

Represents a matching the W3C definition that has an hex value of #3CB371.

```
public static readonly GlyphColor MediumSeaGreen
```

Field Value

GlyphColor

MediumSlateBlue

Represents a matching the W3C definition that has an hex value of #7B68EE.

```
public static readonly GlyphColor MediumSlateBlue
```

Field Value

GlyphColor

MediumSpringGreen

Represents a matching the W3C definition that has an hex value of #00FA9A.

```
public static readonly GlyphColor MediumSpringGreen
```

Field Value

GlyphColor

MediumTurquoise

Represents a matching the W3C definition that has an hex value of #48D1CC.

```
public static readonly GlyphColor MediumTurquoise
```

Field Value

GlyphColor

MediumVioletRed

Represents a matching the W3C definition that has an hex value of #C71585.

```
public static readonly GlyphColor MediumVioletRed
```

Field Value

GlyphColor

MidnightBlue

Represents a matching the W3C definition that has an hex value of #191970.

```
public static readonly GlyphColor MidnightBlue
```

Field Value

GlyphColor

MintCream

Represents a matching the W3C definition that has an hex value of #F5FFFA.

```
public static readonly GlyphColor MintCream
```

Field Value

GlyphColor

MistyRose

Represents a matching the W3C definition that has an hex value of #FFE4E1.

```
public static readonly GlyphColor MistyRose
```

Field Value

GlyphColor

Moccasin

Represents a matching the W3C definition that has an hex value of #FFE4B5.

```
public static readonly GlyphColor Moccasin
```

Field Value

GlyphColor

NavajoWhite

Represents a matching the W3C definition that has an hex value of #FFDEAD.

```
public static readonly GlyphColor NavajoWhite
```

Field Value

GlyphColor

Navy

Represents a matching the W3C definition that has an hex value of #000080.

```
public static readonly GlyphColor Navy
```

Field Value

GlyphColor

OldLace

Represents a matching the W3C definition that has an hex value of #FDF5E6.

```
public static readonly GlyphColor OldLace
```

Field Value

GlyphColor

Olive

Represents a matching the W3C definition that has an hex value of #808000.

```
public static readonly GlyphColor Olive
```

Field Value

GlyphColor

OliveDrab

Represents a matching the W3C definition that has an hex value of #6B8E23.

```
public static readonly GlyphColor OliveDrab
```

Field Value

GlyphColor

Orange

Represents a matching the W3C definition that has an hex value of #FFA500.

```
public static readonly GlyphColor Orange
```

Field Value

GlyphColor

OrangeRed

Represents a matching the W3C definition that has an hex value of #FF4500.

```
public static readonly GlyphColor OrangeRed
```

Field Value

GlyphColor

Orchid

Represents a matching the W3C definition that has an hex value of #DA70D6.

```
public static readonly GlyphColor Orchid
```

Field Value

GlyphColor

PaleGoldenrod

Represents a matching the W3C definition that has an hex value of #EEE8AA.

```
public static readonly GlyphColor PaleGoldenrod
```

Field Value

GlyphColor

PaleGreen

Represents a matching the W3C definition that has an hex value of #98FB98.

```
public static readonly GlyphColor PaleGreen
```

Field Value

GlyphColor

PaleTurquoise

Represents a matching the W3C definition that has an hex value of #AFEEEE.

```
public static readonly GlyphColor PaleTurquoise
```

Field Value

GlyphColor

PaleVioletRed

Represents a matching the W3C definition that has an hex value of #DB7093.

```
public static readonly GlyphColor PaleVioletRed
```

Field Value

GlyphColor

PapayaWhip

Represents a matching the W3C definition that has an hex value of #FFEFD5.

```
public static readonly GlyphColor PapayaWhip
```

Field Value

GlyphColor

PeachPuff

Represents a matching the W3C definition that has an hex value of #FFDAB9.

```
public static readonly GlyphColor PeachPuff
```

Field Value

GlyphColor

Peru

Represents a matching the W3C definition that has an hex value of #CD853F.

```
public static readonly GlyphColor Peru
```

Field Value

GlyphColor

Pink

Represents a matching the W3C definition that has an hex value of #FFC0CB.

```
public static readonly GlyphColor Pink
```

Field Value

GlyphColor

Plum

Represents a matching the W3C definition that has an hex value of #DDA0DD.

```
public static readonly GlyphColor Plum
```

Field Value

GlyphColor

PowderBlue

Represents a matching the W3C definition that has an hex value of #B0E0E6.

```
public static readonly GlyphColor PowderBlue
```

Field Value

GlyphColor

Purple

Represents a matching the W3C definition that has an hex value of #800080.

```
public static readonly GlyphColor Purple
```

Field Value

GlyphColor

RebeccaPurple

Represents a matching the W3C definition that has an hex value of #663399.

```
public static readonly GlyphColor RebeccaPurple
```

Field Value

GlyphColor

Red

Represents a matching the W3C definition that has an hex value of #FF0000.

```
public static readonly GlyphColor Red
```

Field Value

GlyphColor

RosyBrown

Represents a matching the W3C definition that has an hex value of #BC8F8F.

```
public static readonly GlyphColor RosyBrown
```

Field Value

GlyphColor

RoyalBlue

Represents a matching the W3C definition that has an hex value of #4169E1.

```
public static readonly GlyphColor RoyalBlue
```

Field Value

GlyphColor

SaddleBrown

Represents a matching the W3C definition that has an hex value of #8B4513.

```
public static readonly GlyphColor SaddleBrown
```

Field Value

GlyphColor

Salmon

Represents a matching the W3C definition that has an hex value of #FA8072.

```
public static readonly GlyphColor Salmon
```

Field Value

GlyphColor

SandyBrown

Represents a matching the W3C definition that has an hex value of #F4A460.

```
public static readonly GlyphColor SandyBrown
```

Field Value

GlyphColor

SeaGreen

Represents a matching the W3C definition that has an hex value of #2E8B57.

```
public static readonly GlyphColor SeaGreen
```

Field Value

GlyphColor

SeaShell

Represents a matching the W3C definition that has an hex value of #FFF5EE.

```
public static readonly GlyphColor SeaShell
```

Field Value

GlyphColor

Sienna

Represents a matching the W3C definition that has an hex value of #A0522D.

```
public static readonly GlyphColor Sienna
```

Field Value

GlyphColor

Silver

Represents a matching the W3C definition that has an hex value of #C0C0C0.

```
public static readonly GlyphColor Silver
```

Field Value

GlyphColor

SkyBlue

Represents a matching the W3C definition that has an hex value of #87CEEB.

```
public static readonly GlyphColor SkyBlue
```

Field Value

GlyphColor

SlateBlue

Represents a matching the W3C definition that has an hex value of #6A5ACD.

```
public static readonly GlyphColor SlateBlue
```

Field Value

GlyphColor

SlateGray

Represents a matching the W3C definition that has an hex value of #708090.

```
public static readonly GlyphColor SlateGray
```

Field Value

GlyphColor

SlateGrey

Represents a matching the W3C definition that has an hex value of #708090.

```
public static readonly GlyphColor SlateGrey
```

Field Value

GlyphColor

Snow

Represents a matching the W3C definition that has an hex value of #FFFAFA.

```
public static readonly GlyphColor Snow
```

Field Value

GlyphColor

SpringGreen

Represents a matching the W3C definition that has an hex value of #00FF7F.

```
public static readonly GlyphColor SpringGreen
```

Field Value

GlyphColor

SteelBlue

Represents a matching the W3C definition that has an hex value of #4682B4.

```
public static readonly GlyphColor SteelBlue
```

Field Value

GlyphColor

Tan

Represents a matching the W3C definition that has an hex value of #D2B48C.

```
public static readonly GlyphColor Tan
```

Field Value

GlyphColor

Teal

Represents a matching the W3C definition that has an hex value of #008080.

```
public static readonly GlyphColor Teal
```

Field Value

GlyphColor

Thistle

Represents a matching the W3C definition that has an hex value of #D8BFD8.

```
public static readonly GlyphColor Thistle
```

Field Value

GlyphColor

Tomato

Represents a matching the W3C definition that has an hex value of #FF6347.

```
public static readonly GlyphColor Tomato
```

Field Value

GlyphColor

Transparent

Represents a matching the W3C definition that has an hex value of #00000000.

```
public static readonly GlyphColor Transparent
```

Field Value

GlyphColor

Turquoise

Represents a matching the W3C definition that has an hex value of #40E0D0.

```
public static readonly GlyphColor Turquoise
```

Field Value

GlyphColor

Violet

Represents a matching the W3C definition that has an hex value of #EE82EE.

```
public static readonly GlyphColor Violet
```

Field Value

GlyphColor

Wheat

Represents a matching the W3C definition that has an hex value of #F5DEB3.

```
public static readonly GlyphColor Wheat
```

Field Value

GlyphColor

White

Represents a matching the W3C definition that has an hex value of #FFFFFF.

```
public static readonly GlyphColor White
```

Field Value

GlyphColor

WhiteSmoke

Represents a matching the W3C definition that has an hex value of #F5F5F5.

```
public static readonly GlyphColor WhiteSmoke
```

Field Value

GlyphColor

Yellow

Represents a matching the W3C definition that has an hex value of #FFFF00.

```
public static readonly GlyphColor Yellow
```

Field Value

GlyphColor

YellowGreen

Represents a matching the W3C definition that has an hex value of #9ACD32.

```
public static readonly GlyphColor YellowGreen
```

Field Value

GlyphColor

Properties

A

Gets the alpha component

```
public byte A { get; }
```

Property Value

byte

B

Gets the blue component

```
public byte B { get; }
```

Property Value

byte

G

Gets the green component

```
public byte G { get; }
```

Property Value

byte

R

Gets the red component

```
public byte R { get; }
```

Property Value

byte

Methods

Equals(GlyphColor)

Compares the GlyphColor for equality to this color.

```
public bool Equals(GlyphColor other)
```

Parameters

`other` GlyphColor

The other GlyphColor to compare to.

Returns

bool

True if the current color is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

TryParseHex(string?, out GlyphColor)

Gets the hexadecimal string representation of the color instance in the format RRGGBBAA.

```
public static bool TryParseHex(string? value, out GlyphColor result)
```

Parameters

`value` string

The hexadecimal representation of the combined color components.

`result` GlyphColor

When this method returns, contains the GlyphColor equivalent of the hexadecimal input.

Returns

bool

true if the parsing was successful; otherwise, false.

TryParseNamed(string?, out GlyphColor)

Attempts to parse the specified name into a corresponding named glyph color.

```
public static bool TryParseNamed(string? name, out GlyphColor result)
```

Parameters

`name` string

The name of the glyph color to parse.

`result` GlyphColor

When this method returns, contains the parsed GlyphColor value if the parse operation succeeded; otherwise, contains the default value.

Returns

bool

true if the parsing was successful; otherwise, false.

Operators

operator ==(GlyphColor, GlyphColor)

Compares two GlyphColor objects for equality.

```
public static bool operator ==(GlyphColor left, GlyphColor right)
```

Parameters

`left` GlyphColor

The GlyphColor on the left side of the operand.

`right` GlyphColor

The GlyphColor on the right side of the operand.

Returns

bool

True if the current left is equal to the `right` parameter; otherwise, false.

operator !=(GlyphColor, GlyphColor)

Compares two GlyphColor objects for inequality.

```
public static bool operator !=(GlyphColor left, GlyphColor right)
```

Parameters

`left` GlyphColor

The GlyphColor on the left side of the operand.

`right` GlyphColor

The GlyphColor on the right side of the operand.

Returns

bool

True if the current left is unequal to the `right` parameter; otherwise, false.

Enum GlyphLayoutMode

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Provides enumeration for the various layout mode of an individual glyph within a body of text.

```
public enum GlyphLayoutMode
```

Fields

`Horizontal = 0`

Horizontal.

`Vertical = 1`

Vertical.

`VerticalRotated = 2`

Rotated 90 degrees clockwise.

Struct GlyphMetrics

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents one laid-out glyph entry in final layout order.

```
public readonly struct GlyphMetrics
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

Advance

Gets the positioned logical advance rectangle for the glyph entry in pixel units.

```
public FontRectangle Advance { get; }
```

Property Value

FontRectangle

Bounds

Gets the rendered rectangle for the glyph entry in pixel units.

```
public FontRectangle Bounds { get; }
```

Property Value

FontRectangle

CodePoint

Gets the Unicode code point represented by the glyph entry.

```
public CodePoint CodePoint { get; }
```

Property Value

CodePoint

Font

Gets the font used to shape and render the glyph entry.

```
public Font Font { get; }
```

Property Value

Font

GlyphId

Gets the glyph identifier within Font.

```
public ushort GlyphId { get; }
```

Property Value

ushort

GraphemeIndex

Gets the zero-based grapheme index in the original text.

```
public int GraphemeIndex { get; }
```

Property Value

int

RenderableBounds

Gets the union of the positioned logical advance rectangle and rendered rectangle in pixel units.

```
public FontRectangle RenderableBounds { get; }
```

Property Value

FontRectangle

StringIndex

Gets the zero-based UTF-16 code unit index in the original text.

```
public int StringIndex { get; }
```

Property Value

int

Methods

ToString()

Returns the fully qualified type name of this instance.

```
public override string ToString()
```

Returns

string

The fully qualified type name.

Class GlyphMissingException

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Exception for detailing missing font families.

```
public class GlyphMissingException : FontException, ISerializable
```

Inheritance

object ← Exception ← FontException ← GlyphMissingException

Implements

ISerializable

Inherited Members

Exception.GetBaseException() , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

GlyphMissingException(CodePoint)

Initializes a new instance of the GlyphMissingException class.

```
public GlyphMissingException(CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The code point for the glyph we where unable to find.

See Also

FontException

Class GlyphOptions

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Provides configuration options for rendering a glyph by id.

```
public class GlyphOptions
```

Inheritance

object ← GlyphOptions

Derived

RichGlyphOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BaselineOffset

Gets or sets an additional shift of the text away from its anchored position, in pixel units along the block flow axis. Positive values shift toward the text's over side: upward for horizontal layouts, toward the over column side for vertical layouts, and away from the line along its normal when the text follows a path.

```
public float BaselineOffset { get; set; }
```

Property Value

float

Remarks

The shift composes with TextBaseline and moves rendered glyphs, ink bounds, and decorations as a unit. The logical advance is unaffected, matching the CSS and SVG `baseline-shift` model.

ColorFontSupport

Gets or sets the color font support options.

```
public ColorFontSupport ColorFontSupport { get; set; }
```

Property Value

ColorFontSupport

Remarks

A text run with a non-null ColorFontSupport value replaces this value over that run.

DecorationPositioningMode

Gets or sets the positioning mode used for rendering decorations.

```
public DecorationPositioningMode DecorationPositioningMode { get; set; }
```

Property Value

DecorationPositioningMode

Dpi

Gets or sets the DPI (Dots Per Inch) to render/measure the text at.

Defaults to `72F`.

```
public float Dpi { get; set; }
```

Property Value

float

Font

Gets or sets the font.

```
public required Font Font { get; set; }
```

Property Value

Font

FontPalette

Gets or sets the color palette selection used to resolve COLR glyph colors, or null to use the font's default palette.

```
public FontPalette? FontPalette { get; set; }
```

Property Value

FontPalette

Remarks

A text run with a non-null FontPalette value replaces this value over that run.

GraphemeIndex

Gets or sets the zero-based grapheme cluster index represented by the glyph.

```
public int GraphemeIndex { get; set; }
```

Property Value

int

Remarks

This value is passed to glyph renderers through GraphemeIndex. Callers rendering shaped glyph runs should set this from the shaping cluster so grapheme-aware renderers can group glyph paths correctly. The default value is suitable for rendering a single isolated glyph.

HintingMode

Gets or sets a value indicating whether to apply hinting - The use of mathematical instructions to adjust the display of an outline font so that it lines up with a rasterized grid.

```
public HintingMode HintingMode { get; set; }
```

Property Value

HintingMode

LayoutMode

Gets or sets the layout mode for the text lines.

```
public LayoutMode LayoutMode { get; set; }
```

Property Value

LayoutMode

Origin

Gets or sets the rendering origin.

```
public Vector2 Origin { get; set; }
```

Property Value

Vector2

TextAttributes

Gets or sets the text attributes applied to the glyph.

```
public TextAttributes TextAttributes { get; set; }
```

Property Value

TextAttributes

TextBaseline

Gets or sets which reference line of the glyph's em box is placed at Origin along the block flow axis.

```
public TextBaseline TextBaseline { get; set; }
```

Property Value

TextBaseline

Remarks

Baseline positions derive from the metrics of Font. For a single glyph LineBox anchors the top of the em box, matching text layout.

TextDecorationSkipInk

Gets or sets a value indicating whether underline and overline decorations skip over glyph ink, leaving gaps around descenders and ascenders that cross the decoration. Defaults to Auto.

```
public TextDecorationSkipInk TextDecorationSkipInk { get; set; }
```

Property Value

TextDecorationSkipInk

TextDecorations

Gets or sets the text decorations applied to the glyph.

```
public TextDecorations TextDecorations { get; set; }
```

Property Value

TextDecorations

VisibleBounds

Gets or sets the visible region in pixel units (px) used when rendering the glyph. A glyph that falls outside the region is skipped.

```
public FontRectangle? VisibleBounds { get; set; }
```

Property Value

FontRectangle?

Remarks

If value is null then culling is disabled.

Methods

CreateTextRun()

Creates the text run associated with the glyph.

```
protected virtual TextRun CreateTextRun()
```

Returns

TextRun

The text run associated with the glyph.

Enum GlyphType

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents the various versions of a glyph records.

```
public enum GlyphType
```

Fields

`Fallback = 0`

This is a fall back glyph due to a missing code point.

`Painted = 2`

This is a multi-layer colored glyph (emoji).

`Placeholder = 3`

This is an atomic inline placeholder supplied by the caller.

`Standard = 1`

This is a standard glyph to be drawn in the style the user defines.

Struct GraphemeMetrics

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents one coalesced grapheme in final layout order.

```
public readonly struct GraphemeMetrics
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

Advance

Gets the positioned logical advance rectangle for the grapheme in pixel units.

```
public FontRectangle Advance { get; }
```

Property Value

FontRectangle

Bounds

Gets the rendered glyph bounds for the grapheme in pixel units.

```
public FontRectangle Bounds { get; }
```

Property Value

FontRectangle

Font

Gets the font used to shape and render the grapheme.

```
public Font Font { get; }
```

Property Value

Font

GraphemeIndex

Gets the zero-based grapheme index in the original text.

```
public int GraphemeIndex { get; }
```

Property Value

int

IsLineBreak

Gets a value indicating whether this grapheme represents a line break.

```
public bool IsLineBreak { get; }
```

Property Value

bool

RenderableBounds

Gets the union of the positioned logical advance bounds and rendered glyph bounds in pixel units.

```
public FontRectangle RenderableBounds { get; }
```

Property Value

FontRectangle

StringIndex

Gets the zero-based UTF-16 code unit index in the original text.

```
public int StringIndex { get; }
```

Property Value

int

Enum HintingMode

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Defines modes to determine how to apply hinting. The use of mathematical instructions to adjust the display of an outline font so that it lines up with a rasterized grid.

```
public enum HintingMode
```

Fields

`Full = 2`

Hint the glyph using the complete horizontal and vertical instruction set. Glyph outlines and origins are aligned to the pixel grid on both axes, producing the sharpest results at small sizes at the cost of shape fidelity and up to half a pixel of inter-glyph spacing variation. Measured layout is unaffected.

`None = 0`

Do not hint the glyphs.

`Standard = 1`

Hint the glyph using standard configuration.

Enum HorizontalAlignment

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Horizontal alignment modes.

```
public enum HorizontalAlignment
```

Fields

`Center = 2`

Aligns text from the center.

`Left = 0`

Aligns text from the left.

`Right = 1`

Aligns text from the right.

Class HorizontalMetrics

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represent the metrics of a font face specific to horizontal text.

```
public class HorizontalMetrics : IMetricsHeader
```

Inheritance

object ← HorizontalMetrics

Implements

IMetricsHeader

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AdvanceHeightMax

Gets the maximum advance height, in font units, for all glyphs in this face.This is only relevant for vertical layouts, and is set to LineHeight for fonts that do not provide vertical metrics.

```
public short AdvanceHeightMax { get; }
```

Property Value

short

AdvanceWidthMax

Gets the maximum advance width, in font units, for all glyphs in this face.

```
public short AdvanceWidthMax { get; }
```

Property Value

short

Ascender

Gets the typographic ascender of the face, expressed in font units.

```
public short Ascender { get; }
```

Property Value

short

Descender

Gets the typographic descender of the face, expressed in font units.

```
public short Descender { get; }
```

Property Value

short

LineGap

Gets the typographic line gap of the face, expressed in font units. This field should be combined with the Ascender and Descender values to determine default line spacing.

```
public short LineGap { get; }
```

Property Value

short

LineHeight

Gets the typographic line spacing of the face, expressed in font units.

```
public short LineHeight { get; }
```

Property Value

short

Interface IFontCollection

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

A readable and writable collection of fonts.

```
public interface IFontCollection : IReadOnlyFontCollection
```

Inherited Members
IReadOnlyFontCollection.Families , IReadOnlyFontCollection.Get(string) ,
IReadOnlyFontCollection.TryGet(string, out FontFamily) ,
IReadOnlyFontCollection.GetByCulture(CultureInfo) ,
IReadOnlyFontCollection.GetByCulture(string, CultureInfo) ,
IReadOnlyFontCollection.TryGetByCulture(string, CultureInfo, out FontFamily)

Methods

Add(Stream)

Adds a font to the collection.

```
FontFamily Add(Stream stream)
```

Parameters

`stream` Stream

The font stream.

Returns

FontFamily

The new FontFamily.

Add(Stream, out FontDescription)

Adds a font to the collection.

```
FontFamily Add(Stream stream, out FontDescription description)
```

Parameters

`stream` Stream

The font stream.

`description` FontDescription

The description of the added font.

Returns

FontFamily

The new FontFamily.

Add(string)

Adds a font to the collection.

```
FontFamily Add(string path)
```

Parameters

`path` string

The filesystem path to the font file.

Returns

FontFamily

The new FontFamily.

Add(string, out FontDescription)

Adds a font to the collection.

```
FontFamily Add(string path, out FontDescription description)
```

Parameters

`path` string

The filesystem path to the font file.

`description` FontDescription

The description of the added font.

Returns

FontFamily

The new FontFamily.

AddCollection(Stream)

Adds a true type font collection (.ttc).

```
ReadOnlyMemory<FontFamily> AddCollection(Stream stream)
```

Parameters

`stream` Stream

The font stream.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(Stream, CultureInfo)

Adds a true type font collection (.ttc).

```
ReadOnlyMemory<FontFamily> AddCollection(Stream stream, CultureInfo culture)
```

Parameters

`stream` Stream

The font stream.

`culture` CultureInfo

The culture of the fonts to add.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(Stream, CultureInfo, out ReadOnlyMemory<FontDescription>)

Adds a true type font collection (.ttc).

```
ReadOnlyMemory<FontFamily> AddCollection(Stream stream, CultureInfo culture, out
ReadOnlyMemory<FontDescription> descriptions)
```

Parameters

`stream` Stream

The font stream.

`culture` CultureInfo

The culture of the fonts to add.

`descriptions` ReadOnlyMemory<FontDescription>

The descriptions of the added fonts.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(Stream, out ReadOnlyMemory<FontDescription>)

Adds a true type font collection (.ttc).

```
ReadOnlyMemory<FontFamily> AddCollection(Stream stream, out ReadOnlyMemory<FontDescription>
descriptions)
```

Parameters

`stream` Stream

The font stream.

`descriptions` ReadOnlyMemory<FontDescription>

The descriptions of the added fonts.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(string)

Adds a true type font collection (.ttc).

```
ReadOnlyMemory<FontFamily> AddCollection(string path)
```

Parameters

`path` string

The font collection path.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(string, CultureInfo)

Adds a true type font collection (.ttc).

```
ReadOnlyMemory<FontFamily> AddCollection(string path, CultureInfo culture)
```

Parameters

`path` string

The font collection path.

`culture` CultureInfo

The culture of the fonts to add.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(string, CultureInfo, out ReadOnlyMemory<FontDescription>)

Adds a true type font collection (.ttc).

```
ReadOnlyMemory<FontFamily> AddCollection(string path, CultureInfo culture, out
ReadOnlyMemory<FontDescription> descriptions)
```

Parameters

`path` string

The font collection path.

`culture` CultureInfo

The culture of the fonts to add.

`descriptions` ReadOnlyMemory<FontDescription>

The descriptions of the added fonts.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddCollection(string, out ReadOnlyMemory<FontDescription>)

Adds a true type font collection (.ttc).

```
ReadOnlyMemory<FontFamily> AddCollection(string path, out ReadOnlyMemory<FontDescription>
descriptions)
```

Parameters

`path` string

The font collection path.

`descriptions` ReadOnlyMemory<FontDescription>

The descriptions of the added fonts.

Returns

ReadOnlyMemory<FontFamily>

A read-only memory region containing the new FontFamily values.

AddWithCulture(Stream, CultureInfo)

Adds a font to the collection.

```
FontFamily AddWithCulture(Stream stream, CultureInfo culture)
```

Parameters

`stream` Stream

The font stream.

`culture` CultureInfo

The culture of the font to add.

Returns

FontFamily

The new FontFamily.

AddWithCulture(Stream, CultureInfo, out FontDescription)

Adds a font to the collection.

```
FontFamily AddWithCulture(Stream stream, CultureInfo culture, out
FontDescription description)
```

Parameters

`stream` Stream

The font stream.

`culture` CultureInfo

The culture of the font to add.

`description` FontDescription

The description of the added font.

Returns

FontFamily

The new FontFamily.

AddWithCulture(string, CultureInfo)

Adds a font to the collection.

```
FontFamily AddWithCulture(string path, CultureInfo culture)
```

Parameters

`path` string

The filesystem path to the font file.

`culture` CultureInfo

The culture of the font to add.

Returns

FontFamily

The new FontFamily.

AddWithCulture(string, CultureInfo, out FontDescription)

Adds a font to the collection.

```
FontFamily AddWithCulture(string path, CultureInfo culture, out FontDescription description)
```

Parameters

`path` string

The filesystem path to the font file.

`culture` CultureInfo

The culture of the font to add.

`description` FontDescription

The description of the added font.

Returns

FontFamily

The new FontFamily.

See Also

IReadOnlyFontCollection

Interface IFontFallbackResolver

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Resolves a font family for code points that no configured font can shape. The shaping pipeline consults the resolver only after Font and every FallbackFontFamilies entry have attempted the text, and at most once per distinct unresolved code point per shaping operation.

```
public interface IFontFallbackResolver
```

Methods

TryResolve(CodePoint, FontFamily, FontStyle, CultureInfo?, out FontFamily)

Tries to resolve a font family containing a glyph for the given code point.

```
bool TryResolve(CodePoint codePoint, FontFamily requestedFamily, FontStyle style,
CultureInfo? culture, out FontFamily family)
```

Parameters

`codePoint` CodePoint

The code point no configured font can shape.

`requestedFamily` FontFamily

The family of the requested font, usable as a hint to bias matching toward stylistically compatible faces.

`style` FontStyle

The requested font style.

`culture` CultureInfo

The culture used to select language specific faces, or null.

`family` FontFamily

When this method returns true, the resolved font family.

Returns

bool

true if a family was resolved; otherwise, false.

Interface IMetricsHeader

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Defines the contract for the metrics header of a font face.

```
public interface IMetricsHeader
```

Properties

AdvanceHeightMax

Gets the maximum advance height, in font units, for all glyphs in this face.This is only relevant for vertical layouts, and is set to LineHeight for fonts that do not provide vertical metrics.

```
short AdvanceHeightMax { get; }
```

Property Value

short

AdvanceWidthMax

Gets the maximum advance width, in font units, for all glyphs in this face.

```
short AdvanceWidthMax { get; }
```

Property Value

short

Ascender

Gets the typographic ascender of the face, expressed in font units.

```
short Ascender { get; }
```

Property Value

short

Descender

Gets the typographic descender of the face, expressed in font units.

```
short Descender { get; }
```

Property Value

short

LineGap

Gets the typographic line gap of the face, expressed in font units. This field should be combined with the Ascender and Descender values to determine default line spacing.

```
short LineGap { get; }
```

Property Value

short

LineHeight

Gets the typographic line spacing of the face, expressed in font units.

```
short LineHeight { get; }
```

Property Value

short

Interface IReadOnlyFontCollection

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a readonly collection of fonts.

```
public interface IReadOnlyFontCollection
```

Properties

Families

Gets the collection of FontFamily in this IReadOnlyFontCollection using the invariant culture.

```
IEnumerable<FontFamily> Families { get; }
```

Property Value

IEnumerable<FontFamily>

Methods

Get(string)

Gets the specified font family matching the invariant culture and font family name.

```
FontFamily Get(string name)
```

Parameters

`name` string

The font family name.

Returns

FontFamily

The first FontFamily matching the given name.

Exceptions

ArgumentNullException

`name` is null

FontFamilyNotFoundException

The collection contains no matches.

GetByCulture(CultureInfo)

Gets the collection of FontFamily in this FontCollection using the given culture.

```
IEnumerable<FontFamily> GetByCulture(CultureInfo culture)
```

Parameters

`culture` CultureInfo

The culture of the families to return.

Returns

IEnumerable<FontFamily>

The IEnumerable<T>.

GetByCulture(string, CultureInfo)

Gets the specified font family matching the given culture and font family name.

```
FontFamily GetByCulture(string name, CultureInfo culture)
```

Parameters

`name` string

The font family name.

`culture` CultureInfo

The culture to use when searching for a match.

Returns

FontFamily

The first FontFamily matching the given name.

Exceptions

ArgumentNullException

`name` is null

FontFamilyNotFoundException

The collection contains no matches.

TryGet(string, out FontFamily)

Gets the specified font family matching the invariant culture and font family name.

```
bool TryGet(string name, out FontFamily family)
```

Parameters

`name` string

The font family name.

`family` FontFamily

When this method returns, contains the family associated with the specified name, if the name is found; otherwise, the default value for the type of the family parameter. This parameter is

passed uninitialized.

Returns

bool

true if the IReadOnlyFontCollection contains a family with the specified name; otherwise, false.

Exceptions

ArgumentNullException

`name` is null

TryGetByCulture(string, CultureInfo, out FontFamily)

Gets the specified font family matching the given culture and font family name.

```
bool TryGetByCulture(string name, CultureInfo culture, out FontFamily family)
```

Parameters

`name` string

The font family name.

`culture` CultureInfo

The culture to use when searching for a match.

`family` FontFamily

When this method returns, contains the family associated with the specified name, if the name is found; otherwise, the default value for the type of the family parameter. This parameter is passed uninitialized.

Returns

bool

true if the IReadOnlyFontCollection contains a family with the specified name; otherwise, false.

Exceptions

ArgumentNullException

`name` is null

Interface IReadOnlySystemFontCollection

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a readonly collection of Operating System fonts.

```
public interface IReadOnlySystemFontCollection : IReadOnlyFontCollection
```

Inherited Members
IReadOnlyFontCollection.Families , IReadOnlyFontCollection.Get(string) ,
IReadOnlyFontCollection.TryGet(string, out FontFamily) ,
IReadOnlyFontCollection.GetByCulture(CultureInfo) ,
IReadOnlyFontCollection.GetByCulture(string, CultureInfo) ,
IReadOnlyFontCollection.TryGetByCulture(string, CultureInfo, out FontFamily)

Properties

SearchDirectories

Gets the collection of Operating System directories that were searched for font families.

```
IEnumerable<string> SearchDirectories { get; }
```

Property Value

IEnumerable<string>

Methods

TryMatchCharacter(CodePoint, FontStyle, string?, CultureInfo?, out FontMatch)

Tries to match a character to an installed system font.

```
bool TryMatchCharacter(CodePoint codePoint, FontStyle style, string? familyName,
CultureInfo? culture, out FontMatch match)
```

Parameters

`codePoint` CodePoint

The code point to match.

`style` FontStyle

The requested font style.

`familyName` string

The requested font family name, or null.

`culture` CultureInfo

The culture used for matching, or null.

`match` FontMatch

When this method returns, contains the matched font if one was found; otherwise, the default value. This parameter is passed uninitialized.

Returns

bool

true if a matching system font was found; otherwise, false.

Class InvalidFontFileException

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Exception font loading can throw if it encounters invalid data during font loading.

```
public class InvalidFontFileException : Exception, ISerializable
```

Inheritance

object ← Exception ← InvalidFontFileException

Implements

ISerializable

Derived

InvalidFontTableException, MissingFontTableException

Inherited Members

Exception.GetBaseException() , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

InvalidFontFileException(string)

Initializes a new instance of the InvalidFontFileException class.

```
public InvalidFontFileException(string message)
```

Parameters

`message` string

The message that describes the error.

See Also

Exception

Class InvalidFontTableException

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Exception font loading can throw if it encounters invalid data during font loading.

```
public class InvalidFontTableException : InvalidFontFileException, ISerializable
```

Inheritance

object ← Exception ← InvalidFontFileException ← InvalidFontTableException

Implements

ISerializable

Inherited Members

Exception.GetBaseException() , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

InvalidFontTableException(string, string)

Initializes a new instance of the InvalidFontTableException class.

```
public InvalidFontTableException(string message, string table)
```

Parameters

`message` string

The message that describes the error.

`table` string

The table.

Properties

Table

Gets the table where the error originated.

```
public string Table { get; }
```

Property Value

string

See Also

Exception

Enum KerningMode

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Kerning is the contextual adjustment of inter-glyph spacing. This property controls metric kerning, kerning that utilizes adjustment data contained in the font.

```
public enum KerningMode
```

Fields

`Auto = 2`

Specifies that kerning is applied at the discretion of the layout engine.

`None = 1`

Specifies that kerning is not applied.

`Standard = 0`

Specifies that kerning is applied.

Class KnownVariationAxes

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Defines the registered design-variation axis tags for variable fonts. These tags are used with Font Variation to control font design axes. https://learn.microsoft.com/en-us/typography/opentype/spec/dvaraxisreg

```
public static class KnownVariationAxes
```

Inheritance

object ← KnownVariationAxes

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Fields

Italic

Italic axis ('ital'). Controls the italic angle of the font. Value range: 0 (upright) to 1 (italic). https://learn.microsoft.com/en-us/typography/opentype/spec/dvaraxistag_ital

```
public static readonly Tag Italic
```

Field Value

Tag

OpticalSize

Optical size axis ('opsz'). Adjusts the design for a specific text size in points. Typical range: 6 to 144. Larger values optimize for display use; smaller for body text. https://learn.microsoft.com/en-us/typography/opentype/spec/dvaraxistag_opsz

```
public static readonly Tag OpticalSize
```

Field Value

Tag

Slant

Slant axis ('slnt'). Controls the slant angle of upright glyphs in degrees. Typical range: -90 to 90. Negative values slant to the right (the common direction). https://learn.microsoft.com/en-us/typography/opentype/spec/dvaraxistag_slnt

```
public static readonly Tag Slant
```

Field Value

Tag

Weight

Weight axis ('wght'). Controls the weight (boldness) of the font. Range: 1 to 1000. Common values: 100 (Thin), 300 (Light), 400 (Regular), 700 (Bold), 900 (Black). https://learn.microsoft.com/en-us/typography/opentype/spec/dvaraxistag_wght

```
public static readonly Tag Weight
```

Field Value

Tag

Width

Width axis ('wdth'). Controls the relative width of the font as a percentage of normal. Typical range: 75 (condensed) to 125 (expanded). 100 represents the normal width.
https://learn.microsoft.com/en-us/typography/opentype/spec/dvaraxistag_wdth

```
public static readonly Tag Width
```

Field Value

Tag

Enum LayoutMode

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Defines modes to determine the layout direction of text.

```
[Flags]
public enum LayoutMode
```

Extension Methods
LayoutModeExtensions.IsHorizontal(LayoutMode) ,
LayoutModeExtensions.IsVertical(LayoutMode) ,
LayoutModeExtensions.IsVerticalMixed(LayoutMode)

Fields

`HorizontalBottomTop = 1`

Text is laid out horizontally from bottom to top.

`HorizontalTopBottom = 0`

Text is laid out horizontally from top to bottom.

`VerticalLeftRight = 2`

Text is laid out vertically from left to right.

`VerticalMixedLeftRight = 8`

Text is laid out vertically from left to right. Horizontal glyphs are rotated 90 degrees clockwise.

`VerticalMixedRightLeft = 16`

Text is laid out vertically from right to left. Horizontal glyphs are rotated 90 degrees clockwise.

`VerticalRightLeft = 4`

Text is laid out vertically from right to left.

Class LayoutModeExtensions

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Extensions to LayoutMode.

```
public static class LayoutModeExtensions
```

Inheritance

object ← LayoutModeExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

IsHorizontal(LayoutMode)

Gets a value indicating whether the layout mode is horizontal.

```
public static bool IsHorizontal(this LayoutMode mode)
```

Parameters

`mode` LayoutMode

The layout mode.

Returns

bool

The bool.

IsVertical(LayoutMode)

Gets a value indicating whether the layout mode is vertical.

```
public static bool IsVertical(this LayoutMode mode)
```

Parameters

`mode` LayoutMode

The layout mode.

Returns

bool

The bool.

IsVerticalMixed(LayoutMode)

Gets a value indicating whether the layout mode is vertical-mixed only.

```
public static bool IsVerticalMixed(this LayoutMode mode)
```

Parameters

`mode` LayoutMode

The layout mode.

Returns

bool

The bool.

Class LineLayout

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents one laid-out line from a TextBlock.

```
public sealed class LineLayout
```

Inheritance

object ← LineLayout

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

GraphemeMetrics

Gets the grapheme metrics entries for this line in final layout order.

```
public ReadOnlySpan<GraphemeMetrics> GraphemeMetrics { get; }
```

Property Value

ReadOnlySpan<GraphemeMetrics>

LineMetrics

Gets the measured line metrics.

```
public LineMetrics LineMetrics { get; }
```

Property Value

LineMetrics

TextDecorationSkipInk

Gets a value indicating whether underline and overline decorations skip over glyph ink when this line is rendered.

```
public TextDecorationSkipInk TextDecorationSkipInk { get; }
```

Property Value

TextDecorationSkipInk

Methods

GetCaret(CaretPlacement)

Gets an absolute caret position in the laid-out line.

```
public CaretPosition GetCaret(CaretPlacement placement)
```

Parameters

`placement` CaretPlacement

The absolute caret placement.

Returns

CaretPosition

The caret position in pixel units.

GetCaretPosition(TextHit)

Gets the caret position for the supplied hit.

```
public CaretPosition GetCaretPosition(TextHit hit)
```

Parameters

`hit` TextHit

The hit-tested grapheme position.

Returns

CaretPosition

The caret position in pixel units.

GetGlyphMetrics()

Gets the positioned metrics of each laid-out glyph entry.

```
public ReadOnlyMemory<GlyphMetrics> GetGlyphMetrics()
```

Returns

ReadOnlyMemory<GlyphMetrics>

A read-only memory region containing per-glyph metrics entries.

GetSelectionBounds(CaretPosition, CaretPosition)

Gets selection bounds between two caret positions.

```
public ReadOnlyMemory<FontRectangle> GetSelectionBounds(CaretPosition anchor,
CaretPosition focus)
```

Parameters

`anchor` CaretPosition

The fixed selection endpoint.

`focus` CaretPosition

The active selection endpoint.

Returns

ReadOnlyMemory<FontRectangle>

A read-only memory region containing the selection bounds in visual order and pixel units.

GetSelectionBounds(GraphemeMetrics)

Gets line-local selection bounds for the supplied grapheme metrics.

```
public ReadOnlyMemory<FontRectangle> GetSelectionBounds(GraphemeMetrics metrics)
```

Parameters

`metrics` GraphemeMetrics

The grapheme metrics to select.

Returns

ReadOnlyMemory<FontRectangle>

A read-only memory region containing the selection bounds in visual order and pixel units.

GetSelectionBounds(TextHit, TextHit)

Gets selection bounds between two hit-tested grapheme positions.

```
public ReadOnlyMemory<FontRectangle> GetSelectionBounds(TextHit anchor, TextHit focus)
```

Parameters

`anchor` TextHit

The fixed selection endpoint.

`focus` TextHit

The active selection endpoint.

Returns

ReadOnlyMemory<FontRectangle>

A read-only memory region containing the selection bounds in visual order and pixel units.

GetSelectionBounds(WordMetrics)

Gets line-local selection bounds for the supplied word metrics.

```
public ReadOnlyMemory<FontRectangle> GetSelectionBounds(WordMetrics metrics)
```

Parameters

`metrics` WordMetrics

The word metrics to select.

Returns

ReadOnlyMemory<FontRectangle>

A read-only memory region containing the selection bounds in visual order and pixel units.

GetWordMetrics(CaretPosition)

Gets the word metrics for the word-boundary segment containing the supplied caret position.

```
public WordMetrics GetWordMetrics(CaretPosition caret)
```

Parameters

`caret` CaretPosition

The caret position.

Returns

WordMetrics

The word metrics containing the caret's grapheme insertion index.

GetWordMetrics(TextHit)

Gets the word metrics for the word-boundary segment containing the supplied hit-tested grapheme position.

```
public WordMetrics GetWordMetrics(TextHit hit)
```

Parameters

`hit` TextHit

The hit-tested grapheme position.

Returns

WordMetrics

The word metrics containing the hit grapheme.

HitTest(Vector2)

Hit tests the supplied point against this line's grapheme advance bounds.

```
public TextHit HitTest(Vector2 point)
```

Parameters

`point` Vector2

The point in pixel units.

Returns

TextHit

The hit-tested grapheme position.

MoveCaret(CaretPosition, CaretMovement)

Moves the supplied caret by the requested operation within this line.

```
public CaretPosition MoveCaret(CaretPosition caret, CaretMovement movement)
```

Parameters

`caret` CaretPosition

The current caret position.

`movement` CaretMovement

The movement operation.

Returns

CaretPosition

The moved caret position in pixel units.

RenderTo(IGlyphRenderer)

Renders this line to the supplied glyph renderer.

```
public void RenderTo(IGlyphRenderer renderer)
```

Parameters

`renderer` IGlyphRenderer

The target renderer.

Class LineLayoutEnumerator

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Walks a TextBlock one laid-out line at a time.

```
public sealed class LineLayoutEnumerator
```

Inheritance

object ← LineLayoutEnumerator

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Each produced line is positioned independently, without cumulative offsets from earlier or later lines.

Properties

Current

Gets the current line layout.

```
public LineLayout Current { get; }
```

Property Value

LineLayout

Methods

MoveNext(float)

Advances to the next line using the supplied wrapping length.

```
public bool MoveNext(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

bool

true when a line was produced.

Remarks

The wrapping length applies only to the line being produced by this call.

Struct LineMetrics

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Encapsulates measured metrics for a single laid-out text line.

```
public readonly struct LineMetrics
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

Ascender

Gets the ascender line position within the line box.

```
public float Ascender { get; }
```

Property Value

float

Remarks

This is a position value (not a baseline-relative distance). Use this value to draw the ascender guide line relative to the current line origin.

Baseline

Gets the baseline position within the line box.

```
public float Baseline { get; }
```

Property Value

float

Remarks

Use this value as the guide-line position for drawing a baseline relative to the current line origin.

Descender

Gets the descender line position within the line box.

```
public float Descender { get; }
```

Property Value

float

Remarks

This is a position value (not a baseline-relative distance). Use this value to draw the descender guide line relative to the current line origin.

Extent

Gets the logical line box extent in pixel units.

```
public Vector2 Extent { get; }
```

Property Value

Vector2

GraphemeCount

Gets the number of graphemes in the line.

```
public int GraphemeCount { get; }
```

Property Value

int

GraphemeIndex

Gets the zero-based grapheme index in the original text.

```
public int GraphemeIndex { get; }
```

Property Value

int

LineHeight

Gets the total line-box size for this line.

```
public float LineHeight { get; }
```

Property Value

float

Start

Gets the logical line box start position in pixel units.

```
public Vector2 Start { get; }
```

Property Value

Vector2

StringIndex

Gets the zero-based UTF-16 code unit index in the original text.

```
public int StringIndex { get; }
```

Property Value

int

Class MissingFontTableException

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Exception font loading can throw if it finds a required table is missing during font loading.

```
public class MissingFontTableException : InvalidFontFileException, ISerializable
```

Inheritance

object ← Exception ← InvalidFontFileException ← MissingFontTableException

Implements

ISerializable

Inherited Members

Exception.GetBaseException() , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , Exception.SerializeObjectState , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

MissingFontTableException(string, string)

Initializes a new instance of the MissingFontTableException class.

```
public MissingFontTableException(string message, string table)
```

Parameters

`message` string

The message that describes the error.

`table` string

The table.

Properties

Table

Gets the table where the error originated.

```
public string Table { get; }
```

Property Value

string

See Also

Exception

Struct ShapedGlyph

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a single shaped glyph: the result of substitution and positioning, before line breaking.

```
public readonly struct ShapedGlyph
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Remarks

Advances and offsets are scaled to the size of the font the run was shaped against and preserve the shaper's Y-up coordinate system.

Properties

AdvanceHeight

Gets the scaled vertical advance after positioning features have been applied.

```
public float AdvanceHeight { get; }
```

Property Value

float

AdvanceWidth

Gets the scaled horizontal advance after positioning features have been applied.

```
public float AdvanceWidth { get; }
```

Property Value

float

GlyphId

Gets the glyph identifier within the font the run was shaped against. Glyph id 0 is the font's missing glyph, produced when the font cannot map the character.

```
public ushort GlyphId { get; }
```

Property Value

ushort

GraphemeIndex

Gets the zero-based grapheme index in the input text.

```
public int GraphemeIndex { get; }
```

Property Value

int

Offset

Gets the scaled placement offset in Y-up shaping space. The offset positions the glyph outline relative to its pen position and does not contribute to the advance.

```
public Vector2 Offset { get; }
```

Property Value

Vector2

StringIndex

Gets the zero-based index of the first input char represented by this glyph.

```
public int StringIndex { get; }
```

Property Value

int

Remarks

Glyphs produced from the same input chars share this value.

Class SystemFontFallbackResolver

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Resolves fallback font families from the fonts installed on the current machine using the operating system's character-to-font matching service. Results depend on the machine's installed fonts: the same input can resolve different families, or none, on different machines.

```
public sealed class SystemFontFallbackResolver : IFontFallbackResolver
```

Inheritance

object ← SystemFontFallbackResolver

Implements

IFontFallbackResolver

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Methods

TryResolve(CodePoint, FontFamily, FontStyle, CultureInfo?, out FontFamily)

Tries to resolve a font family containing a glyph for the given code point.

```
public bool TryResolve(CodePoint codePoint, FontFamily requestedFamily, FontStyle style,
CultureInfo? culture, out FontFamily family)
```

Parameters

`codePoint` CodePoint

The code point no configured font can shape.

`requestedFamily` FontFamily

The family of the requested font, usable as a hint to bias matching toward stylistically compatible faces.

`style` FontStyle

The requested font style.

`culture` CultureInfo

The culture used to select language specific faces, or null.

`family` FontFamily

When this method returns true, the resolved font family.

Returns

bool

true if a family was resolved; otherwise, false.

Class SystemFonts

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Provides a collection of fonts.

```
public static class SystemFonts
```

Inheritance

object ← SystemFonts

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Collection

Gets the collection containing the globally installed system fonts.

```
public static IReadOnlySystemFontCollection Collection { get; }
```

Property Value

IReadOnlySystemFontCollection

FallbackResolver

Gets a resolver that selects fallback font families from the installed system fonts. Assign it to Font FallbackResolver to let shaping consult the operating system for code points no configured font can shape.

```
public static IFontFallbackResolver FallbackResolver { get; }
```

Property Value

IFontFallbackResolver

Families

Gets the collection of FontFamilys installed on current system.

```
public static IEnumerable<FontFamily> Families { get; }
```

Property Value

IEnumerable<FontFamily>

Methods

CreateFont(string, CultureInfo, float)

Create a new instance of the Font for the named font family with regular styling.

```
public static Font CreateFont(string name, CultureInfo culture, float size)
```

Parameters

`name` string

The font family name.

`culture` CultureInfo

The font culture.

`size` float

The size of the font in PT units.

Returns

Font

The new Font.

CreateFont(string, CultureInfo, float, FontStyle)

Create a new instance of the Font for the named font family.

```
public static Font CreateFont(string name, CultureInfo culture, float size, FontStyle style)
```

Parameters

`name` string

The font family name.

`culture` CultureInfo

The font culture.

`size` float

The size of the font in PT units.

`style` FontStyle

The font style.

Returns

Font

The new Font.

CreateFont(string, float)

Create a new instance of the Font for the named font family with regular styling.

```
public static Font CreateFont(string name, float size)
```

Parameters

`name` string

The font family name.

`size` float

The size of the font in PT units.

Returns

Font

The new Font.

CreateFont(string, float, FontStyle)

Create a new instance of the Font for the named font family.

```
public static Font CreateFont(string name, float size, FontStyle style)
```

Parameters

`name` string

The font family name.

`size` float

The size of the font in PT units.

`style` FontStyle

The font style.

Returns

Font

The new Font.

Get(string)

Gets the specified font family matching the invariant culture and font family name.

```
public static FontFamily Get(string name)
```

Parameters

`name` string

The font family name.

Returns

FontFamily

The first FontFamily matching the given name.

Exceptions

ArgumentNullException

`name` is null

FontFamilyNotFoundException

The collection contains no matches.

GetByCulture(CultureInfo)

Gets the collection of FontFamily in this FontCollection using the given culture.

```
public static IEnumerable<FontFamily> GetByCulture(CultureInfo culture)
```

Parameters

`culture` CultureInfo

The culture of the families to return.

Returns

IEnumerable<FontFamily>

The IEnumerable<T>.

GetByCulture(string, CultureInfo)

Gets the specified font family matching the given culture and font family name.

```
public static FontFamily GetByCulture(string fontFamily, CultureInfo culture)
```

Parameters

`fontFamily` string

`culture` CultureInfo

The culture to use when searching for a match.

Returns

FontFamily

The first FontFamily matching the given name.

Exceptions

ArgumentNullException

`name` is null

FontFamilyNotFoundException

The collection contains no matches.

GetDefaultFamilyName()

Gets the operating system default font family name.

```
public static string GetDefaultFamilyName()
```

Returns

string

The default font family name.

Exceptions

InvalidOperationException

No system font families are available.

GetFamilyNames(bool)

Gets the names of font families installed on current system.

```
public static string[] GetFamilyNames(bool checkForUpdates = false)
```

Parameters

`checkForUpdates` bool

Whether the operating system should check for updates to the system font collection.

Returns

string[]

The installed system font family names.

TryGet(string, out FontFamily)

Gets the specified font family matching the invariant culture and font family name.

```
public static bool TryGet(string fontFamily, out FontFamily family)
```

Parameters

`fontFamily` string

`family` FontFamily

When this method returns, contains the family associated with the specified name, if the name is found; otherwise, the default value for the type of the family parameter. This parameter is passed uninitialized.

Returns

bool

true if the IReadOnlyFontCollection contains a family with the specified name; otherwise, false.

Exceptions

ArgumentNullException

`name` is null

TryGetByCulture(string, CultureInfo, out FontFamily)

Gets the specified font family matching the given culture and font family name.

```
public static bool TryGetByCulture(string fontFamily, CultureInfo culture, out
FontFamily family)
```

Parameters

`fontFamily` string

`culture` CultureInfo

The culture to use when searching for a match.

`family` FontFamily

When this method returns, contains the family associated with the specified name, if the name is found; otherwise, the default value for the type of the family parameter. This parameter is passed uninitialized.

Returns

bool

true if the IReadOnlyFontCollection contains a family with the specified name; otherwise, false.

Exceptions

ArgumentNullException

`name` is null

TryGetDefaultFamilyName(out string?)

Tries to get the operating system default font family name.

```
public static bool TryGetDefaultFamilyName(out string? familyName)
```

Parameters

`familyName` string

The operating system default font family name.

Returns

bool

true when the operating system returned a default font family name.

TryMatchCharacter(CodePoint, FontStyle, string?, CultureInfo?, out FontMatch)

Tries to match a character to an installed system font.

```
public static bool TryMatchCharacter(CodePoint codePoint, FontStyle style, string?
familyName, CultureInfo? culture, out FontMatch match)
```

Parameters

`codePoint` CodePoint

The code point to match.

`style` FontStyle

The requested font style.

`familyName` string

The requested font family name, or null.

`culture` CultureInfo

The culture used for matching, or null.

`match` FontMatch

When this method returns, contains the matched font if one was found; otherwise, the default value. This parameter is passed uninitialized.

Returns

bool

true if a matching system font was found; otherwise, false.

Enum TextAlignment

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Text alignment modes.

```
public enum TextAlignment
```

Fields

`Center = 2`

Aligns text from the center.

`End = 1`

Aligns text from the right or bottom when the text direction is LeftToRight and from the left or top when the text direction is RightToLeft.

`Start = 0`

Aligns text from the left or top when the text direction is LeftToRight and from the right or bottom when the text direction is RightToLeft.

Enum TextAttributes

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Provides enumeration of various text attributes.

```
[Flags]
public enum TextAttributes
```

Fields

`None = 0`

No attributes are applied

`Subscript = 1`

The text set slightly below the normal line of type.

`Superscript = 2`

The text set slightly above the normal line of type.

Enum TextBaseline

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies which reference line of laid-out text is placed at Origin along the block flow axis, and equivalently which line rides the path when text follows one. Values match the CSS `dominant-baseline` and HTML canvas `textBaseline` vocabulary.

```
public enum TextBaseline
```

Fields

`Alphabetic = 5`

The alphabetic baseline, the line Latin glyphs sit on, is placed at the origin. Matches CSS and SVG `alphabetic` and the HTML canvas default.

`Central = 4`

The central baseline, the middle of the em box, is placed at the origin. The conventional anchor for vertical CJK layout; matches CSS and SVG `central`.

`Hanging = 2`

The hanging baseline, from which Tibetan and similar scripts hang, is placed at the origin. Matches CSS and SVG `hanging`. Positioned by the font's baseline table when the font provides one and derived from the ascender otherwise.

`Ideographic = 6`

The ideographic-under baseline, beneath CJK ideographs, is placed at the origin. Matches CSS and SVG `ideographic`. Positioned by the font's baseline table when the font provides one and derived from the descender otherwise.

`LineBox = 0`

The line box anchors at the origin and block alignment positions it along the flow axis.

`Middle = 3`

The middle baseline, half the x-height above the alphabetic baseline, is placed at the origin. Matches CSS and SVG `middle`.

`TextBottom = 7`

The bottom of the em box is placed at the origin. Matches CSS `text-bottom`.

`TextTop = 1`

The top of the em box is placed at the origin. Matches CSS `text-top`.

Enum TextBidiMode

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies how bidirectional text is resolved.

```
public enum TextBidiMode
```

Fields

`Normal = 0`

Uses the Unicode Bidirectional Algorithm with each character's bidirectional class.

`Override = 1`

Lays out text in the resolved text direction, ignoring each character's normal bidirectional class.

Class TextBlock

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents text prepared for repeated line layout, measurement, and rendering.

```
public sealed class TextBlock
```

Inheritance

object ← TextBlock

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Instances are safe to measure and render concurrently. The block retains the most recent line-broken layout; concurrent calls can only duplicate deterministic layout work, never observe partial state.

Constructors

TextBlock(ReadOnlySpan<char>, TextOptions)

Initializes a new instance of the TextBlock class.

```
public TextBlock(ReadOnlySpan<char> text, TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text to prepare.

`options` TextOptions

The text options used to prepare, measure, and render the block.

Remarks

WrappingLength and VisibleBounds are ignored while preparing the block; pass the wrapping length (use `-1` to disable wrapping) and any visible bounds to the measurement or rendering methods.

TextBlock(string, TextOptions)

Initializes a new instance of the TextBlock class.

```
public TextBlock(string text, TextOptions options)
```

Parameters

`text` string

The text to prepare.

`options` TextOptions

The text options used to prepare, measure, and render the block.

Remarks

WrappingLength and VisibleBounds are ignored while preparing the block; pass the wrapping length (use `-1` to disable wrapping) and any visible bounds to the measurement or rendering methods.

Properties

TextDecorationSkipInk

Gets a value indicating whether underline and overline decorations skip over glyph ink when this block is rendered.

```
public TextDecorationSkipInk TextDecorationSkipInk { get; }
```

Property Value

TextDecorationSkipInk

Methods

CountLines(float)

Gets the number of laid-out lines at the supplied wrapping length.

```
public int CountLines(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

int

The laid-out line count.

EnumerateLineLayouts()

Creates an enumerator that lays out this block one line at a time.

```
public LineLayoutEnumerator EnumerateLineLayouts()
```

Returns

LineLayoutEnumerator

A line layout enumerator for this block.

GetGlyphMetrics(float)

Gets the positioned metrics of each laid-out glyph entry.

```
public ReadOnlyMemory<GlyphMetrics> GetGlyphMetrics(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

ReadOnlyMemory<GlyphMetrics>

A read-only memory region containing per-glyph metrics entries.

GetGraphemeMetrics(float)

Gets the positioned metrics of each laid-out grapheme.

```
public ReadOnlyMemory<GraphemeMetrics> GetGraphemeMetrics(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

ReadOnlyMemory<GraphemeMetrics>

A read-only memory region containing per-grapheme metrics entries.

GetIntersections(float, float, float)

Gets the x-axis intervals where the laid-out glyph outlines cross a horizontal band. Glyphs whose bounds do not touch the band skip outline decoding entirely.

```
public ReadOnlyMemory<float> GetIntersections(float wrappingLength, float lowerLimit,
float upperLimit)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

`lowerLimit` float

One edge of the horizontal band.

`upperLimit` float

The other edge of the horizontal band.

Returns

ReadOnlyMemory<float>

A read-only memory region containing merged, x-sorted interval pairs (start, end, start, end, ...); empty when no outline crosses the band.

GetLineLayouts(float)

Gets visual line layouts for this block at the supplied wrapping length.

```
public ReadOnlyMemory<LineLayout> GetLineLayouts(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

ReadOnlyMemory<LineLayout>

A read-only memory region containing LineLayout entries in final layout order.

Remarks

The returned memory contains every laid-out line, including lines produced by hard line breaks.

GetLineMetrics(float)

Gets per-line layout metrics at the supplied wrapping length.

```
public ReadOnlyMemory<LineMetrics> GetLineMetrics(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

ReadOnlyMemory<LineMetrics>

A read-only memory region containing LineMetrics in pixel units.

GetWordMetrics(float)

Gets the positioned metrics of each Unicode word-boundary segment.

```
public ReadOnlyMemory<WordMetrics> GetWordMetrics(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

ReadOnlyMemory<WordMetrics>

A read-only memory region containing per-word-boundary segment metrics entries.

Measure(float)

Measures the full set of layout metrics for this block at the supplied wrapping length.

```
public TextMetrics Measure(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

TextMetrics

A TextMetrics instance containing every measurement for the laid-out text.

MeasureAdvance(float)

Measures the logical advance of this block at the supplied wrapping length.

```
public FontRectangle MeasureAdvance(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

FontRectangle

The logical advance rectangle.

MeasureBounds(float)

Measures the rendered glyph bounds of this block at the supplied wrapping length.

```
public FontRectangle MeasureBounds(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

FontRectangle

The rendered glyph bounds.

MeasureRenderableBounds(float)

Measures the union of logical advance and rendered glyph bounds at the supplied wrapping length.

```
public FontRectangle MeasureRenderableBounds(float wrappingLength)
```

Parameters

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

Returns

FontRectangle

The full renderable bounds.

RenderTo(IGlyphRenderer, float)

Renders this block to the supplied glyph renderer at the supplied wrapping length.

```
public void RenderTo(IGlyphRenderer renderer, float wrappingLength)
```

Parameters

`renderer` IGlyphRenderer

The target renderer.

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

RenderTo(IGlyphRenderer, float, in FontRectangle)

Renders the visible portion of this block to the supplied glyph renderer at the supplied wrapping length, culling whole lines that lie outside the supplied visible bounds.

```
public void RenderTo(IGlyphRenderer renderer, float wrappingLength, in
FontRectangle visibleBounds)
```

Parameters

`renderer` IGlyphRenderer

The target renderer.

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

`visibleBounds` FontRectangle

The visible region in pixels, in the same space as the rendered output.

Remarks

Culling is line-granular: every glyph on a line intersecting `visibleBounds` is rendered, and the comparison inflates each line box by one line height on each side so ink or decorations overhanging a line box never disappear at the band edges. A culled line still advances layout, so visible lines render at positions identical to a full render, and BeginText(in FontRectangle) always receives the full text bounds.

Class TextDecorationOptions

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents overrides for the geometry of a single text decoration line, allowing a TextRun to replace the values that would otherwise be derived from the font metrics.

```
public class TextDecorationOptions
```

Inheritance

object ← TextDecorationOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Values are expressed in device pixels. Any property left null falls back to the font's metric-derived value. Further overrides (for example a position override) can be added as new nullable properties without changing the GetDecorationOptions(TextDecorations) contract.

Properties

Thickness

Gets or sets the thickness of the decoration line, in device pixels, or null to use the font's metric-derived thickness.

```
public float? Thickness { get; set; }
```

Property Value

float?

Enum TextDecorationSkipInk

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Identifies whether underline and overline decorations are interrupted where they would otherwise cross glyph ink, mirroring the CSS `text-decoration-skip-ink` property. Strikethrough decorations always cross the ink regardless of this setting.

```
public enum TextDecorationSkipInk : byte
```

Fields

`Auto = 0`

Underlines and overlines skip over glyph ink, leaving gaps around descenders and ascenders that cross the decoration.

`None = 1`

Decorations are drawn continuously, crossing any glyph ink in their path.

Enum TextDecorations

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Provides enumeration of various text decorations.

```
[Flags]
public enum TextDecorations
```

Fields

`None = 0`

No attributes are applied

`Overline = 4`

The text contains a horizontal line above it

`Strikeout = 2`

The text contains a horizontal line through the center.

`Underline = 1`

The text is underlined

Enum TextDirection

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies the writing direction for text.

```
public enum TextDirection
```

Fields

`Auto = 2`

Automatically determined.

`LeftToRight = 0`

Left to right.

`RightToLeft = 1`

Right to left.

Enum TextEllipsis

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies ellipsis behavior when laid-out text is limited to a maximum number of lines.

```
public enum TextEllipsis
```

Fields

`Custom = 2`

Insert the marker specified by CustomEllipsis.

`None = 0`

Do not insert an ellipsis marker.

`Standard = 1`

Insert the standard ellipsis marker.

Struct TextHit

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a hit-tested grapheme position in laid-out text.

```
public readonly struct TextHit
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

GraphemeIndex

Gets the zero-based grapheme index in the original text.

```
public int GraphemeIndex { get; }
```

Property Value

int

GraphemeInsertionIndex

Gets the grapheme insertion index represented by this hit.

```
public int GraphemeInsertionIndex { get; }
```

Property Value

int

IsTrailing

Gets a value indicating whether the hit is on the trailing side of the grapheme.

```
public bool IsTrailing { get; }
```

Property Value

bool

LineIndex

Gets the zero-based line index.

```
public int LineIndex { get; }
```

Property Value

int

StringIndex

Gets the zero-based UTF-16 code unit index in the original text.

```
public int StringIndex { get; }
```

Property Value

int

Enum TextHyphenation

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies hyphenation marker behavior when text breaks at hyphenation opportunities.

```
public enum TextHyphenation
```

Fields

`Custom = 2`

Insert the marker specified by CustomHyphen.

`None = 0`

Do not insert a hyphenation marker.

`Standard = 1`

Insert the standard hyphenation marker.

Enum TextInteractionMode

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies how text interaction positions are modeled for laid-out text.

```
public enum TextInteractionMode
```

Fields

`Editor = 1`

Uses editor-style interaction where ordinary trailing breaking whitespace at line ends remains addressable by caret movement and selection.

`Paragraph = 0`

Uses paragraph-style interaction where trailing breaking whitespace at line ends does not create additional caret stops.

Enum TextJustification

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Text justification modes.

```
public enum TextJustification
```

Fields

`InterCharacter = 2`

The text is justified by adding space between characters (effectively varying letter-spacing), which is most appropriate for languages like Japanese.

`InterWord = 1`

The text is justified by adding space between words (effectively varying word-spacing), which is most appropriate for languages that separate words using spaces, like English or Korean.

`None = 0`

No justification

Class TextMeasurer

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Encapsulates logic for laying out and then measuring text properties.

```
public static class TextMeasurer
```

Inheritance

object ← TextMeasurer

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

CountLines(ReadOnlySpan<char>, TextOptions)

Gets the number of laid-out lines contained within the text.

```
public static int CountLines(ReadOnlySpan<char> text, TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

int

The laid-out line count.

CountLines(string, TextOptions)

Gets the number of laid-out lines contained within the text.

```
public static int CountLines(string text, TextOptions options)
```

Parameters

`text` string

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

int

The laid-out line count.

GetGlyphMetrics(TextShapingBuffer, GlyphOptions)

Gets the positioned metrics of shaped glyphs in pixel units.

```
public static ReadOnlyMemory<GlyphMetrics> GetGlyphMetrics(TextShapingBuffer buffer,
GlyphOptions options)
```

Parameters

`buffer` TextShapingBuffer

The buffer containing the shaped glyphs.

`options` GlyphOptions

The glyph options supplying the shaping font, resolution, and baseline origin.

Returns

ReadOnlyMemory<GlyphMetrics>

A read-only memory region containing one positioned metrics entry per shaped glyph.

GetGlyphMetrics(ReadOnlySpan<char>, TextOptions)

Gets the positioned metrics of each laid-out glyph entry in pixel units.

```
public static ReadOnlyMemory<GlyphMetrics> GetGlyphMetrics(ReadOnlySpan<char> text,
TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

ReadOnlyMemory<GlyphMetrics>

A read-only memory region containing the per-glyph metrics entries of the text if it was to be rendered.

GetGlyphMetrics(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions)

Gets the positioned metrics of glyphs at supplied points in pixel units.

```
public static ReadOnlyMemory<GlyphMetrics> GetGlyphMetrics(ReadOnlySpan<ushort> glyphIds,
ReadOnlySpan<Vector2> points, GlyphOptions options)
```

Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`options` GlyphOptions

The glyph options, including the font and layout mode.

Returns

ReadOnlyMemory<GlyphMetrics>

A read-only memory region containing one positioned metrics entry per input glyph.

Remarks

The metrics are computed directly from the font's cached per-glyph metrics without decoding outlines or running the layout engine. Each glyph is measured at its supplied point exactly as RenderTo(IGlyphRenderer, ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions) renders it; Origin is replaced per glyph and restored. One entry is returned per input glyph so results correlate with run indices: each entry's grapheme index is GraphemeIndex plus the run index and its string index is the run index. Glyph ids the font does not contain, and glyphs that never render, produce entries with empty rectangles.

GetGlyphMetrics(string, TextOptions)

Gets the positioned metrics of each laid-out glyph entry in pixel units.

```
public static ReadOnlyMemory<GlyphMetrics> GetGlyphMetrics(string text, TextOptions options)
```

Parameters

`text` string

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

ReadOnlyMemory<GlyphMetrics>

A read-only memory region containing the per-glyph metrics entries of the text if it was to be rendered.

GetGlyphMetrics(ushort, GlyphOptions)

Gets the positioned metrics of a single glyph, identified by its glyph id, in pixel units.

```
public static GlyphMetrics GetGlyphMetrics(ushort glyphId, GlyphOptions options)
```

Parameters

`glyphId` ushort

The glyph identifier within the font face referenced by `options`.

`options` GlyphOptions

The glyph options, including the font, origin, and layout mode.

Returns

GlyphMetrics

The positioned metrics entry of the glyph if it was to be rendered.

Remarks

The metrics are computed directly from the font's cached per-glyph metrics without decoding outlines or running the layout engine, positioned at Origin. The entry's grapheme index is GraphemeIndex and its string index is `0`: glyph ids carry no source text, so the index identifies the

glyph within the measured input instead. A glyph the font does not contain, or one that never renders, produces an entry with empty rectangles.

GetGraphemeMetrics(ReadOnlySpan<char>, TextOptions)

Gets the positioned metrics of each laid-out grapheme in pixel units.

```
public static ReadOnlyMemory<GraphemeMetrics> GetGraphemeMetrics(ReadOnlySpan<char> text,
TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

ReadOnlyMemory<GraphemeMetrics>

A read-only memory region containing the per-grapheme metrics entries of the text if it was to be rendered.

GetGraphemeMetrics(string, TextOptions)

Gets the positioned metrics of each laid-out grapheme in pixel units.

```
public static ReadOnlyMemory<GraphemeMetrics> GetGraphemeMetrics(string text,
TextOptions options)
```

Parameters

`text` string

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

ReadOnlyMemory<GraphemeMetrics>

A read-only memory region containing the per-grapheme metrics entries of the text if it was to be rendered.

GetIntersections(TextShapingBuffer, GlyphOptions, float, float)

Gets the x-axis intervals where shaped glyph outlines cross a horizontal band, in pixel units.

```
public static ReadOnlyMemory<float> GetIntersections(TextShapingBuffer buffer, GlyphOptions
options, float lowerLimit, float upperLimit)
```

Parameters

`buffer` TextShapingBuffer

The buffer containing the shaped glyphs.

`options` GlyphOptions

The glyph options supplying the shaping font, resolution, and baseline origin.

`lowerLimit` float

One edge of the horizontal band.

`upperLimit` float

The other edge of the horizontal band.

Returns

ReadOnlyMemory<float>

A read-only memory region containing merged, x-sorted interval pairs (start, end, start, end, ...); empty when no outline crosses the band.

GetIntersections(ReadOnlySpan<char>, TextOptions, float, float)

Gets the x-axis intervals where the laid-out text's glyph outlines cross a horizontal band, in pixel units.

```
public static ReadOnlyMemory<float> GetIntersections(ReadOnlySpan<char> text, TextOptions
options, float lowerLimit, float upperLimit)
```

Parameters

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

`lowerLimit` float

One edge of the horizontal band.

`upperLimit` float

The other edge of the horizontal band.

Returns

ReadOnlyMemory<float>

A read-only memory region containing merged, x-sorted interval pairs (start, end, start, end, ...); empty when no outline crosses the band.

Remarks

The intervals are computed from the exact outline geometry the renderer would draw (including hinting), so text decorations can be broken precisely around descenders. Glyphs whose bounds

do not touch the band skip outline decoding entirely. The band and the returned x-values share the laid-out text's coordinate space.

GetIntersections(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions, float, float)

Gets the x-axis intervals where positioned glyph outlines cross a horizontal band, in pixel units.

```
public static ReadOnlyMemory<float> GetIntersections(ReadOnlySpan<ushort> glyphIds,
ReadOnlySpan<Vector2> points, GlyphOptions options, float lowerLimit, float upperLimit)
```

Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`options` GlyphOptions

The glyph options, including the font and layout mode.

`lowerLimit` float

One edge of the horizontal band.

`upperLimit` float

The other edge of the horizontal band.

Returns

ReadOnlyMemory<float>

A read-only memory region containing merged, x-sorted interval pairs (start, end, start, end, ...); empty when no outline crosses the band.

Remarks

The intervals are computed from the exact outline geometry the renderer would draw (including hinting), each glyph at its own run origin, so text decorations can be broken precisely around descenders. Glyphs whose bounds do not touch the band skip outline decoding entirely. The band and the returned x-values share the points' coordinate space.

GetIntersections(string, TextOptions, float, float)

Gets the x-axis intervals where the laid-out text's glyph outlines cross a horizontal band, in pixel units.

```
public static ReadOnlyMemory<float> GetIntersections(string text, TextOptions options, float
lowerLimit, float upperLimit)
```

Parameters

`text` string

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

`lowerLimit` float

One edge of the horizontal band.

`upperLimit` float

The other edge of the horizontal band.

Returns

ReadOnlyMemory<float>

A read-only memory region containing merged, x-sorted interval pairs (start, end, start, end, ...); empty when no outline crosses the band.

Remarks

The intervals are computed from the exact outline geometry the renderer would draw (including hinting), so text decorations can be broken precisely around descenders. Glyphs whose bounds

do not touch the band skip outline decoding entirely. The band and the returned x-values share the laid-out text's coordinate space.

GetIntersections(ushort, GlyphOptions, float, float)

Gets the x-axis intervals where a single glyph's outline, identified by its glyph id, crosses a horizontal band, in pixel units.

```
public static ReadOnlyMemory<float> GetIntersections(ushort glyphId, GlyphOptions options,
float lowerLimit, float upperLimit)
```

Parameters

`glyphId` ushort

The glyph identifier within the font face referenced by `options`.

`options` GlyphOptions

The glyph options, including the font, origin, and layout mode.

`lowerLimit` float

One edge of the horizontal band.

`upperLimit` float

The other edge of the horizontal band.

Returns

ReadOnlyMemory<float>

A read-only memory region containing merged, x-sorted interval pairs (start, end, start, end, ...); empty when the outline does not cross the band.

Remarks

The intervals are computed from the exact outline geometry the renderer would draw (including hinting) at Origin. The band and the returned x-values share the glyph origin's coordinate space.

GetLineMetrics(ReadOnlySpan<char>, TextOptions)

Gets per-line layout metrics for the supplied text.

```
public static ReadOnlyMemory<LineMetrics> GetLineMetrics(ReadOnlySpan<char> text,
TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text to measure.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

ReadOnlyMemory<LineMetrics>

A read-only memory region containing LineMetrics in pixel units, one entry per laid-out line.

GetLineMetrics(string, TextOptions)

Gets per-line layout metrics for the supplied text.

```
public static ReadOnlyMemory<LineMetrics> GetLineMetrics(string text, TextOptions options)
```

Parameters

`text` string

The text to measure.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

ReadOnlyMemory<LineMetrics>

A read-only memory region containing LineMetrics in pixel units, one entry per laid-out line.

GetWordMetrics(ReadOnlySpan<char>, TextOptions)

Gets the positioned metrics of each Unicode word-boundary segment in pixel units.

```
public static ReadOnlyMemory<WordMetrics> GetWordMetrics(ReadOnlySpan<char> text,
TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

ReadOnlyMemory<WordMetrics>

A read-only memory region containing the per-word-boundary segment metrics entries of the text if it was to be rendered.

GetWordMetrics(string, TextOptions)

Gets the positioned metrics of each Unicode word-boundary segment in pixel units.

```
public static ReadOnlyMemory<WordMetrics> GetWordMetrics(string text, TextOptions options)
```

Parameters

`text` string

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

ReadOnlyMemory<WordMetrics>

A read-only memory region containing the per-word-boundary segment metrics entries of the text if it was to be rendered.

Measure(ReadOnlySpan<char>, TextOptions)

Measures the full set of layout metrics for the supplied text in a single pass.

```
public static TextMetrics Measure(ReadOnlySpan<char> text, TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

TextMetrics

A TextMetrics instance containing every measurement for the laid-out text.

Measure(string, TextOptions)

Measures the full set of layout metrics for the supplied text in a single pass.

```
public static TextMetrics Measure(string text, TextOptions options)
```

Parameters

`text` string

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

TextMetrics

A TextMetrics instance containing every measurement for the laid-out text.

MeasureAdvance(TextShapingBuffer, GlyphOptions)

Measures the logical advance of shaped glyphs in pixel units.

```
public static FontRectangle MeasureAdvance(TextShapingBuffer buffer, GlyphOptions options)
```

Parameters

`buffer` TextShapingBuffer

The buffer containing the shaped glyphs.

`options` GlyphOptions

The glyph options supplying the shaping font, resolution, and baseline origin.

Returns

FontRectangle

The zero-based logical advance extent of the glyphs if they were to be rendered, or Empty when no glyph participates in rendering.

MeasureAdvance(ReadOnlySpan<char>, TextOptions)

Measures the logical advance of the text in pixel units.

```
public static FontRectangle MeasureAdvance(ReadOnlySpan<char> text, TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

FontRectangle

The logical advance rectangle of the text if it was to be rendered.

MeasureAdvance(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions)

Measures the logical advance of positioned glyphs in pixel units.

```
public static FontRectangle MeasureAdvance(ReadOnlySpan<ushort> glyphIds,
ReadOnlySpan<Vector2> points, GlyphOptions options)
```

Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`options` GlyphOptions

The glyph options, including the font and layout mode.

Returns

FontRectangle

The zero-based logical advance extent of the glyphs if they were to be rendered, or Empty when no glyph participates in rendering.

Remarks

Matching the text-level advance contract, the rectangle is zero-based: the extent the glyph advance cells cover at their points, reported independent of position and of TextBaseline. Glyph ids the font does not contain and glyphs that never render are skipped, matching renderer behavior.

MeasureAdvance(string, TextOptions)

Measures the logical advance of the text in pixel units.

```
public static FontRectangle MeasureAdvance(string text, TextOptions options)
```

Parameters

`text` string

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

FontRectangle

The logical advance rectangle of the text if it was to be rendered.

MeasureAdvance(ushort, GlyphOptions)

Measures the logical advance of a single glyph, identified by its glyph id, in pixel units.

```
public static FontRectangle MeasureAdvance(ushort glyphId, GlyphOptions options)
```

Parameters

`glyphId` ushort

The glyph identifier within the font face referenced by `options`.

`options` GlyphOptions

The glyph options, including the font, origin, and layout mode.

Returns

FontRectangle

The logical advance rectangle of the glyph if it was to be rendered, or Empty when the font does not contain the glyph or the glyph never renders.

Remarks

The advance is computed directly from the font's cached per-glyph metrics without decoding outlines or running the layout engine. Matching the text-level advance contract, the rectangle is zero-based: the advance width by the em height for horizontal layouts, and the advance the layout direction consumes for vertical layouts, independent of Origin and TextBaseline. Positioned geometry is reported by the bounds overloads.

MeasureBounds(TextShapingBuffer, GlyphOptions)

Measures the union of rendered bounds for shaped glyphs in pixel units.

```
public static FontRectangle MeasureBounds(TextShapingBuffer buffer, GlyphOptions options)
```

Parameters

`buffer` TextShapingBuffer

The buffer containing the shaped glyphs.

`options` GlyphOptions

The glyph options supplying the shaping font, resolution, and baseline origin.

Returns

FontRectangle

The union of the rendered glyph bounds if they were to be rendered, or Empty when no glyph participates in rendering.

MeasureBounds(ReadOnlySpan<char>, TextOptions)

Measures the rendered glyph bounds of the text in pixel units.

```
public static FontRectangle MeasureBounds(ReadOnlySpan<char> text, TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

FontRectangle

The rendered glyph bounds of the text if it was to be rendered.

MeasureBounds(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions)

Measures the union of rendered glyph bounds for positioned glyphs in pixel units.

```
public static FontRectangle MeasureBounds(ReadOnlySpan<ushort> glyphIds,
ReadOnlySpan<Vector2> points, GlyphOptions options)
```

Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`options` GlyphOptions

The glyph options, including the font and layout mode.

Returns

FontRectangle

The union of the rendered glyph bounds if they were to be rendered, or Empty when no glyph participates in rendering.

Remarks

Each glyph is measured at its supplied point exactly as RenderTo(IGlyphRenderer, ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions) renders it; Origin is replaced per glyph and restored. The result matches the union of the bounding boxes the renderer reports for the same glyphs and options. Glyph ids the font does not contain and glyphs that never render are skipped, matching renderer behavior.

MeasureBounds(string, TextOptions)

Measures the rendered glyph bounds of the text in pixel units.

```
public static FontRectangle MeasureBounds(string text, TextOptions options)
```

Parameters

`text` string

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

FontRectangle

The rendered glyph bounds of the text if it was to be rendered.

MeasureBounds(ushort, GlyphOptions)

Measures the rendered bounds of a single glyph, identified by its glyph id, in pixel units.

```
public static FontRectangle MeasureBounds(ushort glyphId, GlyphOptions options)
```

Parameters

`glyphId` ushort

The glyph identifier within the font face referenced by `options`.

`options` GlyphOptions

The glyph options, including the font, origin, and layout mode.

Returns

FontRectangle

The rendered bounds of the glyph at Origin if it was to be rendered, or Empty when the font does not contain the glyph or the glyph never renders.

Remarks

The bounds are computed directly from the font's cached per-glyph metrics without decoding outlines or running the layout engine, and are identical to the bounding box the renderer reports for the same glyph and options through RenderTo(IGlyphRenderer, ushort, GlyphOptions).

MeasureRenderableBounds(TextShapingBuffer, GlyphOptions)

Measures the full renderable bounds of shaped glyphs in pixel units.

```
public static FontRectangle MeasureRenderableBounds(TextShapingBuffer buffer,
GlyphOptions options)
```

Parameters

`buffer` TextShapingBuffer

The buffer containing the shaped glyphs.

`options` GlyphOptions

The glyph options supplying the shaping font, resolution, and baseline origin.

Returns

FontRectangle

The union of the positioned advances and rendered glyph bounds, or Empty when no glyph participates in rendering.

MeasureRenderableBounds(ReadOnlySpan<char>, TextOptions)

Measures the full renderable bounds of the text in pixel units.

```
public static FontRectangle MeasureRenderableBounds(ReadOnlySpan<char> text,
TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

FontRectangle

The union of the logical advance rectangle and the rendered glyph bounds if the text was to be rendered.

MeasureRenderableBounds(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions)

Measures the full renderable bounds of positioned glyphs in pixel units.

```
public static FontRectangle MeasureRenderableBounds(ReadOnlySpan<ushort> glyphIds,
ReadOnlySpan<Vector2> points, GlyphOptions options)
```

Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`options` GlyphOptions

The glyph options, including the font and layout mode.

Returns

FontRectangle

The union of the advances placed at the supplied points and the rendered glyph bounds, or Empty when no glyph participates in rendering.

MeasureRenderableBounds(string, TextOptions)

Measures the full renderable bounds of the text in pixel units.

```
public static FontRectangle MeasureRenderableBounds(string text, TextOptions options)
```

Parameters

`text` string

The text.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Returns

FontRectangle

The union of the logical advance rectangle and the rendered glyph bounds if the text was to be rendered.

MeasureRenderableBounds(ushort, GlyphOptions)

Measures the full renderable bounds of a single glyph, identified by its glyph id, in pixel units.

```
public static FontRectangle MeasureRenderableBounds(ushort glyphId, GlyphOptions options)
```

Parameters

`glyphId` ushort

The glyph identifier within the font face referenced by `options`.

`options` GlyphOptions

The glyph options, including the font, origin, and layout mode.

Returns

FontRectangle

The union of the advance placed at Origin and the rendered bounds of the glyph if it was to be rendered, or Empty when the font does not contain the glyph or the glyph never renders.

Remarks

The bounds are computed directly from the font's cached per-glyph metrics without decoding outlines or running the layout engine.

Class TextMetrics

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Encapsulates the full set of measurement results for laid-out text.

```
public sealed class TextMetrics
```

Inheritance

object ← TextMetrics

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Advance

Gets the logical advance rectangle of the text in pixel units.

```
public FontRectangle Advance { get; }
```

Property Value

FontRectangle

Remarks

Reflects line-box height and horizontal or vertical text advance from the layout model. Does not guarantee that all rendered glyph pixels fit within the returned rectangle.

Bounds

Gets the rendered glyph bounds of the text in pixel units.

```
public FontRectangle Bounds { get; }
```

Property Value

FontRectangle

Remarks

This is the tight ink bounds enclosing all rendered glyphs and may be smaller or larger than the logical advance. May have a non-zero origin.

GraphemeMetrics

Gets the grapheme metrics entries in final layout order.

```
public ReadOnlySpan<GraphemeMetrics> GraphemeMetrics { get; }
```

Property Value

ReadOnlySpan<GraphemeMetrics>

LineCount

Gets the number of laid-out lines in the text.

```
public int LineCount { get; }
```

Property Value

int

LineMetrics

Gets the per-line layout metrics for the text.

```
public ReadOnlySpan<LineMetrics> LineMetrics { get; }
```

Property Value

ReadOnlySpan<LineMetrics>

RenderableBounds

Gets the union of the logical advance rectangle (positioned at the text options origin) and the rendered glyph bounds in pixel units.

```
public FontRectangle RenderableBounds { get; }
```

Property Value

FontRectangle

Remarks

Use this rectangle when both typographic advance and rendered glyph overshoot must fit within the same bounding box.

WordMetrics

Gets the word-boundary segment metrics in source order.

```
public ReadOnlySpan<WordMetrics> WordMetrics { get; }
```

Property Value

ReadOnlySpan<WordMetrics>

Methods

GetCaret(CaretPlacement)

Gets an absolute caret position in the laid-out text.

```
public CaretPosition GetCaret(CaretPlacement placement)
```

Parameters

`placement` CaretPlacement

The absolute caret placement.

Returns

CaretPosition

The caret position in pixel units.

GetCaretPosition(TextHit)

Gets the caret position for the supplied hit.

```
public CaretPosition GetCaretPosition(TextHit hit)
```

Parameters

`hit` TextHit

The hit-tested grapheme position.

Returns

CaretPosition

The caret position in pixel units.

GetGlyphMetrics()

Gets the positioned metrics of each laid-out glyph entry.

```
public ReadOnlyMemory<GlyphMetrics> GetGlyphMetrics()
```

Returns

ReadOnlyMemory<GlyphMetrics>

A read-only memory region containing per-glyph metrics entries.

GetSelectionBounds(CaretPosition, CaretPosition)

Gets selection bounds between two caret positions.

```
public ReadOnlyMemory<FontRectangle> GetSelectionBounds(CaretPosition anchor,
CaretPosition focus)
```

Parameters

`anchor` CaretPosition

The fixed selection endpoint.

`focus` CaretPosition

The active selection endpoint.

Returns

ReadOnlyMemory<FontRectangle>

A read-only memory region containing the selection bounds in visual order and pixel units.

GetSelectionBounds(GraphemeMetrics)

Gets selection bounds for the supplied grapheme metrics.

```
public ReadOnlyMemory<FontRectangle> GetSelectionBounds(GraphemeMetrics metrics)
```

Parameters

`metrics` GraphemeMetrics

The grapheme metrics to select.

Returns

ReadOnlyMemory<FontRectangle>

A read-only memory region containing the selection bounds in visual order and pixel units.

GetSelectionBounds(TextHit, TextHit)

Gets selection bounds between two hit-tested grapheme positions.

```
public ReadOnlyMemory<FontRectangle> GetSelectionBounds(TextHit anchor, TextHit focus)
```

Parameters

`anchor` TextHit

The fixed selection endpoint.

`focus` TextHit

The active selection endpoint.

Returns

ReadOnlyMemory<FontRectangle>

A read-only memory region containing the selection bounds in visual order and pixel units.

GetSelectionBounds(WordMetrics)

Gets selection bounds for the supplied word metrics.

```
public ReadOnlyMemory<FontRectangle> GetSelectionBounds(WordMetrics metrics)
```

Parameters

`metrics` WordMetrics

The word metrics to select.

Returns

ReadOnlyMemory<FontRectangle>

A read-only memory region containing the selection bounds in visual order and pixel units.

GetWordMetrics(CaretPosition)

Gets the word metrics for the word-boundary segment containing the supplied caret position.

```
public WordMetrics GetWordMetrics(CaretPosition caret)
```

Parameters

`caret` CaretPosition

The caret position.

Returns

WordMetrics

The word metrics containing the caret's grapheme insertion index.

GetWordMetrics(TextHit)

Gets the word metrics for the word-boundary segment containing the supplied hit-tested grapheme position.

```
public WordMetrics GetWordMetrics(TextHit hit)
```

Parameters

`hit` TextHit

The hit-tested grapheme position.

Returns

WordMetrics

The word metrics containing the hit grapheme.

HitTest(Vector2)

Hit tests the supplied point against the laid-out grapheme advance bounds.

```
public TextHit HitTest(Vector2 point)
```

Parameters

`point` Vector2

The point in pixel units.

Returns

TextHit

The hit-tested grapheme position.

MoveCaret(CaretPosition, CaretMovement)

Moves the supplied caret by the requested operation.

```
public CaretPosition MoveCaret(CaretPosition caret, CaretMovement movement)
```

Parameters

`caret` CaretPosition

The current caret position.

`movement` CaretMovement

The movement operation.

Returns

CaretPosition

The moved caret position in pixel units.

Class TextOptions

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Provides configuration options for rendering and shaping text.

```
public class TextOptions
```

Inheritance

object ← TextOptions

Derived

RichTextOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TextOptions(Font)

Initializes a new instance of the TextOptions class.

```
public TextOptions(Font font)
```

Parameters

`font` Font

The font.

TextOptions(TextOptions)

Initializes a new instance of the TextOptions class from properties copied from the given instance.

```
public TextOptions(TextOptions options)
```

Parameters

`options` TextOptions

The options whose properties are copied into this instance.

Properties

BaselineOffset

Gets or sets an additional shift of the text away from its anchored position, in pixel units along the block flow axis. Positive values shift toward the text's over side: upward for horizontal layouts, toward the over column side for vertical layouts, and away from the line along its normal when the text follows a path.

```
public float BaselineOffset { get; set; }
```

Property Value

float

Remarks

The shift composes with TextBaseline and moves rendered glyphs, ink bounds, and decorations as a unit. The logical advance is unaffected, matching the CSS and SVG `baseline-shift` model.

ColorFontSupport

Gets or sets the color font support options.

```
public ColorFontSupport ColorFontSupport { get; set; }
```

Property Value

ColorFontSupport

Remarks

A text run with a non-null ColorFontSupport value replaces this value over that run.

Culture

Gets or sets the culture used to select language specific glyph shaping.

```
public CultureInfo? Culture { get; set; }
```

Property Value

CultureInfo

Remarks

Fonts can substitute and position glyphs differently per language, for example the Turkish dotless i or Serbian italic letterforms. When set, shaping selects the matching language system within each script's OpenType feature tables, falling back to the font's default language system when the font does not distinguish the language. When null the current culture applies, matching the ambient language model of the reference shaping engines; use InvariantCulture to express no language preference and always shape with the font's default language system.

CustomEllipsis

Gets or sets the ellipsis marker to use when TextEllipsis is `Custom`.

```
public CodePoint? CustomEllipsis { get; set; }
```

Property Value

CodePoint?

CustomHyphen

Gets or sets the hyphenation marker to use when TextHyphenation is `Custom`.

```
public CodePoint? CustomHyphen { get; set; }
```

Property Value

CodePoint?

DecorationPositioningMode

Gets or sets the positioning mode used for rendering decorations.

```
public DecorationPositioningMode DecorationPositioningMode { get; set; }
```

Property Value

DecorationPositioningMode

Dpi

Gets or sets the DPI (Dots Per Inch) to render/measure the text at.

Defaults to `72F`.

```
public float Dpi { get; set; }
```

Property Value

float

FallbackFontFamilies

Gets or sets the collection of fallback font families to use when a specific glyph is missing from Font.

```
public IReadOnlyList<FontFamily> FallbackFontFamilies { get; set; }
```

Property Value

IReadOnlyList<FontFamily>

FeatureTags

Gets or sets the collection of additional feature tags to apply during glyph shaping.

```
public IReadOnlyList<Tag> FeatureTags { get; set; }
```

Property Value

IReadOnlyList<Tag>

Remarks

A text run with a non-null FeatureTags collection replaces this collection over that run.

Font

Gets or sets the font.

```
public Font Font { get; set; }
```

Property Value

Font

FontFallbackResolver

Gets or sets the resolver consulted for code points that neither Font nor any FallbackFontFamilies entry can shape, or null to leave such code points rendered as the missing-glyph outline.

```
public IFontFallbackResolver? FontFallbackResolver { get; set; }
```

Property Value

IFontFallbackResolver

Remarks

Resolution happens per distinct unresolved code point after every configured font has attempted the text. FallbackResolver selects from the fonts installed on the current machine, so rendered output can differ between machines.

FontPalette

Gets or sets the color palette selection used to resolve COLR glyph colors, or null to use the font's default palette.

```
public FontPalette? FontPalette { get; set; }
```

Property Value

FontPalette

Remarks

A text run with a non-null FontPalette value replaces this value over that run.

FontWeight

Gets or sets the font weight, or null to use the weight implied by Font.

```
public FontWeight? FontWeight { get; set; }
```

Property Value

FontWeight?

HintingMode

Gets or sets a value indicating whether to apply hinting - The use of mathematical instructions to adjust the display of an outline font so that it lines up with a rasterized grid.

```
public HintingMode HintingMode { get; set; }
```

Property Value

HintingMode

HorizontalAlignment

Gets or sets the horizontal alignment of the text box.

```
public HorizontalAlignment HorizontalAlignment { get; set; }
```

Property Value

HorizontalAlignment

KerningMode

Gets or sets the kerning mode indicating whether to apply kerning (character spacing adjustments) to the glyph positions from information found within the font.

```
public KerningMode KerningMode { get; set; }
```

Property Value

KerningMode

LayoutMode

Gets or sets the layout mode for the text lines.

```
public LayoutMode LayoutMode { get; set; }
```

Property Value

LayoutMode

LineSpacing

Gets or sets the line spacing. Applied as a multiple of the line height.

Defaults to `1F`.

```
public float LineSpacing { get; set; }
```

Property Value

float

MaxLines

Gets or sets the maximum number of lines to lay out.

```
public int MaxLines { get; set; }
```

Property Value

int

Remarks

If value is -1 then the number of lines is unlimited.

Origin

Gets or sets the rendering origin.

```
public Vector2 Origin { get; set; }
```

Property Value

Vector2

Script

Gets or sets the script applied to the whole shaping request, or null when script runs are inferred from the text.

```
public ScriptClass? Script { get; set; }
```

Property Value

ScriptClass?

TabWidth

Gets or sets the width of the tab. Measured as the distance in spaces (U+0020).

```
public float TabWidth { get; set; }
```

Property Value

float

Remarks

If value is -1 then the font default tab width is used.

TextAlignment

Gets or sets the text alignment of the text within the box.

```
public TextAlignment TextAlignment { get; set; }
```

Property Value

TextAlignment

TextBaseline

Gets or sets which reference line of the first laid-out line is placed at Origin along the block flow axis.

```
public TextBaseline TextBaseline { get; set; }
```

Property Value

TextBaseline

Remarks

Baseline positions derive from the metrics of Font: horizontal layouts anchor along Y from the alphabetic baseline, vertical layouts along X from the central column axis. When the value is not LineBox the block alignment along the flow axis does not apply; additional wrapped lines stack relative to the anchored first line.

TextBidiMode

Gets or sets how bidirectional text is resolved.

```
public TextBidiMode TextBidiMode { get; set; }
```

Property Value

TextBidiMode

TextDecorationSkipInk

Gets or sets a value indicating whether underline and overline decorations skip over glyph ink, leaving gaps around descenders and ascenders that cross the decoration. Defaults to Auto.

```
public TextDecorationSkipInk TextDecorationSkipInk { get; set; }
```

Property Value

TextDecorationSkipInk

TextDirection

Gets or sets the text direction.

```
public TextDirection TextDirection { get; set; }
```

Property Value

TextDirection

TextEllipsis

Gets or sets the ellipsis behavior to use when laid-out text is limited to a maximum number of lines.

```
public TextEllipsis TextEllipsis { get; set; }
```

Property Value

TextEllipsis

TextHyphenation

Gets or sets the hyphenation marker behavior to use when text breaks at hyphenation opportunities.

```
public TextHyphenation TextHyphenation { get; set; }
```

Property Value

TextHyphenation

TextInteractionMode

Gets or sets how caret movement and selection model trailing breaking whitespace.

```
public TextInteractionMode TextInteractionMode { get; set; }
```

Property Value

TextInteractionMode

TextJustification

Gets or sets the justification of the text within the box.

```
public TextJustification TextJustification { get; set; }
```

Property Value

TextJustification

TextRuns

Gets or sets an optional collection of text runs to apply to the body of text.

```
public IReadOnlyList<TextRun> TextRuns { get; set; }
```

Property Value

IReadOnlyList<TextRun>

Tracking

Gets or sets the tracking (letter-spacing) value. Tracking adjusts the spacing between all characters uniformly and is measured in em. Positive values increase spacing, negative values decrease spacing, and zero applies no adjustment.

```
public float Tracking { get; set; }
```

Property Value

float

VerticalAlignment

Gets or sets the vertical alignment of the text box.

```
public VerticalAlignment VerticalAlignment { get; set; }
```

Property Value

VerticalAlignment

VisibleBounds

Gets or sets the visible region in pixel units (px) used when rendering text. Whole lines that fall outside the region are skipped and the remaining lines are positioned as if all lines had been rendered.

```
public FontRectangle? VisibleBounds { get; set; }
```

Property Value

FontRectangle?

Remarks

If value is null then culling is disabled.

WordBreaking

Gets or sets the word breaking mode to use when wrapping text.

```
public WordBreaking WordBreaking { get; set; }
```

Property Value

WordBreaking

WrappingLength

Gets or sets the length in pixel units (px) at which text will automatically wrap onto a new line. This property also affects the width or height (depending on the LayoutMode) of the text box for alignment of text.

```
public float WrappingLength { get; set; }
```

Property Value

float

Remarks

If value is -1 then wrapping is disabled.

Struct TextPlaceholder

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents an atomic inline placeholder with caller-supplied dimensions.

```
public readonly struct TextPlaceholder
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

TextPlaceholder(float, float, TextPlaceholderAlignment, float)

Initializes a new instance of the TextPlaceholder struct.

```
public TextPlaceholder(float width, float height, TextPlaceholderAlignment alignment,
float baselineOffset)
```

Parameters

`width` float

The placeholder width in pixel units.

`height` float

The placeholder height in pixel units.

`alignment` TextPlaceholderAlignment

The placeholder alignment against surrounding text.

`baselineOffset` float

The distance from the placeholder top edge to its baseline in pixel units.

Properties

Alignment

Gets the placeholder alignment against surrounding text.

```
public TextPlaceholderAlignment Alignment { get; }
```

Property Value

TextPlaceholderAlignment

BaselineOffset

Gets the distance from the placeholder top edge to its baseline in pixel units.

```
public float BaselineOffset { get; }
```

Property Value

float

Height

Gets the placeholder height in pixel units.

```
public float Height { get; }
```

Property Value

float

Width

Gets the placeholder width in pixel units.

```
public float Width { get; }
```

Property Value

float

Enum TextPlaceholderAlignment

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Specifies how an inline placeholder is aligned against surrounding text.

```
public enum TextPlaceholderAlignment
```

Fields

`AboveBaseline = 1`

Align the placeholder above the surrounding text baseline.

`Baseline = 0`

Align the placeholder baseline with the surrounding text baseline.

`BelowBaseline = 2`

Align the placeholder below the surrounding text baseline.

`Bottom = 4`

Align the placeholder with the bottom of the surrounding line box.

`Middle = 5`

Align the placeholder with the middle of the surrounding line box.

`Top = 3`

Align the placeholder with the top of the surrounding line box.

Class TextRun

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents a run of text spanning a series of graphemes within a string.

```
public class TextRun
```

Inheritance

object ← TextRun

Derived

RichTextRun

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Properties

ColorFontSupport

Gets or sets the color font support options for this run, or null to use those supplied by the owning text options.

```
public ColorFontSupport? ColorFontSupport { get; set; }
```

Property Value

ColorFontSupport?

Culture

Gets or sets the culture used to select language-specific shaping for this run, or null to use the culture supplied by the owning text options.

```
public CultureInfo? Culture { get; set; }
```

Property Value

CultureInfo

End

Gets or sets the exclusive end index of the last grapheme in this TextRun.

```
public int End { get; set; }
```

Property Value

int

FeatureTags

Gets or sets the additional shaping features for this run, or null to use those supplied by the owning text options. A non-null empty collection applies no additional features to the run.

```
public IReadOnlyList<Tag>? FeatureTags { get; set; }
```

Property Value

IReadOnlyList<Tag>

Font

Gets or sets the font for this run.

```
public Font? Font { get; set; }
```

Property Value

Font

FontPalette

Gets or sets the color palette selection used to resolve COLR glyph colors for this run, or null to use the palette supplied by the owning text options.

```
public FontPalette? FontPalette { get; set; }
```

Property Value

FontPalette

FontWeight

Gets or sets the font weight for this run, or null to use the weight supplied by the owning text options.

```
public FontWeight? FontWeight { get; set; }
```

Property Value

FontWeight?

Placeholder

Gets or sets the inline placeholder represented by this run.

```
public TextPlaceholder? Placeholder { get; set; }
```

Property Value

TextPlaceholder?

Remarks

Placeholder runs are inserted at Start and must have End equal to Start.

Script

Gets or sets the script to use for this run, or null to use the script selection supplied by the owning text options.

```
public ScriptClass? Script { get; set; }
```

Property Value

ScriptClass?

Start

Gets or sets the inclusive start index of the first grapheme in this TextRun.

```
public int Start { get; set; }
```

Property Value

int

TextAttributes

Gets or sets the text attributes applied to this run.

```
public TextAttributes TextAttributes { get; set; }
```

Property Value

TextAttributes

TextDecorations

Gets or sets the text decorations applied to this run.

```
public TextDecorations TextDecorations { get; set; }
```

Property Value

TextDecorations

Methods

GetDecorationOptions(TextDecorations)

Gets the geometry overrides to apply to the specified decoration, or null to use the values derived from the font's metrics.

```
public virtual TextDecorationOptions? GetDecorationOptions(TextDecorations decoration)
```

Parameters

`decoration` TextDecorations

The decoration to retrieve overrides for.

Returns

TextDecorationOptions

The TextDecorationOptions for the decoration, or null.

Remarks

The base implementation returns null for every decoration. Derived runs can override this to supply values sourced from styling that the font metrics do not describe, such as an explicit stroke width.

Slice(ReadOnlySpan<char>)

Returns the slice of the given text representing this TextRun.

```
public ReadOnlySpan<char> Slice(ReadOnlySpan<char> text)
```

Parameters

`text` ReadOnlySpan<char>

The text to slice.

Returns

ReadOnlySpan<char>

The ReadOnlySpan<T>.

ToString()

Returns a string that represents the current object.

```
public override string ToString()
```

Returns

string

A string that represents the current object.

Class TextShaper

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Encapsulates logic for shaping text into a positioned glyph stream.

```
public static class TextShaper
```

Inheritance

object ← TextShaper

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The text and its properties are set on a TextShapingBuffer, the buffer is shaped against a font, and the glyphs are read back from it. Shaping applies the font's substitution and positioning features but does not wrap the text.

Shape(Font, TextShapingBuffer) treats the text as unwrapped logical lines separated by hard breaks and resolves mixed-direction text within each line. ShapeRun(Font, TextShapingBuffer) shapes one directional run that has already been selected by the caller.

Advances and offsets are scaled to the supplied font's size.

Methods

Shape(Font, TextShapingBuffer)

Shapes the buffer's text as unwrapped logical lines separated by hard breaks, replacing the buffer's glyphs with each line's visually ordered glyphs.

```
public static void Shape(Font font, TextShapingBuffer buffer)
```

Parameters

`font` Font

The font to shape against.

`buffer` TextShapingBuffer

The buffer holding the line, which receives the glyphs.

Shape(Font, TextShapingBuffer, Tag[])

Shapes the buffer's text as unwrapped logical lines separated by hard breaks with the given features turned on, replacing the buffer's glyphs with each line's visually ordered glyphs.

```
public static void Shape(Font font, TextShapingBuffer buffer, Tag[] features)
```

Parameters

`font` Font

The font to shape against.

`buffer` TextShapingBuffer

The buffer holding the line, which receives the glyphs.

`features` Tag[]

The feature tags to turn on for the line.

ShapeRun(Font, TextShapingBuffer)

Shapes the buffer's text as one directional run, replacing the buffer's glyphs with the glyphs in reading order.

```
public static void ShapeRun(Font font, TextShapingBuffer buffer)
```

Parameters

`font` Font

The font to shape against.

`buffer` TextShapingBuffer

The buffer holding the run, which receives the glyphs.

ShapeRun(Font, TextShapingBuffer, Tag[])

Shapes the buffer's text as one directional run with the given features turned on, replacing the buffer's glyphs with the glyphs in reading order.

```
public static void ShapeRun(Font font, TextShapingBuffer buffer, Tag[] features)
```

Parameters

`font` Font

The font to shape against.

`buffer` TextShapingBuffer

The buffer holding the run, which receives the glyphs.

`features` Tag[]

The feature tags to turn on for the run.

Class TextShapingBuffer

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

A reusable, caller-owned buffer holding text, the properties it is shaped under, and the glyphs shaping produces. Text and properties are set on the buffer, the buffer is shaped, and the glyphs are read back from it. Each shaping call replaces the glyphs.

```
public sealed class TextShapingBuffer
```

Inheritance

object ← TextShapingBuffer

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Shape(Font, TextShapingBuffer) treats the text as unwrapped logical lines separated by hard breaks.

ShapeRun(Font, TextShapingBuffer) treats the text as one directional run. An instance is not thread safe.

Glyph identifiers and positions are specific to the Font and LayoutMode used while shaping. Pass that same font and layout mode in the GlyphOptions used to measure or render the shaped buffer.

Properties

Count

Gets the number of shaped glyphs the last shaping call produced.

```
public int Count { get; }
```

Property Value

int

Glyphs

Gets the shaped glyphs in visual order within each logical line, or reading order for a directional run.

```
public ReadOnlySpan<ShapedGlyph> Glyphs { get; }
```

Property Value

ReadOnlySpan<ShapedGlyph>

this[int]

Gets a read-only reference to the shaped glyph at the given index.

```
public ref readonly ShapedGlyph this[int index] { get; }
```

Parameters

`index` int

The zero-based glyph index.

Property Value

ShapedGlyph

KerningMode

Gets or sets the kerning mode used while positioning glyphs. Defaults to Standard.

```
public KerningMode KerningMode { get; set; }
```

Property Value

KerningMode

Language

Gets or sets the language the run is written in, which selects the language specific behaviour of the font's features.

```
public CultureInfo Language { get; set; }
```

Property Value

CultureInfo

LayoutMode

Gets or sets the layout mode used to select horizontal or vertical shaping. Defaults to Horizontal TopBottom.

```
public LayoutMode LayoutMode { get; set; }
```

Property Value

LayoutMode

LineEnds

Gets the glyph-end indices of hard-delimited lines. An empty span represents one directional run or one logical line.

```
public ReadOnlySpan<int> LineEnds { get; }
```

Property Value

ReadOnlySpan<int>

Script

Gets or sets the script to use for the entire shaping request, or null to infer scripts from the text.

```
public ScriptClass? Script { get; set; }
```

Property Value

ScriptClass?

Text

Gets the text of the run.

```
public ReadOnlySpan<char> Text { get; }
```

Property Value

ReadOnlySpan<char>

TextDirection

Gets or sets the base direction of a logical line, or the direction of a directional run. Defaults to Auto.

```
public TextDirection TextDirection { get; set; }
```

Property Value

TextDirection

Methods

Add(ReadOnlySpan<char>)

Replaces the text of the run, discarding any glyphs already shaped.

```
public void Add(ReadOnlySpan<char> value)
```

Parameters

`value` ReadOnlySpan<char>

The text of the run.

Add(string)

Replaces the text of the run, discarding any glyphs already shaped.

```
public void Add(string value)
```

Parameters

`value` string

The text of the run.

Clear()

Removes the text and the glyphs while retaining the storage.

```
public void Clear()
```

Enum VerticalAlignment

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Vertical alignment modes.

```
public enum VerticalAlignment
```

Fields

`Baseline = 3`

Aligns text to the baseline.

`Bottom = 2`

Aligns text upwards from the bottom.

`Center = 1`

Aligns text up and down from the middle.

`Top = 0`

Aligns downward from the top.

Class VerticalMetrics

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represent the metrics of a font face specific to vertical text.

```
public class VerticalMetrics : IMetricsHeader
```

Inheritance

object ← VerticalMetrics

Implements

IMetricsHeader

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AdvanceHeightMax

Gets the maximum advance height, in font units, for all glyphs in this face.This is only relevant for vertical layouts, and is set to LineHeight for fonts that do not provide vertical metrics.

```
public short AdvanceHeightMax { get; }
```

Property Value

short

AdvanceWidthMax

Gets the maximum advance width, in font units, for all glyphs in this face.

```
public short AdvanceWidthMax { get; }
```

Property Value

short

Ascender

Gets the typographic ascender of the face, expressed in font units.

```
public short Ascender { get; }
```

Property Value

short

Descender

Gets the typographic descender of the face, expressed in font units.

```
public short Descender { get; }
```

Property Value

short

LineGap

Gets the typographic line gap of the face, expressed in font units. This field should be combined with the Ascender and Descender values to determine default line spacing.

```
public short LineGap { get; }
```

Property Value

short

LineHeight

Gets the typographic line spacing of the face, expressed in font units.

```
public short LineHeight { get; }
```

Property Value

short

Enum WordBreaking

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Defines modes to determine when line breaks should appear when words overflow their content box.

```
public enum WordBreaking
```

Fields

`BreakAll = 1`

To prevent overflow, word breaks should be inserted between any two characters (excluding Chinese/Japanese/Korean text).

`BreakWord = 3`

Uses a combination of Standard and BreakAll rules in that order.

`KeepAll = 2`

Word breaks should not be used for Chinese/Japanese/Korean (CJK) text. Non-CJK text behavior is the same as for Standard

`Standard = 0`

Use the default line break rule.

Struct WordMetrics

Namespace: SixLabors.Fonts

Assembly: SixLabors.Fonts.dll

Represents the positioned metrics for one Unicode word-boundary segment.

```
public readonly struct WordMetrics
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

Advance

Gets the positioned logical advance rectangle for the word-boundary segment in pixel units.

```
public FontRectangle Advance { get; }
```

Property Value

FontRectangle

Bounds

Gets the rendered glyph bounds for the word-boundary segment in pixel units.

```
public FontRectangle Bounds { get; }
```

Property Value

FontRectangle

GraphemeEnd

Gets the exclusive grapheme insertion index where the word-boundary segment ends.

```
public int GraphemeEnd { get; }
```

Property Value

int

GraphemeStart

Gets the inclusive grapheme insertion index where the word-boundary segment starts.

```
public int GraphemeStart { get; }
```

Property Value

int

RenderableBounds

Gets the union of the positioned logical advance bounds and rendered glyph bounds in pixel units.

```
public FontRectangle RenderableBounds { get; }
```

Property Value

FontRectangle

StringEnd

Gets the exclusive UTF-16 index where the word-boundary segment ends.

```
public int StringEnd { get; }
```

Property Value

int

StringStart

Gets the inclusive UTF-16 index where the word-boundary segment starts.

```
public int StringStart { get; }
```

Property Value

int

Namespace SixLabors.Fonts.Rendering

Classes

GlyphRendererExtensions
: A surface that can have a glyph rendered to it as a series of actions.

LinearGradientPaint
: Linear gradient paint.

Paint
: Base type for normalized paint definitions that can be used by any renderer. Glyph sources must pre-apply all relevant transforms and resolve any palette or format-specific constructs before creating a paint instance.

PaintedGlyphMetrics
: Provides painted (layered) glyph rendering for color formats such as COLR v1 and OT-SVG. Geometry and paints are supplied in document-space by an interpreter; all layout transforms (UPEM mapping, DPI/point-size scaling, rotation, final placement) are applied here.

RadialGradientPaint
: Represents a radial gradient paint defined by two circles. The first circle is centered at Center0 with radius Radius0. The second circle is centered at Center1 with radius Radius1. The color transition is computed between these two circles. Compatible with two-circle radial gradients used by HTML Canvas and OpenType COLR v1.

SolidPaint
: Solid color paint (direct RGBA). Interpreters must resolve palettes to RGBA. Compatible with OT-SVG solid fills and COLR v1 PaintSolid after CPAL resolution.

SweepGradientPaint
: Sweep (conic) gradient paint. Angles are expressed in degrees in the renderer's y-down space.

TextRenderer
: Encapsulates logic for laying out and then rendering text to a IGlyphRenderer surface.

Structs

GlyphRendererParameters
: The combined set of properties that uniquely identify the glyph that is to be rendered at a particular size and dpi.

GradientStop
: Defines a color stop for gradient paints. Offsets must be clamped to the range [0, 1] by the interpreter. Colors are direct RGBA and must not reference palettes.

Interfaces

IGlyphRenderer
: A surface that can have a glyph rendered to it as a series of actions.

Enums

CompositeMode
: Defines compositing and blending operations used when combining source and destination colors.

  Values 0–12 correspond to standard Porter–Duff compositing modes. These determine how source and destination alpha interact to produce transparency. The remaining values (13–27) correspond to separable and non-separable blend modes used in modern graphics systems such as ImageSharp.

FillRule
: Specifies the fill rule for path rasterization.

GradientUnits
: Coordinate system to interpret gradient geometry.

SpreadMethod
: Specifies how a gradient should extend beyond the [0, 1] range.

Enum CompositeMode

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Defines compositing and blending operations used when combining source and destination colors.

Values 0–12 correspond to standard Porter–Duff compositing modes. These determine how source and destination alpha interact to produce transparency. The remaining values (13–27) correspond to separable and non-separable blend modes used in modern graphics systems such as ImageSharp.

```
public enum CompositeMode
```

Fields

`Clear = 0`

Clears both the source and destination. The output is fully transparent regardless of the input colors.

`Color = 26`

Combines the hue and saturation of the source with the luminosity of the destination.

`ColorBurn = 18`

Darkens the destination to reflect the source. Formula: `1 - (1 - D) / S`.

`ColorDodge = 17`

Brightens the destination to reflect the source. Formula: `D / (1 - S)`.

`Darken = 15`

Chooses the darker of source and destination values per color channel.

`Dest = 2`

Keeps the destination as-is and ignores the source. Equivalent to no drawing operation.

`DestAtop = 10`

Draws the destination over the source but only within the source's alpha region. Outside that region, the source is visible.

`DestIn = 6`

Shows the destination only where it overlaps the source. The source's alpha acts as a mask for the destination.

`DestOut = 8`

Shows the destination only where it does not overlap the source. Produces the inverse of Dest In.

`DestOver = 4`

Draws the destination over the source. The destination appears on top and the source shows through transparent areas.

`Difference = 21`

Subtracts darker colors from lighter ones to highlight differences. Often used for comparison or edge detection effects.

`Exclusion = 22`

Similar to Difference, but with reduced contrast. Midtones are preserved, producing a lower-contrast difference.

`HardLight = 19`

Applies overlay logic using the source's lightness. Used for strong highlight and shadow effects.

`Hue = 24`

Combines the hue of the source with the saturation and luminosity of the destination.

`Lighten = 16`

Chooses the lighter of source and destination values per color channel.

`Luminosity = 27`

Combines the luminosity of the source with the hue and saturation of the destination.

`Multiply = 23`

Multiplies source and destination colors. Always results in a darker composite.

`Overlay = 14`

Multiplies or screens colors depending on destination lightness. Preserves highlights and shadows while mixing source and destination tones.

`Plus = 12`

Adds the source and destination color values. Alpha is also added, producing a brightening effect.

`Saturation = 25`

Combines the saturation of the source with the hue and luminosity of the destination.

`Screen = 13`

Combines colors using an inverse multiply. Formula: `1 - (1 - S) × (1 - D)`. Produces a lighter result similar to photographic screen exposure.

`SoftLight = 20`

Similar to HardLight, but with reduced contrast. Produces a softer transition between tones.

`Src = 1`

Replaces the destination entirely with the source. The destination pixels are ignored.

`SrcAtop = 9`

Draws the source over the destination but only within the destination's alpha region. Outside that region, the destination remains unchanged.

`SrcIn = 5`

Shows the source only where it overlaps the destination. The destination's alpha acts as a mask for the source.

`SrcOut = 7`

Shows the source only where it does not overlap the destination. Produces the inverse of SrcIn.

`SrcOver = 3`

Draws the source over the destination using standard alpha compositing. The source appears on top and the destination shows through transparent areas.

`Xor = 11`

Exclusive OR. Shows the source and destination only where they do not overlap. Overlapping regions become transparent.

Enum FillRule

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Specifies the fill rule for path rasterization.

```
public enum FillRule
```

Fields

`EvenOdd = 1`

Even-odd rule.

`NonZero = 0`

Non-zero winding rule.

Class GlyphRendererExtensions

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

A surface that can have a glyph rendered to it as a series of actions.

```
public static class GlyphRendererExtensions
```

Inheritance

object ← GlyphRendererExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Render(IGlyphRenderer, ReadOnlySpan<char>, TextOptions)

Renders the text.

```
public static IGlyphRenderer Render(this IGlyphRenderer renderer, ReadOnlySpan<char> text,
TextOptions options)
```

Parameters

`renderer` IGlyphRenderer

The target renderer surface.

`text` ReadOnlySpan<char>

The text.

`options` TextOptions

The options.

Returns

IGlyphRenderer

Returns the original `renderer`

Struct GlyphRendererParameters

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

The combined set of properties that uniquely identify the glyph that is to be rendered at a particular size and dpi.

```
public readonly struct GlyphRendererParameters : IEquatable<GlyphRendererParameters>
```

Implements
IEquatable<GlyphRendererParameters>

Inherited Members
ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

ClipBounds

Gets the clip bounds for the glyph, or null when the font defines none. The renderer restricts every operation the glyph emits to these bounds. The transform carries the glyph's placement, so the value is excluded from equality: equality identifies a glyph variant across positions.

```
public ClipBounds? ClipBounds { get; }
```

Property Value

ClipBounds?

CodePoint

Gets the codepoint represented by this glyph.

```
public CodePoint CodePoint { get; }
```

Property Value

CodePoint

CompositeGlyphId

Gets the id of the composite glyph if the GlyphType is Painted;

```
public ushort CompositeGlyphId { get; }
```

Property Value

ushort

Dpi

Gets the dots-per-inch the glyph is to be rendered at.

```
public float Dpi { get; }
```

Property Value

float

Font

Gets the name of the Font this glyph belongs to.

```
public string Font { get; }
```

Property Value

string

FontPalette

Gets the color palette selection the glyph's colors were resolved with, or null when the glyph resolves no palette colors. The selection changes the emitted layer paints so it participates in identity, allowing consumers to cache per palette variant.

```
public FontPalette? FontPalette { get; }
```

Property Value

FontPalette

FontStyle

Gets the style of the font this glyph belongs to.

```
public FontStyle FontStyle { get; }
```

Property Value

FontStyle

GlyphId

Gets the id of the glyph within the font tables.

```
public ushort GlyphId { get; }
```

Property Value

ushort

GlyphType

Gets the type of this glyph.

```
public GlyphType GlyphType { get; }
```

Property Value

GlyphType

GraphemeIndex

Gets the zero-based grapheme index in the original text.

```
public int GraphemeIndex { get; }
```

Property Value

int

HintingMode

Gets the hinting mode requested for the glyph. The mode shapes the emitted outline so it participates in identity, allowing consumers to cache per mode.

```
public HintingMode HintingMode { get; }
```

Property Value

HintingMode

LayoutMode

Gets the layout mode applied to the glyph.

```
public GlyphLayoutMode LayoutMode { get; }
```

Property Value

GlyphLayoutMode

PointSize

Gets the rendered point size.

```
public float PointSize { get; }
```

Property Value

float

TextRun

Gets the text run that this glyph belongs to.

```
public TextRun TextRun { get; }
```

Property Value

TextRun

Methods

Equals(GlyphRendererParameters)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(GlyphRendererParameters other)
```

Parameters

`other` GlyphRendererParameters

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

Operators

operator ==(GlyphRendererParameters, GlyphRendererParameters)

Compares two GlyphRendererParameters objects for equality.

```
public static bool operator ==(GlyphRendererParameters left, GlyphRendererParameters right)
```

Parameters

`left` GlyphRendererParameters

The GlyphRendererParameters on the left side of the operand.

`right` GlyphRendererParameters

The GlyphRendererParameters on the right side of the operand.

Returns

bool

True if the current left is equal to the `right` parameter; otherwise, false.

operator !=(GlyphRendererParameters, GlyphRendererParameters)

Compares two GlyphRendererParameters objects for inequality.

```
public static bool operator !=(GlyphRendererParameters left, GlyphRendererParameters right)
```

Parameters

`left` GlyphRendererParameters

The GlyphRendererParameters on the left side of the operand.

`right` GlyphRendererParameters

The GlyphRendererParameters on the right side of the operand.

Returns

bool

True if the current left is unequal to the `right` parameter; otherwise, false.

Struct GradientStop

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Defines a color stop for gradient paints. Offsets must be clamped to the range [0, 1] by the interpreter. Colors are direct RGBA and must not reference palettes.

```
public readonly struct GradientStop
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

GradientStop(float, GlyphColor)

Initializes a new instance of the GradientStop struct.

```
public GradientStop(float offset, GlyphColor color)
```

Parameters

`offset` float

The stop position in the range [0, 1].

`color` GlyphColor

The color at the stop.

Properties

Color

Gets the color at the stop (direct RGBA).

```
public GlyphColor Color { get; }
```

Property Value

GlyphColor

Offset

Gets the stop position in the range [0, 1].

```
public float Offset { get; }
```

Property Value

float

Enum GradientUnits

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Coordinate system to interpret gradient geometry.

```
public enum GradientUnits
```

Fields

`ObjectBoundingBox = 0`

Coordinates are normalized to the painted geometry's bounds ([0, 1] in X and Y). The renderer will map these to the actual path bounds at paint time.

`UserSpaceOnUse = 1`

Coordinates are absolute in the same space as the already-transformed geometry. Interpreters must pre-apply any gradient transforms before creating the paint.

Interface IGlyphRenderer

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

A surface that can have a glyph rendered to it as a series of actions.

```
public interface IGlyphRenderer
```

Extension Methods
GlyphRendererExtensions.Render(IGlyphRenderer, ReadOnlySpan<char>, TextOptions)

Methods

ArcTo(float, float, float, bool, bool, Vector2)

Adds an elliptical arc to the current figure. The arc curves from the last point to `point`, choosing one of four possible routes: clockwise or counterclockwise, and smaller or larger.

The arc sweep is always less than 360 degrees. The method appends a line to the last point if either radii are zero, or if last point is equal to `point`. In addition the method scales the radii to fit last point and `point` if both are greater than zero but too small to describe an arc.

```
void ArcTo(float radiusX, float radiusY, float rotation, bool largeArc, bool sweep,
Vector2 point)
```

Parameters

`radiusX` float

The x-radius of the ellipsis.

`radiusY` float

The y-radius of the ellipsis.

`rotation` float

The rotation along the X-axis; measured in degrees clockwise.

`largeArc` bool

The large arc flag, and is false if an arc spanning less than or equal to 180 degrees is chosen, or true if an arc spanning greater than 180 degrees is chosen.

`sweep` bool

The sweep flag, and is false if the line joining center to arc sweeps through decreasing angles, or true if it sweeps through increasing angles.

`point` Vector2

The end point of the arc.

BeginFigure()

Begins the figure.

```
void BeginFigure()
```

BeginGlyph(in FontRectangle, in GlyphRendererParameters)

Begins the glyph.

```
bool BeginGlyph(in FontRectangle bounds, in GlyphRendererParameters parameters)
```

Parameters

`bounds` FontRectangle

The bounds the glyph will be rendered at and at what size.

`parameters` GlyphRendererParameters

The set of parameters that uniquely represents a version of a glyph at particular font size, font family, font style and DPI.

Returns

bool

Returns true if the glyph should be rendered otherwise it returns false.

BeginGroup(CompositeMode)

Begins an isolated group. Layers and nested groups painted before the matching EndGroup() compose as one unit.

```
void BeginGroup(CompositeMode mode)
```

Parameters

`mode` CompositeMode

The mode used to blend the finished group onto the content below it within its parent.

BeginLayer(Paint?, FillRule)

Begins a new painted layer with the specified paint and fill rule. All geometry commands issued after this call belong to the layer until EndLayer() is called.

```
void BeginLayer(Paint? paint, FillRule fillRule)
```

Parameters

`paint` Paint

The paint definition.

`fillRule` FillRule

The fill rule to use when rasterizing this layer.

BeginText(in FontRectangle)

Called before any glyphs have been rendered.

```
void BeginText(in FontRectangle bounds)
```

Parameters

`bounds` FontRectangle

The rectangle within the text will be rendered.

CubicBezierTo(Vector2, Vector2, Vector2)

Draw a cubic bezier curve connecting the previous point to `point`.

```
void CubicBezierTo(Vector2 secondControlPoint, Vector2 thirdControlPoint, Vector2 point)
```

Parameters

`secondControlPoint` Vector2

The second control point.

`thirdControlPoint` Vector2

The third control point.

`point` Vector2

The point.

EnabledDecorations()

Provides a callback to enable custom logic to request decoration details. A custom TextRun might use alternative triggers to determine what decorations it needs access to.

```
TextDecorations EnabledDecorations()
```

Returns

TextDecorations

The text decorations the render wants render info for.

EndFigure()

Ends the figure.

```
void EndFigure()
```

EndGlyph()

Ends the glyph.

```
void EndGlyph()
```

EndGroup()

Ends the current group and blends it onto the content below it within its parent, using the mode given to BeginGroup(CompositeMode).

```
void EndGroup()
```

EndLayer()

Ends the current painted layer.

```
void EndLayer()
```

EndText()

Called once all glyphs have completed rendering.

```
void EndText()
```

LineTo(Vector2)

Draw a straight line connecting the previous point to `point`.

```
void LineTo(Vector2 point)
```

Parameters

`point` Vector2

The point.

MoveTo(Vector2)

Sets a new start point to draw lines from.

```
void MoveTo(Vector2 point)
```

Parameters

`point` Vector2

The point.

QuadraticBezierTo(Vector2, Vector2)

Draw a quadratic bezier curve connecting the previous point to `point`.

```
void QuadraticBezierTo(Vector2 secondControlPoint, Vector2 point)
```

Parameters

`secondControlPoint` Vector2

The second control point.

`point` Vector2

The point.

SetDecoration(TextDecorations, Vector2, Vector2, float, ReadOnlyMemory<float>)

Sets the details of a text decoration to be rendered. This only gets called if the decoration type was requested via EnabledDecorations() and after the glyph has been rendered via BeginGlyph(in FontRectangle, in GlyphRendererParameters) and EndGlyph().

```
void SetDecoration(TextDecorations textDecorations, Vector2 start, Vector2 end, float
thickness, ReadOnlyMemory<float> intersections)
```

Parameters

`textDecorations` TextDecorations

The type of decoration these details correspond to.

`start` Vector2

The start position from where to draw the decorations from.

`end` Vector2

The end position from where to draw the decorations to.

`thickness` float

The thickness to draw the decoration.

`intersections` ReadOnlyMemory<float>

The along-line intervals, in device pixels, where the glyph's ink crosses the decoration band, given as sorted `[start, end]` pairs. Empty when skip-ink does not apply. The full, untrimmed line is reported; a renderer that honours skip-ink carves the line around these intervals, widened by the thickness, and may extend a gap into an adjacent glyph.

Class LinearGradientPaint

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Linear gradient paint.

```
public sealed class LinearGradientPaint : Paint
```

Inheritance

object ← Paint ← LinearGradientPaint

Inherited Members

Paint.Opacity , Paint.Transform , Paint.CompositeMode , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

P0

Gets the gradient start point. Normalized if SixLabors.Fonts.Rendering.LinearGradientPaint.Units is ObjectBoundingBox.

```
public Vector2 P0 { get; init; }
```

Property Value

Vector2

P1

Gets the gradient end point. Normalized if SixLabors.Fonts.Rendering.LinearGradientPaint.Units is ObjectBoundingBox.

```
public Vector2 P1 { get; init; }
```

Property Value

Vector2

P2

Gets the rotation point for the gradient. Normalized if SixLabors.Fonts.Rendering.LinearGradient Paint.Units is ObjectBoundingBox.

```
public Vector2? P2 { get; init; }
```

Property Value

Vector2?

Spread

Gets the spread method applied when sampling outside the [0, 1] range.

```
public SpreadMethod Spread { get; init; }
```

Property Value

SpreadMethod

Stops

Gets the ordered gradient stops (ascending by Offset).

```
public GradientStop[] Stops { get; init; }
```

Property Value

GradientStop[]

Class Paint

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Base type for normalized paint definitions that can be used by any renderer. Glyph sources must pre-apply all relevant transforms and resolve any palette or format-specific constructs before creating a paint instance.

```
public abstract class Paint
```

Inheritance

object ← Paint

Derived

LinearGradientPaint, RadialGradientPaint, SolidPaint, SweepGradientPaint

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

CompositeMode

Gets the composite mode to use when applying this paint over existing content.

```
public CompositeMode CompositeMode { get; init; }
```

Property Value

CompositeMode

Opacity

Gets the per-layer opacity multiplier in the range [0, 1]. Renderers should multiply this value into the alpha channel of the final brush.

```
public float Opacity { get; init; }
```

Property Value

float

Transform

Gets the transform from the paint's own coordinate space to the space the glyph is rendered in. Gradient geometry is defined in the paint's space, so a renderer maps each sample through the inverse of this transform before evaluating the gradient.

```
public Matrix3x2 Transform { get; init; }
```

Property Value

Matrix3×2

Class PaintedGlyphMetrics

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Provides painted (layered) glyph rendering for color formats such as COLR v1 and OT-SVG. Geometry and paints are supplied in document-space by an interpreter; all layout transforms (UPEM mapping, DPI/point-size scaling, rotation, final placement) are applied here.

```
public sealed class PaintedGlyphMetrics : FontGlyphMetrics
```

Inheritance

object ← FontGlyphMetrics ← PaintedGlyphMetrics

Inherited Members

FontGlyphMetrics.CodePoint , FontGlyphMetrics.AdvanceWidth , FontGlyphMetrics.AdvanceHeight , FontGlyphMetrics.LeftSideBearing , FontGlyphMetrics.RightSideBearing , FontGlyphMetrics.TopSideBearing , FontGlyphMetrics.BottomSideBearing , FontGlyphMetrics.Width , FontGlyphMetrics.Height , FontGlyphMetrics.GlyphType , FontGlyphMetrics.UnitsPerEm , FontGlyphMetrics.GlyphId , FontGlyphMetrics.ScaleFactor , FontGlyphMetrics.TextAttributes , FontGlyphMetrics.TextDecorations , FontGlyphMetrics.TryGetHintedAdvanceWidth(float, float, HintingMode, out float) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Class RadialGradientPaint

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Represents a radial gradient paint defined by two circles. The first circle is centered at Center0 with radius Radius0. The second circle is centered at Center1 with radius Radius1. The color transition is computed between these two circles. Compatible with two-circle radial gradients used by HTML Canvas and OpenType COLR v1.

```
public sealed class RadialGradientPaint : Paint
```

Inheritance

object ← Paint ← RadialGradientPaint

Inherited Members

Paint.Opacity , Paint.Transform , Paint.CompositeMode , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Center0

Gets the center of the starting circle of the gradient.

```
public Vector2 Center0 { get; init; }
```

Property Value

Vector2

Center1

Gets the center of the ending circle of the gradient.

```
public Vector2 Center1 { get; init; }
```

Property Value

Vector2

Radius0

Gets the radius of the starting circle of the gradient. If SixLabors.Fonts.Rendering.RadialGradient Paint.Units is ObjectBoundingBox, the radius is normalized to the bounds.

```
public float Radius0 { get; init; }
```

Property Value

float

Radius1

Gets the radius of the ending circle of the gradient. If SixLabors.Fonts.Rendering.RadialGradient Paint.Units is ObjectBoundingBox, the radius is normalized to the bounds.

```
public float Radius1 { get; init; }
```

Property Value

float

Spread

Gets the spread method applied when sampling outside the [0, 1] range.

```
public SpreadMethod Spread { get; init; }
```

Property Value

SpreadMethod

Stops

Gets the ordered gradient stops, ascending by Offset.

```
public GradientStop[] Stops { get; init; }
```

Property Value

GradientStop[]

Class SolidPaint

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Solid color paint (direct RGBA). Interpreters must resolve palettes to RGBA. Compatible with OT-SVG solid fills and COLR v1 PaintSolid after CPAL resolution.

```
public sealed class SolidPaint : Paint
```

Inheritance

object ← Paint ← SolidPaint

Inherited Members

Paint.Opacity , Paint.Transform , Paint.CompositeMode , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Color

Gets the color to use for the fill. Alpha is respected and further multiplied by Opacity.

```
public GlyphColor Color { get; init; }
```

Property Value

GlyphColor

Enum SpreadMethod

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Specifies how a gradient should extend beyond the [0, 1] range.

```
public enum SpreadMethod
```

Fields

`Pad = 0`

Clamp to the end colors (pad).

`Reflect = 1`

Mirror the gradient (reflect).

`Repeat = 2`

Repeat the gradient (tile).

Class SweepGradientPaint

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Sweep (conic) gradient paint. Angles are expressed in degrees in the renderer's y-down space.

```
public sealed class SweepGradientPaint : Paint
```

Inheritance

object ← Paint ← SweepGradientPaint

Inherited Members

Paint.Opacity , Paint.Transform , Paint.CompositeMode , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Center

Gets the center of the sweep gradient.

```
public Vector2 Center { get; init; }
```

Property Value

Vector2

EndAngle

Gets the end angle in degrees.

```
public float EndAngle { get; init; }
```

Property Value

float

Spread

Gets the spread method applied when sampling outside the [0, 1] range.

```
public SpreadMethod Spread { get; init; }
```

Property Value

SpreadMethod

StartAngle

Gets the start angle in degrees.

```
public float StartAngle { get; init; }
```

Property Value

float

Stops

Gets the ordered gradient stops (ascending by Offset).

```
public GradientStop[] Stops { get; init; }
```

Property Value

GradientStop[]

Class TextRenderer

Namespace: SixLabors.Fonts.Rendering

Assembly: SixLabors.Fonts.dll

Encapsulates logic for laying out and then rendering text to a IGlyphRenderer surface.

```
public class TextRenderer
```

Inheritance

object ← TextRenderer

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TextRenderer(IGlyphRenderer)

Initializes a new instance of the TextRenderer class.

```
public TextRenderer(IGlyphRenderer renderer)
```

Parameters

`renderer` IGlyphRenderer

The renderer.

Methods

Render(TextShapingBuffer, GlyphOptions)

Renders shaped glyphs to the configured renderer.

```
public void Render(TextShapingBuffer buffer, GlyphOptions options)
```

Parameters

`buffer` TextShapingBuffer

The buffer containing the shaped glyphs.

`options` GlyphOptions

The glyph options supplying the shaping font, resolution, and baseline origin.

Render(ReadOnlySpan<char>, TextOptions)

Renders the text to the configured renderer.

```
public void Render(ReadOnlySpan<char> text, TextOptions options)
```

Parameters

`text` ReadOnlySpan<char>

The text to render.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Render(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions)

Renders positioned glyph ids to the configured renderer.

```
public void Render(ReadOnlySpan<ushort> glyphIds, ReadOnlySpan<Vector2> points,
GlyphOptions options)
```

Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`options` GlyphOptions

The glyph options.

Render(string, TextOptions)

Renders the text to the configured renderer.

```
public void Render(string text, TextOptions options)
```

Parameters

`text` string

The text to render.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

Render(ushort, GlyphOptions)

Renders a glyph id to the configured renderer.

```
public void Render(ushort glyphId, GlyphOptions options)
```

Parameters

`glyphId` ushort

The glyph identifier.

`options` GlyphOptions

The glyph options.

RenderTo(IGlyphRenderer, TextShapingBuffer, GlyphOptions)

Renders shaped glyphs to the `renderer`.

```
public static void RenderTo(IGlyphRenderer renderer, TextShapingBuffer buffer,
GlyphOptions options)
```

Parameters

`renderer` IGlyphRenderer

The target renderer.

`buffer` TextShapingBuffer

The buffer containing the shaped glyphs.

`options` GlyphOptions

The glyph options supplying the shaping font, resolution, and baseline origin.

RenderTo(IGlyphRenderer, ReadOnlySpan<char>, TextOptions)

Renders the text to the `renderer`.

```
public static void RenderTo(IGlyphRenderer renderer, ReadOnlySpan<char> text,
TextOptions options)
```

Parameters

`renderer` IGlyphRenderer

The target renderer.

`text` ReadOnlySpan<char>

The text to render.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

RenderTo(IGlyphRenderer, ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions)

Renders positioned glyph ids to the `renderer`.

```
public static void RenderTo(IGlyphRenderer renderer, ReadOnlySpan<ushort> glyphIds,
ReadOnlySpan<Vector2> points, GlyphOptions options)
```

Parameters

`renderer` IGlyphRenderer

The target renderer.

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`options` GlyphOptions

The glyph options.

RenderTo(IGlyphRenderer, string, TextOptions)

Renders the text to the `renderer`.

```
public static void RenderTo(IGlyphRenderer renderer, string text, TextOptions options)
```

Parameters

`renderer` IGlyphRenderer

The target renderer.

`text` string

The text to render.

`options` TextOptions

The text options. WrappingLength controls wrapping; use `-1` to disable wrapping.

RenderTo(IGlyphRenderer, ushort, GlyphOptions)

Renders a glyph id to the `renderer`.

```
public static void RenderTo(IGlyphRenderer renderer, ushort glyphId, GlyphOptions options)
```

Parameters

`renderer` IGlyphRenderer

The target renderer.

`glyphId` ushort

The glyph identifier.

`options` GlyphOptions

The glyph options.

Namespace SixLabors.Fonts.Tables.AdvancedTypographic

Structs

Tag
: Data type for tag identifiers. Tags are four byte integers, each byte representing a character. Tags are used to identify tables, design-variation axes, scripts, languages, font features, and baselines with human-readable names.

Enums

GlyphClassDef
: The GSUB and GPOS tables use the Glyph Class Definition table (GlyphClassDef) to identify which glyph classes to adjust with lookups. https://docs.microsoft.com/en-us/typography/opentype/spec/gdef#glyph-class-definition-table

KnownFeatureTags
: Provides enumeration for the different font features. https://docs.microsoft.com/en-us/typography/opentype/spec/featuretags

Enum GlyphClassDef

Namespace: SixLabors.Fonts.Tables.AdvancedTypographic

Assembly: SixLabors.Fonts.dll

The GSUB and GPOS tables use the Glyph Class Definition table (GlyphClassDef) to identify which glyph classes to adjust with lookups. https://docs.microsoft.com/en-us/typography/opentype/spec/gdef#glyph-class-definition-table

```
public enum GlyphClassDef
```

Fields

`BaseGlyph = 1`

Base glyph (single character, spacing glyph).

`ComponentGlyph = 4`

Component glyph (part of single character, spacing glyph).

`LigatureGlyph = 2`

Ligature glyph (multiple character, spacing glyph).

`MarkGlyph = 3`

Mark glyph (non-spacing combining glyph).

Enum KnownFeatureTags

Namespace: SixLabors.Fonts.Tables.AdvancedTypographic

Assembly: SixLabors.Fonts.dll

Provides enumeration for the different font features. https://docs.microsoft.com/en-us/typography/opentype/spec/featuretags

```
public enum KnownFeatureTags : uint
```

Fields

`AboveBaseForms = 1633842790`

Above-base Forms. Shortcode: abvf. Substitutes the above-base form of a vowel.

`AboveBaseMarkPositioning = 1633842797`

Above-base Mark Positioning. Shortcode: abvm. Positions marks above base glyphs.

`AboveBaseSubstitutions = 1633842803`

Above-base Substitutions. Shortcode: abvs. Substitutes a ligature for a base glyph and mark that's above it.

`AccessAllAlternates = 1633774708`

Access All Alternates. Shortcode: aalt. This feature makes all variations of a selected character accessible. This serves several purposes: An application may not support the feature by which the desired glyph would normally be accessed; the user may need a glyph outside the context supported by the normal substitution, or the user may not know what feature produces the desired glyph. Since many-to-one substitutions are not covered, ligatures would not appear in this table unless they were variant forms of another ligature.

`Akhand = 1634429038`

Akhand. Shortcode: akhn. Preferentially substitutes a sequence of characters with a ligature. This substitution is done irrespective of any characters that may precede or follow the sequence.

`AlternateAnnotationForms = 1851878516`

Alternate Annotation Forms. Shortcode: nalt. Replaces default glyphs with various notational forms (e.g. glyphs placed in open or solid circles, squares, parentheses, diamonds or rounded boxes). In some cases an annotation form may already be present, but the user may want a different one.

`AlternateHalfWidths = 1751215220`

Alternate Half Widths. Shortcode: halt. Respaces glyphs designed to be set on full-em widths, fitting them onto half-em widths. This differs from 'hwid' in that it does not substitute new glyphs.

`AlternateVerticalHalfMetrics = 1986552172`

Alternate Vertical Half Metrics. Shortcode: vhal. Respaces glyphs designed to be set on full-em heights, fitting them onto half-em heights.

`AlternateVerticalMetrics = 1986096244`

Alternate Vertical Metrics. Shortcode: valt. Repositions glyphs to visually center them within full-height metrics, for use in vertical setting. Typically applies to full-width Latin glyphs, which are aligned on a common horizontal baseline and not rotated when set vertically in CJKV fonts.

`AlternativeFractions = 1634103907`

Alternative Fractions. Shortccde: afrc. Replaces figures separated by a slash with an alternative form.

`BelowBaseForms = 1651275622`

Below-base Forms. Shortcode: blwf. Substitutes the below-base form of a consonant in conjuncts.

`BelowBaseMarkPositioning = 1651275629`

Below-base Mark Positioning. Shortcode: blwm. Positions marks below base glyphs.

`BelowBaseSubstitutions = 1651275635`

Below-base Substitutions. Shortcode: blws. Produces ligatures that comprise of base glyph and below-base forms.

`CapitalSpacing = 1668313968`

Capital Spacing. Shortcode: cpsp. Globally adjusts inter-glyph spacing for all-capital text. Most typefaces contain capitals and lowercase characters, and the capitals are positioned to work

with the lowercase. When capitals are used for words, they need more space between them for legibility and esthetics. This feature would not apply to monospaced designs. Of course the user may want to override this behavior in order to do more pronounced letterspacing for esthetic reasons.

`CaseSensitiveForms = 1667330917`

Case-Sensitive Forms. Shortcode: case. Shifts various punctuation marks up to a position that works better with all-capital sequences or sets of lining figures; also changes oldstyle figures to lining figures. By default, glyphs in a text face are designed to work with lowercase characters. Some characters should be shifted vertically to fit the higher visual center of all-capital or lining text. Also, lining figures are the same height (or close to it) as capitals, and fit much better with all-capital text.

`CenteredCjkPunctuation = 1668309876`

Centered CJK Punctuation. Shortcode: cpct. Centers specific punctuation marks for those fonts that do not include centered and non-centered forms.

`ConjunctFormAfterRo = 1667654002`

Conjunct Form After Ro. Shortcode: cfar. Substitutes alternate below-base or post-base forms in Khmer script when occurring after conjoined Ro ("Coeng Ra").

`ConjunctForms = 1667916660`

Conjunct Forms. Shortcode: cjct. Produces conjunct forms of consonants in Indic scripts. This is similar to the Akhands feature, but is applied at a different sequential point in the process of shaping an Indic syllable.

`ContextualAlternates = 1667329140`

Contextual Alternates. Shortcode: calt. In specified situations, replaces default glyphs with alternate forms which provide better joining behavior. Used in script typefaces which are designed to have some or all of their glyphs join.

`ContextualLigatures = 1668049255`

Contextual Ligatures. Shortcode: clig. Replaces a sequence of glyphs with a single glyph which is preferred for typographic purposes. Unlike other ligature features, 'clig' specifies the context in which the ligature is recommended. This capability is important in some script designs and for swash ligatures.

`ContextualSwash = 1668511592`

Contextual Swash. Shortcode: cswh. This feature replaces default character glyphs with corresponding swash glyphs in a specified context. Note that there may be more than one swash alternate for a given character.

`CursivePositioning = 1668641395`

Cursive Positioning. Shortcode: curs. In cursive scripts like Arabic, this feature cursively positions adjacent glyphs.

`Denominators = 1684959085`

Denominators. Shortcode: dnom. Replaces selected figures which follow a slash with denominator figures.

`DiscretionaryLigatures = 1684826471`

Discretionary Ligatures. Shortcode: dlig. Replaces a sequence of glyphs with a single glyph which is preferred for typographic purposes. This feature covers those ligatures which may be used for special effect, at the user's preference.

`Distances = 1684632436`

Distances. Shortcode: dist. Provides a means to control distance between glyphs.

`DotlessForms = 1685351539`

Dotless Forms. Shortcode: dtls. This feature provides dotless forms for Math Alphanumeric characters, such as U+1D422 MATHEMATICAL BOLD SMALL I, U+1D423 MATHEMATICAL BOLD SMALL J, U+1D456 U+MATHEMATICAL ITALIC SMALL I, U+1D457 MATHEMATICAL ITALIC SMALL J, and so on. The dotless forms are to be used as base forms for placing mathematical accents over them.

`ExpertForms = 1702391924`

Expert Forms. Shortcode: expt. Like the JIS78 Forms feature, this feature replaces standard forms in Japanese fonts with corresponding forms preferred by typographers. Although most of the JIS78 substitutions are included, the expert substitution goes on to handle many more characters.

`FinalGlyphOnLineAlternates = 1717660788`

Final Glyph on Line Alternates. Shortcode: falt. Replaces line final glyphs with alternate forms specifically designed for this purpose (they would have less or more advance width as need may be), to help justification of text.

`FlattenedAscentForms = 1718378851`

Flattened ascent forms. Shortcode: flac. This feature provides flattened forms of accents to be used over high-rise bases such as capitals. This feature should only change the shape of the accent and should not move it in the vertical or horizontal direction. Moving of the accents is done by the math handling client. Accents are flattened by the Math engine if their base is higher than MATH.MathConstants.FlattenedAccentBaseHeight.

`Fractions = 1718772067`

Fractions. Shortcode: frac. Replaces figures separated by a slash with "common" (diagonal) fractions.

`FullWidths = 1719101796`

Full Widths. Shortcode: fwid. Replaces glyphs set on other widths with glyphs set on full (usually em) widths. In a CJKV font, this may include "lower ASCII" Latin characters and various symbols. In a European font, this feature replaces proportionally-spaced glyphs with monospaced glyphs, which are generally set on widths of 0.6 em.

`GlyphCompositionDecomposition = 1667460464`

Glyph Composition/Decomposition. Shortcode: ccmp. To minimize the number of glyph alternates, it is sometimes desirable to decompose the default glyph for a character into two or more glyphs. Additionally, it may be preferable to compose default glyphs for two or more characters into a single glyph for better glyph processing. This feature permits such composition/decomposition. The feature should be processed as the first feature processed, and should be processed only when it is called.

`HalantForms = 1751215214`

Halant Forms. Shortcode: haln. Produces the halant forms of consonants in Indic scripts.

`HalfForms = 1751215206`

Half Forms. Shortcode: half. Produces the half forms of consonants in Indic scripts.

`HalfWidths = 1752656228`

Half Widths. Shortcode: hwid. Replaces glyphs on proportional widths, or fixed widths other than half an em, with glyphs on half-em (en) widths. Many CJKV fonts have glyphs which are set on multiple widths; this feature selects the half-em version. There are various contexts in which this is the preferred behavior, including compatibility with older desktop documents.

`Hangul = 1752065900`

Hangul. Shortcode: hngl. Replaces hanja (Chinese-style) Korean characters with the corresponding hangul (syllabic) characters. This effectively reverses the standard input method, in which hangul are entered and replaced by hanja. Many of these substitutions are one-to-one (GSUB lookup type 1), but hanja substitution often requires the user to choose from several possible hangul characters (GSUB lookup type 3).

```
HistoricalForms = 1751741300
```

Historical Forms. Shortcode: hist. Some letterforms were in common use in the past, but appear anachronistic today. The best-known example is the long form of s; others would include the old Fraktur k. Some fonts include the historical forms as alternates, so they can be used for a "period" effect. This feature replaces the default (current) forms with the historical alternates. While some ligatures are also used for historical effect, this feature deals only with single characters.

```
HistoricalLigatures = 1751935335
```

Historical Ligatures. Shortcode: hlig. Some ligatures were in common use in the past, but appear anachronistic today. Some fonts include the historical forms as alternates, so they can be used for a "period" effect. This feature replaces the default (current) forms with the historical alternates.

```
HojoKanjiForms = 1752132207
```

Hojo Kanji Forms (JIS X 0212-1990 Kanji Forms). Shortcode: hojo. The JIS X 0212-1990 (aka, "Hojo Kanji") and JIS X 0213:2004 character sets overlap significantly. In some cases their prototypical glyphs differ. When building fonts that support both JIS X 0212-1990 and JIS X 0213:2004 (such as those supporting the Adobe-Japan 1-6 character collection), it is recommended that JIS X 0213:2004 forms be preferred as the encoded form. The 'hojo' feature is used to access the JIS X 0212-1990 glyphs for the cases when the JIS X 0213:2004 form is encoded.

```
HorizontalKanaAlternates = 1751871073
```

Horizontal Kana Alternates. Shortcode: hkna. Replaces standard kana with forms that have been specially designed for only horizontal writing. This is a typographic optimization for improved fit and more even color. Also see 'vkna'.

```
InitialForms = 1768843636
```

Initial Forms. Shortcode: init. Replaces glyphs for characters that have applicable joining properties with an alternate form when occurring in an initial context.

```
IsolatedForms = 1769172844
```

Isolated Forms. Shortcode: isol. Replaces glyphs for characters that have applicable joining properties with an alternate form when occurring in a isolate (non-joining) context.

```
Italics = 1769234796
```

Italics. Shortcode: ital. Some fonts (such as Adobe's Pro Japanese fonts) will have both Roman and Italic forms of some characters in a single font. This feature replaces the Roman glyphs with the corresponding Italic glyphs.

```
Jis2004 = 1785737268
```

JIS2004 Forms. Shortcode: jp04. The National Language Council (NLC) of Japan has defined new glyph shapes for a number of JIS characters, which were incorporated into JIS X 0213:2004 as new prototypical forms. The 'jp04' feature is a subset of the 'nlck' feature, and is used to access these prototypical glyphs in a manner that maintains the integrity of JIS X 0213:2004.

```
Jis78Forms = 1785739064
```

JIS78 Forms. Shortcode: jp78. This feature replaces default (JIS90) Japanese glyphs with the corresponding forms from the JIS C 6226-1978 (JIS78) specification.

```
Jis83Forms = 1785739315
```

JIS83 Forms. Shortcode: jp83. This feature replaces default (JIS90) Japanese glyphs with the corresponding forms from the JIS X 0208-1983 (JIS83) specification.

```
Jis90Forms = 1785739568
```

JIS90 Forms. Shortcode: jp90. This feature replaces Japanese glyphs from the JIS78 or JIS83 specifications with the corresponding forms from the JIS X 0208-1990 (JIS90) specification.

```
JustificationAlternates = 1784769652
```

Justification Alternates. Shortcode: jalt. Improves justification of text by replacing glyphs with alternate forms specifically designed for this purpose (they would have less or more advance width as need may be).

```
Kerning = 1801810542
```

Kerning. Shortcode: kern. Adjusts amount of space between glyphs, generally to provide optically consistent spacing between glyphs. Although a well-designed typeface has consistent inter-glyph spacing overall, some glyph combinations require adjustment for improved legibility. Besides standard adjustment in the horizontal direction, this feature can supply size-dependent kerning data via device tables, "cross-stream" kerning in the Y text direction, and adjustment of

glyph placement independent of the advance adjustment. Note that this feature may apply to runs of more than two glyphs, and would not be used in monospaced fonts. Also note that this feature does not apply to text set vertically.

`LeadingJamoForms = 1818914159`

Leading Jamo Forms. Shortcode: ljmo. Substitutes the leading jamo form of a cluster.

`LeftBounds = 1818649188`

Left Bounds. Shortcode: lfbd. Aligns glyphs by their apparent left extents at the left ends of horizontal lines of text, replacing the default behavior of aligning glyphs by their origins. This feature is called by the Optical Bounds ('opbd') feature.

`LeftToRightGlyphAlternates = 1819570785`

Left-to-right glyph alternates. Shortcode: ltra. This feature applies glyphic variants (other than mirrored forms) appropriate for left-to-right text (for mirrored forms, see 'ltrm').

`LeftToRightMirroredForms = 1819570797`

Left-to-right mirrored forms. Shortcode: ltrm. This feature applies mirrored forms appropriate for left-to-right text. (For left-to-right glyph alternates, see 'ltra').

`Ligatures = 1818847073`

Standard Ligatures. Shortcode: liga. Replaces a sequence of glyphs with a single glyph which is preferred for typographic purposes. This feature covers the ligatures which the designer/manufacturer judges should be used in normal conditions.

`LiningFigures = 1819178349`

Lining Figures. Shortcode: lnum. This feature changes selected non-lining figures to lining figures.

`LocalizedForms = 1819239276`

Localized Forms. Shortcode: locl. Many scripts used to write multiple languages over wide geographical areas have developed localized variant forms of specific letters, which are used by individual literary communities. For example, a number of letters in the Bulgarian and Serbian alphabets have forms distinct from their Russian counterparts and from each other. In some cases the localized form differs only subtly from the script "norm", in others the forms are radically distinct. This feature enables localized forms of glyphs to be substituted for default forms.

`MarkPositioning = 1835102827`

Mark Positioning. Shortcode: mark. Positions mark glyphs with respect to base glyphs.

`MarkToMarkPositioning = 1835756907`

Mark to Mark Positioning. Shortcode: mkmk. Positions marks with respect to other marks. Required in various non-Latin scripts like Arabic.

`MathScriptStyleAlternates = 1936946297`

Math script style alternates. Shortcode: ssty. This feature provides glyph variants adjusted to be more suitable for use in subscripts and superscripts.

`MathematicalGreek = 1835496043`

Mathematical Greek. Shortcode: mgrk. Replaces standard typographic forms of Greek glyphs with corresponding forms commonly used in mathematical notation (which are a subset of the Greek alphabet).

`MedialForms = 1835361385`

Medial Forms. Shortcode: medi Replaces glyphs for characters that have applicable joining properties with an alternate form when occurring in a medial context. This applies to characters that have the Unicode Joining_Type property value Dual_Joining.

`MedialForms2 = 1835361330`

Medial Forms #2. Shortcode: med2. Replaces Alaph glyphs in the middle of Syriac words when the preceding base character can be joined to.

`Mset = 1836279156`

Shortcode: mset. Positions Arabic combining marks in fonts for Windows 95 using glyph substitution.

`NlcKanjiForms = 1852597099`

NLC Kanji Forms. Shortcode: nlck. The National Language Council (NLC) of Japan has defined new glyph shapes for a number of JIS characters in 2000. The 'nlck' feature is used to access those glyphs.

`NuktaForms = 1853188980`

Nukta Forms. Shortcode: nukt. Produces Nukta forms in Indic scripts.

`Numerators = 1853189490`

Numerators. Shortcode: numr. Replaces selected figures which precede a slash with numerator figures, and replaces the typographic slash with the fraction slash.

`OldstyleFigures = 1869509997`

Oldstyle Figures. Shortcode: onum. This feature changes selected figures from the default or lining style to oldstyle form.

`OpticalBounds = 1869636196`

Optical Bounds. Shortcode: opbd. Aligns glyphs by their apparent left or right extents in horizontal setting, or apparent top or bottom extents in vertical setting, replacing the default behavior of aligning glyphs by their origins. Another name for this behavior would be visual justification. The optical edge of a given glyph is only indirectly related to its advance width or bounding box; this feature provides a means for getting true visual alignment.

`OpticalSize = 1936292453`

Optical size. Shortcode: size. This feature stores two kinds of information about the optical size of the font: design size (the point size for which the font is optimized) and size range (the range of point sizes which the font can serve well), as well as other information which helps applications use the size range. The design size is useful for determining proper tracking behavior. The size range is useful in families which have fonts covering several ranges. Additional values serve to identify the set of fonts which share related size ranges, and to identify their shared name. Note that sizes refer to nominal final output size, and are independent of viewing magnification or resolution.

`Ordinals = 1869767790`

Ordinals. Shortcode: ordn. Replaces default alphabetic glyphs with the corresponding ordinal forms for use after figures. One exception to the follows-a-figure rule is the numero character (U+2116), which is actually a ligature substitution, but is best accessed through this feature.

`Ornaments = 1869770349`

Ornaments. Shortcode: ornm. This is a dual-function feature, which uses two input methods to give the user access to ornament glyphs (e.g. fleurons, dingbats and border elements) in the font. One method replaces the bullet character with a selection from the full set of available ornaments; the other replaces specific "lower ASCII" characters with ornaments assigned to them. The first approach supports the general or browsing user; the second supports the power user.

`PetiteCapitals = 1885561200`

Petite Capitals. Shortcode: pcap. Some fonts contain an additional size of capital letters, shorter than the regular smallcaps and whimsically referred to as petite caps. Such forms are most likely to be found in designs with a small lowercase x-height, where they better harmonise with lowercase text than the taller smallcaps (for examples of petite caps, see the Emigre type families Mrs Eaves and Filosofia). This feature turns lowercase characters into petite capitals. Forms related to petite capitals, such as specially designed figures, may be included.

`PetiteCapitalsFromCapitals = 1664249955`

Petite Capitals From Capitals. Shortcode: c2pc. This feature turns capital characters into petite capitals. It is generally used for words which would otherwise be set in all caps, such as acronyms, but which are desired in petite-cap form to avoid disrupting the flow of text. See the 'pcap' feature description for notes on the relationship of caps, smallcaps and petite caps.

`PostBaseForms = 1886614630`

Post-base Forms. Shortcode: pstf. Substitutes the post-base form of a consonant.

`PostBaseSubstitutions = 1886614643`

Post-base Substitutions. Shortcode: psts. Substitutes a sequence of a base glyph and post-base glyph, with its ligaturised form.

`PreBaseForms = 1886545254`

Pre-base Forms. Shortcode: pref. Substitutes the pre-base form of a consonant.

`PreBaseSubstitutions = 1886545267`

Pre-base Substitutions. Shortcode: pres. Produces the pre-base forms of conjuncts in Indic scripts. It can also be used to substitute the appropriate glyph variant for pre-base vowel signs.

`ProportionalAlternateVerticalMetrics = 1987076460`

Proportional Alternate Vertical Metrics. Shortcode: vpal. Respaces glyphs designed to be set on full-em heights, fitting them onto individual (more or less proportional) vertical heights. This differs from 'valt' in that it does not substitute new glyphs (GPOS, not GSUB feature). The user may prefer the monospaced form, or may simply want to ensure that the glyph is well-fit.

`ProportionalAlternateWidths = 1885432948`

Proportional Alternate Widths. Shortcode: palt. Respaces glyphs designed to be set on full-em widths, fitting them onto individual (more or less proportional) horizontal widths. This differs

from 'pwid' in that it does not substitute new glyphs (GPOS, not GSUB feature). The user may prefer the monospaced form, or may simply want to ensure that the glyph is well-fit and not rotated in vertical setting (Latin forms designed for proportional spacing would be rotated).

`ProportionalFigures = 1886287213`

Proportional Figures. Shortcode: pnum. Replaces figure glyphs set on uniform (tabular) widths with corresponding glyphs set on glyph-specific (proportional) widths. Tabular widths will generally be the default, but this cannot be safely assumed. Of course this feature would not be present in monospaced designs.

`ProportionalKana = 1886088801`

Proportional Kana. Shortcode: pkna. Replaces glyphs, kana and kana-related, set on uniform widths (half or full-width) with proportional glyphs.

`ProportionalWidths = 1886873956`

Proportional Widths. Shortcode: pwid. Replaces glyphs set on uniform widths (typically full or half-em) with proportionally spaced glyphs. The proportional variants are often used for the Latin characters in CJKV fonts, but may also be used for Kana in Japanese fonts.

`QuarterWidths = 1903651172`

Quarter Widths. Shortcode: qwid. Replaces glyphs on other widths with glyphs set on widths of one quarter of an em (half an en). The characters involved are normally figures and some forms of punctuation.

`RakarForms = 1919644262`

Rakar Forms. Shortcode: rkrf. Produces conjoined forms for consonants with rakar in Devanagari and Gujarati scripts.

`Randomize = 1918987876`

Randomize. Shortcode: rand. In order to emulate the irregularity and variety of handwritten text, this feature allows multiple alternate forms to be used.

`RephForm = 1919969382`

Reph Form. Shortcode: rphf. Substitutes the Reph form for a consonant and halant sequence.

`RequiredContextualAlternates = 1919118452`

Required Contextual Alternates. Shortcode: rclt. In specified situations, replaces default glyphs with alternate forms which provide for better joining behavior or other glyph relationships. Especially important in script typefaces which are designed to have some or all of their glyphs join, but applicable also to e.g. variants to improve spacing. This feature is similar to 'calt', but with the difference that it should not be possible to turn off 'rclt' substitutions: they are considered essential to correct layout of the font.

`RequiredLigatures = 1919707495`

Required Ligatures. Shortcode: rlig. Replaces a sequence of glyphs with a single glyph which is preferred for typographic purposes. This feature covers those ligatures, which the script determines as required to be used in normal conditions. This feature is important for some scripts to insure correct glyph formation.

`RequiredVariationAlternates = 1920365166`

Required Variation Alternates. Shortcode: rvrn. his feature is used in fonts that support OpenType Font Variations in order to select alternate glyphs for particular variation instances.

`RightBounds = 1920229988`

Right Bounds. Shortcode: rtbd. Aligns glyphs by their apparent right extents at the right ends of horizontal lines of text, replacing the default behavior of aligning glyphs by their origins. This feature is called by the Optical Bounds ('opbd') feature.

`RightToLeftAlternates = 1920232545`

Right-to-left alternates. Shortcode: rtla. This feature applies glyphic variants (other than mirrored forms) appropriate for right-to-left text. (For mirrored forms, see 'rtlm'.)

`RightToLeftMirroredForms = 1920232557`

Right-to-left mirrored forms. Shortcode: rtlm. This feature applies mirrored forms appropriate for right-to-left text other than for those characters that would be covered by the character-level mirroring step performed by an OpenType layout engine. (For right-to-left glyph alternates, see 'rtla'.)

`RubyNotationForms = 1920295545`

Ruby Notation Forms. Shortcode: ruby. Japanese typesetting often uses smaller kana glyphs, generally in superscripted form, to clarify the meaning of kanji which may be unfamiliar to the reader. These are called "ruby", from the old typesetting term for four-point-sized type. This feature identifies glyphs in the font which have been designed for this use, substituting them for the default designs.

`ScientificInferiors = 1936289382`

Scientific Inferiors. Shortcode: sinf. Replaces lining or oldstyle figures with inferior figures (smaller glyphs which sit lower than the standard baseline, primarily for chemical or mathematical notation). May also replace lowercase characters with alphabetic inferiors.

`SimplifiedForms = 1936552044`

Simplified Forms. Shortcode: smpl. Replaces "traditional" Chinese or Japanese forms with the corresponding "simplified" forms.

`SlashedZero = 2053468783`

Slashed Zero. Shortcode: zero. Some fonts contain both a default form of zero, and an alternative form which uses a diagonal slash through the counter. Especially in condensed designs, it can be difficult to distinguish between 0 and O (zero and capital O) in any situation where capitals and lining figures may be arbitrarily mixed. This feature allows the user to change from the default 0 to a slashed form.

`SmallCapitals = 1936548720`

Small Capitals. Shortcode: smcp. This feature turns lowercase characters into small capitals. This corresponds to the common SC font layout. It is generally used for display lines set in Large and small caps, such as titles. Forms related to small capitals, such as oldstyle figures, may be included.

`SmallCapitalsFromCapitals = 1664250723`

Small Capitals From Capitals. Shortcode: c2sc. This feature turns capital characters into small capitals. It is generally used for words which would otherwise be set in all caps, such as acronyms, but which are desired in small-cap form to avoid disrupting the flow of text.

`StretchingGlyphDecomposition = 1937007464`

Stretching Glyph Decomposition. Shortcode: stch. Unicode characters, such as the Syriac Abbreviation Mark (U+070F), that enclose other characters need to be able to stretch in order to dynamically adapt to the width of the enclosed text. This feature defines a decomposition set consisting of an odd number of glyphs which describe the stretching glyph. The odd numbered glyphs in the decomposition are fixed reference points which are distributed evenly from the start to the end of the enclosed text. The even numbered glyphs may be repeated as necessary to fill the space between the fixed glyphs. The first and last glyphs may either be simple glyphs with width at the baseline, or mark glyphs. All other decomposition glyphs should have width, but must be defined as mark glyphs.

`StylisticAlternates = 1935764596`

Stylistic Alternates. Shortcode: salt. Many fonts contain alternate glyph designs for a purely esthetic effect; these don’t always fit into a clear category like swash or historical. As in the case of swash glyphs, there may be more than one alternate form. This feature replaces the default forms with the stylistic alternates.

`Subscript = 1937072755`

Subscript. Shortcode: subs. The 'subs' feature may replace a default glyph with a subscript glyph, or it may combine a glyph substitution with positioning adjustments for proper placement.

`Superscript = 1937076339`

Superscript. Shortcode: sups. Replaces lining or oldstyle figures with superior figures (primarily for footnote indication), and replaces lowercase letters with superior letters (primarily for abbreviated French titles).

`Swash = 1937208168`

Swash. Shortcode: swsh. This feature replaces default character glyphs with corresponding swash glyphs. Note that there may be more than one swash alternate for a given character.

`TabularFigures = 1953396077`

Tabular Figures. Shortcode: tnum. Replaces figure glyphs set on proportional widths with corresponding glyphs set on uniform (tabular) widths. Tabular widths will generally be the default, but this cannot be safely assumed. Of course this feature would not be present in monospaced designs.

`TerminalForm2 = 1718185522`

Terminal Form #2. Shortcode: fin2. Replaces the Alaph glyph at the end of Syriac words with its appropriate form, when the preceding base character cannot be joined to, and that preceding base character is not a Dalath, Rish, or dotless Dalath-Rish.

`TerminalForm3 = 1718185523`

Terminal Form #3. Shortcode: fin3. Replaces Alaph glyphs at the end of Syriac words when the preceding base character is a Dalath, Rish, or dotless Dalath-Rish.

`TerminalForms = 1718185569`

Terminal Forms. Shortcode: fina. Replaces glyphs for characters that have applicable joining properties with an alternate form when occurring in a final context.

`ThirdWidths = 1953982820`

Third Widths. Shortcode: twid. Replaces glyphs on other widths with glyphs set on widths of one third of an em. The characters involved are normally figures and some forms of punctuation.

`Titling = 1953068140`

Titling. Shortcode: titl. This feature replaces the default glyphs with corresponding forms designed specifically for titling. These may be all-capital and/or larger on the body, and adjusted for viewing at larger sizes.

`TraditionalForms = 1953653092`

Traditional Forms. Shortcode: trad. Replaces 'simplified' Chinese hanzi or Japanese kanji forms with the corresponding 'traditional' forms.

`TraditionalNameForms = 1953390957`

Traditional Name Forms. Shortcode: tnam. Replaces "simplified" Japanese kanji forms with the corresponding "traditional" forms. This is equivalent to the Traditional Forms feature, but explicitly limited to the traditional forms considered proper for use in personal names (as many as 205 glyphs in some fonts).

`TrailingJamoForms = 1953131887`

Trailing Jamo Forms. Shortcode: tjmo. Substitutes the trailing jamo form of a cluster.

`Unicase = 1970170211`

Unicase. Shortcode: unic. This feature maps upper- and lowercase letters to a mixed set of lowercase and small capital forms, resulting in a single case alphabet (for an example of unicase, see the Emigre type family Filosofia). The letters substituted may vary from font to font, as appropriate to the design. If aligning to the x-height, smallcap glyphs may be substituted, or specially designed unicase forms might be used. Substitutions might also include specially designed figures.

`VattuVariants = 1986098293`

Vattu Variants. Shortcode: vatu. In an Indic consonant conjunct, substitutes a ligature glyph for a base consonant and a following vattu (below-base) form of a conjoining consonant, or for a half form of a consonant and a following vattu form.

`VerticalAlternates = 1986359924`

Vertical Alternates. Shortcode: vert. Transforms default glyphs into glyphs that are appropriate for upright presentation in vertical writing mode.While the glyphs for most characters in East Asian writing systems remain upright when set in vertical writing mode, some must be transformed — usually by rotation, shifting, or different component ordering — for vertical writing mode.

`VerticalAlternatesAndRotation = 1987212338`

Vertical Alternates and Rotation. Shortcode: vrt2. Replaces some fixed-width (half-, third- or quarter-width) or proportional-width glyphs (mostly Latin or katakana) with forms suitable for vertical writing (that is, rotated 90 degrees clockwise). Note that these are a superset of the glyphs covered in the 'vert' table.

`VerticalAlternatesForRotation = 1987212402`

Vertical Alternates for Rotation. Shortcode: vrtr. Transforms default glyphs into glyphs that are appropriate for sideways presentation in vertical writing mode. While the glyphs for most characters in East Asian writing systems remain upright when set in vertical writing mode, glyphs for other characters — such as those of other scripts or for particular Western-style punctuation — are expected to be presented sideways in vertical writing.

`VerticalKanaAlternates = 1986752097`

Vertical Kana Alternates. Shortcode: vkna. Replaces standard kana with forms that have been specially designed for only vertical writing. This is a typographic optimization for improved fit and more even color. Also see 'hkna'.

`VerticalKerning = 1986753134`

Vertical Kerning. Shortcode: vkrn Adjusts amount of space between glyphs, generally to provide optically consistent spacing between glyphs. Although a well-designed typeface has consistent inter-glyph spacing overall, some glyph combinations require adjustment for improved legibility. Besides standard adjustment in the vertical direction, this feature can supply size-dependent kerning data via device tables, "cross-stream" kerning in the X text direction, and adjustment of glyph placement independent of the advance adjustment. Note that this feature may apply to runs of more than two glyphs, and would not be used in monospaced fonts. Also note that this feature applies only to text set vertically.

`VowelJamoForms = 1986686319`

Vowel Jamo Forms. Shortcode: vjmo. Substitutes the vowel jamo form of a cluster.

Struct Tag

Namespace: SixLabors.Fonts.Tables.AdvancedTypographic

Assembly: SixLabors.Fonts.dll

Data type for tag identifiers. Tags are four byte integers, each byte representing a character. Tags are used to identify tables, design-variation axes, scripts, languages, font features, and baselines with human-readable names.

```
public readonly struct Tag : IEquatable<Tag>
```

Implements
IEquatable<Tag>

Inherited Members
object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

Tag(uint)

Initializes a new instance of the Tag struct.

```
public Tag(uint value)
```

Parameters

`value` uint

The tag value.

Properties

Value

Gets the Tag value as 32 bit unsigned integer.

```
public uint Value { get; }
```

Property Value

uint

Methods

Equals(Tag)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(Tag other)
```

Parameters

`other` Tag

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

Parse(string)

Converts the string representation of a number to its Tag equivalent.

```
public static Tag Parse(string value)
```

Parameters

`value` string

A string containing a tag to convert.

Returns

Tag

The Tag.

ToString()

Returns the fully qualified type name of this instance.

```
public override string ToString()
```

Returns

string

The fully qualified type name.

Operators

operator ==(Tag, Tag)

Determines whether two Tag instances are equal.

```
public static bool operator ==(Tag left, Tag right)
```

Parameters

`left` Tag

The left tag.

`right` Tag

The right tag.

Returns

bool

true if the tags are equal; otherwise, false.

implicit operator Tag(KnownFeatureTags)

Implicitly converts a KnownFeatureTags to a Tag.

```
public static implicit operator Tag(KnownFeatureTags value)
```

Parameters

`value` KnownFeatureTags

The feature tag enum value.

Returns

Tag

implicit operator Tag(uint)

Implicitly converts a uint to a Tag.

```
public static implicit operator Tag(uint value)
```

Parameters

`value` uint

The unsigned integer value.

Returns

Tag

operator !=(Tag, Tag)

Determines whether two Tag instances are not equal.

```
public static bool operator !=(Tag left, Tag right)
```

Parameters

`left` Tag

The left tag.

`right` Tag

The right tag.

Returns

bool

true if the tags are not equal; otherwise, false.

Namespace SixLabors.Fonts.Tables.AdvancedTypographic.Variations

Structs

RegionAxisCoordinates

Each RegionAxisCoordinates record provides coordinate values for a region along a single axis. The three values must all be within the range -1.0 to +1.0. startCoord must be less than or equal to peakCoord, and peakCoord must be less than or equal to endCoord. The three values must be either all non-positive or all non-negative with one possible exception: if peakCoord is zero, then startCoord can be negative or 0 while endCoord can be positive or zero.
https://docs.microsoft.com/de-de/typography/opentype/spec/otvarcommonformats#variation-regions

VariationAxis

https://learn.microsoft.com/en-us/typography/opentype/otspec184/fvar#variationaxisrecord

Struct RegionAxisCoordinates

Namespace: SixLabors.Fonts.Tables.AdvancedTypographic.Variations

Assembly: SixLabors.Fonts.dll

Each RegionAxisCoordinates record provides coordinate values for a region along a single axis. The three values must all be within the range -1.0 to +1.0. startCoord must be less than or equal to peakCoord, and peakCoord must be less than or equal to endCoord. The three values must be either all non-positive or all non-negative with one possible exception: if peakCoord is zero, then startCoord can be negative or 0 while endCoord can be positive or zero.
https://docs.microsoft.com/de-de/typography/opentype/spec/otvarcommonformats#variation-regions

```
public readonly struct RegionAxisCoordinates
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

EndCoord

Gets the region end coordinate value for the current axis.

```
public float EndCoord { get; init; }
```

Property Value

float

PeakCoord

Gets the region peak coordinate value for the current axis.

```
public float PeakCoord { get; init; }
```

Property Value

float

StartCoord

Gets the region start coordinate value for the current axis.

```
public float StartCoord { get; init; }
```

Property Value

float

Struct VariationAxis

Namespace: SixLabors.Fonts.Tables.AdvancedTypographic.Variations

Assembly: SixLabors.Fonts.dll

https://learn.microsoft.com/en-us/typography/opentype/otspec184/fvar#variationaxisrecord

```
public readonly struct VariationAxis
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

Default

Gets the default coordinate value for the axis.

```
public float Default { get; init; }
```

Property Value

float

Max

Gets the maximum coordinate value for the axis.

```
public float Max { get; init; }
```

Property Value

float

Min

Gets the minimum coordinate value for the axis.

```
public float Min { get; init; }
```

Property Value

float

Name

Gets the name of the axes.

```
public string Name { get; init; }
```

Property Value

string

Tag

Gets tag identifying the design variation for the axis.

```
public Tag Tag { get; init; }
```

Property Value

Tag

Namespace SixLabors.Fonts.Tables.TrueType

Classes

TrueTypeGlyphMetrics
: Represents a glyph metric from a particular TrueType font face.

Class TrueTypeGlyphMetrics

Namespace: SixLabors.Fonts.Tables.TrueType

Assembly: SixLabors.Fonts.dll

Represents a glyph metric from a particular TrueType font face.

```
public class TrueTypeGlyphMetrics : FontGlyphMetrics
```

Inheritance

object ← FontGlyphMetrics ← TrueTypeGlyphMetrics

Inherited Members

FontGlyphMetrics.CodePoint , FontGlyphMetrics.AdvanceWidth , FontGlyphMetrics.AdvanceHeight , FontGlyphMetrics.LeftSideBearing , FontGlyphMetrics.RightSideBearing , FontGlyphMetrics.TopSideBearing , FontGlyphMetrics.BottomSideBearing , FontGlyphMetrics.Width , FontGlyphMetrics.Height , FontGlyphMetrics.GlyphType , FontGlyphMetrics.UnitsPerEm , FontGlyphMetrics.GlyphId , FontGlyphMetrics.ScaleFactor , FontGlyphMetrics.TextAttributes , FontGlyphMetrics.TextDecorations , FontGlyphMetrics.ShouldSkipGlyphRendering(CodePoint) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

TryGetHintedAdvanceWidth(float, float, HintingMode, out float)

Attempts to compute the whole pixel advance width for the glyph under full hinting. The value comes from the 'hdmx' device record when the font carries one, exactly as classic rasterizers resolve device advances, or from the design advance rounded to whole pixels otherwise: fonts without device records do not adjust advances in their instructions. Neither path executes hinting or touches the outline cache, so this is safe on the layout hot path.

```
public override bool TryGetHintedAdvanceWidth(float pointSize, float dpi, HintingMode
```

```
  hintingMode, out float advancePx)
```

Parameters

`pointSize` float

The font size in pt units.

`dpi` float

The DPI (Dots Per Inch) to render/measure the glyph at

`hintingMode` HintingMode

The requested hinting mode.

`advancePx` float

The advance width in whole device pixels.

Returns

bool

true if a hinted advance applies; otherwise, false.

Namespace SixLabors.Fonts.Unicode

Classes

MemoryExtensions
Contains extension methods for memory types.

Structs

ArabicJoiningClass
Represents the Unicode joining properties of a given CodePoint.
https://www.unicode.org/reports/tr44/#Joining_Type
https://www.unicode.org/reports/tr44/#Joining_Group

BidiClass
Represents the Unicode Bidi value of a given CodePoint. https://unicode.org/reports/tr9/#Table

CodePoint
Represents a Unicode value ([U+0000..U+10FFFF], inclusive).

GraphemeCluster
Represents a Unicode grapheme cluster and metadata derived while enumerating it.

SpanCodePointEnumerator
An enumerator for retrieving CodePoint instances from a ReadOnlySpan<T>. Methods are pattern-matched by compiler to allow using foreach pattern.

SpanGraphemeEnumerator
An enumerator for retrieving Grapheme instances from a ReadOnlySpan<T>.
Implements the Unicode Grapheme Cluster Algorithm. UAX:29
https://www.unicode.org/reports/tr29/
Supports the UAX #29 extended grapheme cluster rule for Indic conjunct sequences (GB9c) using the IndicConjunctBreakClass property.
Methods are pattern-matched by compiler to allow using foreach pattern.

SpanWordEnumerator
An enumerator for retrieving word-boundary segments from a ReadOnlySpan<T>.
Implements the Unicode Word Boundary Algorithm. UAX #29
https://www.unicode.org/reports/tr29/
Methods are pattern-matched by compiler to allow using foreach pattern.

TerminalWidthOptions
: Defines policy used when resolving grapheme clusters to terminal cell widths.

WordSegment
: Represents a segment between two Unicode word boundaries.

Enums

ArabicJoiningGroup
: Unicode Joining_Group property values used for Arabic-script cursive shaping. https://www.unicode.org/reports/tr44/#Joining_Group

ArabicJoiningType
: Unicode Joining_Type property values used for Arabic-script cursive shaping. https://www.unicode.org/reports/tr44/#Joining_Type

BidiCharacterType
: Unicode Bidi_Class property values. https://www.unicode.org/reports/tr9/#Bidirectional_Character_Types

BidiPairedBracketType
: Unicode Bidi_Paired_Bracket_Type property values. https://www.unicode.org/reports/tr9/#Paired_Brackets

EastAsianWidthClass
: Unicode East_Asian_Width property values. https://www.unicode.org/reports/tr11/

EmojiProperties
: Binary Unicode emoji properties from UTS #51.

GraphemeClusterClass
: Unicode Grapheme_Cluster_Break property values and local rule sentinels. https://www.unicode.org/reports/tr29/#Grapheme_Cluster_Break_Property_Values

GraphemeClusterFlags
: Flags describing properties of a Unicode grapheme cluster.

IndicConjunctBreakClass
: Unicode Indic_Conjunct_Break property values used by UAX #29 GB9c.

IndicPositionalCategory
: Unicode Indic_Positional_Category property values. https://www.unicode.org/reports/tr44/#Indic_Positional_Category

IndicSyllabicCategory

Unicode Indic_Syllabic_Category property values. https://www.unicode.org/reports/tr44/#Indic_Syllabic_Category

LineBreakClass

Unicode Line_Break property values. https://www.unicode.org/reports/tr14/#Table1

ScriptClass

Script values used when shaping text. https://www.unicode.org/reports/tr24/#Script

TerminalAmbiguousWidth

Defines how East Asian Width Ambiguous scalars resolve for terminal cell measurement.

TerminalControlCharacterWidth

Defines how C0 and C1 control scalars resolve for terminal cell measurement.

TerminalEmojiWidth

Defines how emoji clusters resolve for terminal cell measurement.

VerticalOrientationType

Unicode Vertical_Orientation property values. https://www.unicode.org/reports/tr50/#vo

WordBreakClass

Unicode Word_Break property values. https://www.unicode.org/reports/tr29/#Word_Boundaries

Struct ArabicJoiningClass

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Represents the Unicode joining properties of a given CodePoint.
https://www.unicode.org/reports/tr44/#Joining_Type
https://www.unicode.org/reports/tr44/#Joining_Group

```
public readonly struct ArabicJoiningClass
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Remarks

This combines the Unicode `Joining_Type` and `Joining_Group` properties used by cursive shaping. Unlisted nonspacing marks, enclosing marks, and format controls follow the Unicode default joining behavior.

Constructors

ArabicJoiningClass(CodePoint)

Initializes a new instance of the ArabicJoiningClass struct.

```
public ArabicJoiningClass(CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint.

Properties

JoiningGroup

Gets the Unicode joining group.

```
public ArabicJoiningGroup JoiningGroup { get; }
```

Property Value

ArabicJoiningGroup

JoiningType

Gets the Unicode joining type.

```
public ArabicJoiningType JoiningType { get; }
```

Property Value

ArabicJoiningType

Enum ArabicJoiningGroup

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Joining_Group property values used for Arabic-script cursive shaping.
https://www.unicode.org/reports/tr44/#Joining_Group

```
public enum ArabicJoiningGroup
```

Fields

`AfricanFeh = 0`

African_Feh

`AfricanNoon = 1`

African_Noon

`AfricanQaf = 2`

African_Qaf

`Ain = 3`

Ain

`Alaph = 4`

Alaph

`Alef = 5`

Alef

`Beh = 6`

Beh

`Beth = 7`

Beth

```
BurushaskiYehBarree = 8
```

Burushaski_Yeh_Barree

```
Dal = 9
```

Dal

```
DalathRish = 10
```

Dalath_Rish

```
E = 11
```

E

```
FarsiYeh = 12
```

Farsi_Yeh

```
Fe = 13
```

Fe

```
Feh = 14
```

Feh

```
FinalSemkath = 15
```

Final_Semkath

```
Gaf = 16
```

Gaf

```
Gamal = 17
```

Gamal

```
Hah = 18
```

Hah

```
HanifiRohingyaKinnaYa = 19
```

Hanifi_Rohingya_Kinna_Ya

HanifiRohingyaPa = 20

Hanifi_Rohingya_Pa

He = 21

He

Heh = 22

Heh

HehGoal = 23

Heh_Goal

Heth = 24

Heth

Kaf = 25

Kaf

Kaph = 26

Kaph

KashmiriYeh = 27

Kashmiri_Yeh

Khaph = 28

Khaph

KnottedHeh = 29

Knotted_Heh

Lam = 30

Lam

Lamadh = 31

Lamadh

MalayalamBha = 32

Malayalam_Bha

MalayalamJa = 33

Malayalam_Ja

MalayalamLla = 34

Malayalam_Lla

MalayalamLlla = 35

Malayalam_Llla

MalayalamNga = 36

Malayalam_Nga

MalayalamNna = 37

Malayalam_Nna

MalayalamNnna = 38

Malayalam_Nnna

MalayalamNya = 39

Malayalam_Nya

MalayalamRa = 40

Malayalam_Ra

MalayalamSsa = 41

Malayalam_Ssa

MalayalamTta = 42

Malayalam_Tta

ManichaeanAleph = 43

Manichaean_Aleph

```
ManichaeanAyin = 44
```

Manichaean_Ayin

```
ManichaeanBeth = 45
```

Manichaean_Beth

```
ManichaeanDaleth = 46
```

Manichaean_Daleth

```
ManichaeanDhamedh = 47
```

Manichaean_Dhamedh

```
ManichaeanFive = 48
```

Manichaean_Five

```
ManichaeanGimel = 49
```

Manichaean_Gimel

```
ManichaeanHeth = 50
```

Manichaean_Heth

```
ManichaeanHundred = 51
```

Manichaean_Hundred

```
ManichaeanKaph = 52
```

Manichaean_Kaph

```
ManichaeanLamedh = 53
```

Manichaean_Lamedh

```
ManichaeanMem = 54
```

Manichaean_Mem

```
ManichaeanNun = 55
```

Manichaean_Nun

```
ManichaeanOne = 56
```

Manichaean_One

```
ManichaeanPe = 57
```

Manichaean_Pe

```
ManichaeanQoph = 58
```

Manichaean_Qoph

```
ManichaeanResh = 59
```

Manichaean_Resh

```
ManichaeanSadhe = 60
```

Manichaean_Sadhe

```
ManichaeanSamekh = 61
```

Manichaean_Samekh

```
ManichaeanTaw = 62
```

Manichaean_Taw

```
ManichaeanTen = 63
```

Manichaean_Ten

```
ManichaeanTeth = 64
```

Manichaean_Teth

```
ManichaeanThamedh = 65
```

Manichaean_Thamedh

```
ManichaeanTwenty = 66
```

Manichaean_Twenty

```
ManichaeanWaw = 67
```

Manichaean_Waw

ManichaeanYodh = 68

Manichaean_Yodh

ManichaeanZayin = 69

Manichaean_Zayin

Meem = 70

Meem

Mim = 71

Mim

NoJoiningGroup = 72

No_Joining_Group

Noon = 73

Noon

Nun = 74

Nun

Nya = 75

Nya

Pe = 76

Pe

Qaf = 77

Qaf

Qaph = 78

Qaph

Reh = 79

Reh

```
ReversedPe = 80
```

Reversed_Pe

```
RohingyaYeh = 81
```

Rohingya_Yeh

```
Sad = 82
```

Sad

```
Sadhe = 83
```

Sadhe

```
Seen = 84
```

Seen

```
Semkath = 85
```

Semkath

```
Shin = 86
```

Shin

```
StraightWaw = 87
```

Straight_Waw

```
SwashKaf = 88
```

Swash_Kaf

```
SyriacWaw = 89
```

Syriac_Waw

```
Tah = 90
```

Tah

```
Taw = 91
```

Taw

`TehMarbuta = 92`

Teh_Marbuta

`TehMarbutaGoal = 93`

Teh_Marbuta_Goal, formerly known by the stable alias Hamza_On_Heh_Goal.

`Teth = 94`

Teth

`ThinNoon = 95`

Thin_Noon

`ThinYeh = 96`

Thin_Yeh

`VerticalTail = 97`

Vertical_Tail

`Waw = 98`

Waw

`Yeh = 99`

Yeh

`YehBarree = 100`

Yeh_Barree

`YehWithTail = 101`

Yeh_With_Tail

`Yudh = 102`

Yudh

`YudhHe = 103`

Yudh_He

```
Zain = 104
```

Zain

```
Zhain = 105
```

Zhain

Remarks

Joining groups identify characters that share the same basic joining shape for cursive joining.

Enum ArabicJoiningType

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Joining_Type property values used for Arabic-script cursive shaping.
https://www.unicode.org/reports/tr44/#Joining_Type

```
public enum ArabicJoiningType
```

Fields

`DualJoining = 2`

Dual_Joining (D): joins on both sides.

`JoinCausing = 3`

Join_Causing (C): causes adjacent join-capable characters to join.

`LeftJoining = 1`

Left_Joining (L): joins on the left side only.

`NonJoining = 4`

Non_Joining (U): does not participate in cursive joining.

`RightJoining = 0`

Right_Joining (R): joins on the right side only.

`Transparent = 5`

Transparent (T): ignored when determining the joining relationship of surrounding characters.

`Unlisted = 6`

Non_Joining_Unassigned (X): the character is not listed in the joining data. Its behaviour follows from its general category, resolved where the joining class is read: a mark or a format character is transparent, anything else does not join.

Remarks

These values describe how a character participates in cursive joining with neighboring characters.

Enum BidiCharacterType

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Bidi_Class property values.
https://www.unicode.org/reports/tr9/#Bidirectional_Character_Types

```
public enum BidiCharacterType
```

Fields

`ArabicLetter = 2`

Right-to-Left Arabic (AL).

`ArabicNumber = 6`

Arabic Number (AN).

`BoundaryNeutral = 9`

Boundary Neutral (BN).

`CommonSeparator = 7`

Common Number Separator (CS).

`EuropeanNumber = 3`

European Number (EN).

`EuropeanSeparator = 4`

European Number Separator (ES).

`EuropeanTerminator = 5`

European Number Terminator (ET).

`FirstStrongIsolate = 21`

First Strong Isolate (FSI).

`LeftToRight = 0`

Left-to-Right (L).

`LeftToRightEmbedding = 14`

Left-to-Right Embedding (LRE).

`LeftToRightIsolate = 19`

Left-to-Right Isolate (LRI).

`LeftToRightOverride = 15`

Left-to-Right Override (LRO).

`NonspacingMark = 8`

Nonspacing Mark (NSM).

`OtherNeutral = 13`

Other Neutral (ON).

`ParagraphSeparator = 10`

Paragraph Separator (B).

`PopDirectionalFormat = 18`

Pop Directional Format (PDF).

`PopDirectionalIsolate = 22`

Pop Directional Isolate (PDI).

`RightToLeft = 1`

Right-to-Left (R).

`RightToLeftEmbedding = 16`

Right-to-Left Embedding (RLE).

`RightToLeftIsolate = 20`

Right-to-Left Isolate (RLI).

`RightToLeftOverride = 17`

Right-to-Left Override (RLO).

`SegmentSeparator = 11`

Segment Separator (S).

`Whitespace = 12`

Whitespace (WS).

Remarks

These are the bidirectional character types used by the Unicode Bidirectional Algorithm. The algorithm resolves these input classes into embedding levels and final visual ordering; the enum represents the original character class.

Struct BidiClass

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Represents the Unicode Bidi value of a given CodePoint. https://unicode.org/reports/tr9/#Table

```
public readonly struct BidiClass
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

BidiClass(CodePoint)

Initializes a new instance of the BidiClass struct.

```
public BidiClass(CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint.

Properties

CharacterType

Gets the Unicode Bidirectional character type.

```
public BidiCharacterType CharacterType { get; }
```

Property Value

BidiCharacterType

PairedBracketType

Gets the Unicode Bidirectional paired bracket type.

```
public BidiPairedBracketType PairedBracketType { get; }
```

Property Value

BidiPairedBracketType

Methods

TryGetPairedBracket(out CodePoint)

Gets the codepoint representing the bracket pairing for this instance.

```
public bool TryGetPairedBracket(out CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

When this method returns, contains the codepoint representing the bracket pairing for this instance; otherwise, the default value for the type of the `codePoint` parameter. This parameter is passed uninitialized. .

Returns

bool

true if this instance has a bracket pairing; otherwise, false

Enum BidiPairedBracketType

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Bidi_Paired_Bracket_Type property values.
https://www.unicode.org/reports/tr9/#Paired_Brackets

```
public enum BidiPairedBracketType
```

Fields

`Close = 2`

Closing paired bracket.

`None = 0`

No paired bracket behavior.

`Open = 1`

Opening paired bracket.

Remarks

UAX #9 uses this property with `Bidi_Paired_Bracket` to find bracket pairs while resolving neutral characters.

Struct CodePoint

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Represents a Unicode value ([U+0000..U+10FFFF], inclusive).

```
public readonly struct CodePoint : IComparable, IComparable<CodePoint>,
IEquatable<CodePoint>
```

Implements

IComparable, IComparable<CodePoint>, IEquatable<CodePoint>

Inherited Members

object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Remarks

This type's constructors and conversion operators validate the input, so consumers can call the APIs assuming that the underlying CodePoint instance is well-formed.

Constructors

CodePoint(char)

Initializes a new instance of the CodePoint struct.

```
public CodePoint(char value)
```

Parameters

`value` char

The char representing the UTF-16 code unit

Exceptions

ArgumentOutOfRangeException

If `value` represents a UTF-16 surrogate code point U+D800..U+DFFF, inclusive.

CodePoint(char, char)

Initializes a new instance of the CodePoint struct.

```
public CodePoint(char highSurrogate, char lowSurrogate)
```

Parameters

`highSurrogate` char

A char representing a UTF-16 high surrogate code unit.

`lowSurrogate` char

A char representing a UTF-16 low surrogate code unit.

Exceptions

ArgumentOutOfRangeException

If `highSurrogate` does not represent a UTF-16 high surrogate code unit or `lowSurrogate` does not represent a UTF-16 low surrogate code unit.

CodePoint(int)

Initializes a new instance of the CodePoint struct.

```
public CodePoint(int value)
```

Parameters

`value` int

The value to create the codepoint.

Exceptions

ArgumentOutOfRangeException

If `value` does not represent a value Unicode scalar value.

CodePoint(uint)

Initializes a new instance of the CodePoint struct.

```
public CodePoint(uint value)
```

Parameters

`value` uint

The value to create the codepoint.

Exceptions

ArgumentOutOfRangeException

If `value` does not represent a value Unicode scalar value.

Properties

IsAscii

Gets a value indicating whether this value is ASCII ([U+0000..U+007F]) and therefore representable by a single UTF-8 code unit.

```
public bool IsAscii { get; }
```

Property Value

bool

IsBmp

Gets a value indicating whether this value is within the BMP ([U+0000..U+FFFF]) and therefore representable by a single UTF-16 code unit.

```
public bool IsBmp { get; }
```

Property Value

bool

ObjectReplacementChar

Gets a CodePoint instance that represents the Unicode object replacement character U+FFFC.

```
public static CodePoint ObjectReplacementChar { get; }
```

Property Value

CodePoint

Plane

Gets the Unicode plane (0 to 16, inclusive) which contains this scalar.

```
public int Plane { get; }
```

Property Value

int

ReplacementChar

Gets a CodePoint instance that represents the Unicode replacement character U+FFFD.

```
public static CodePoint ReplacementChar { get; }
```

Property Value

CodePoint

Utf16SequenceLength

Gets the length in code units (char) of the UTF-16 sequence required to represent this scalar value.

```
public int Utf16SequenceLength { get; }
```

Property Value

int

Remarks

The return value will be 1 or 2.

Utf8SequenceLength

Gets the length in code units of the UTF-8 sequence required to represent this scalar value.

```
public int Utf8SequenceLength { get; }
```

Property Value

int

Remarks

The return value will be 1 through 4, inclusive.

Value

Gets the Unicode value as an integer.

```
public int Value { get; }
```

Property Value

int

Methods

CompareTo(CodePoint)

Compares the current instance with another object of the same type and returns an integer that indicates whether the current instance precedes, follows, or occurs in the same position in the sort order as the other object.

```
public int CompareTo(CodePoint other)
```

Parameters

`other` CodePoint

An object to compare with this instance.

Returns

int

A value that indicates the relative order of the objects being compared. The return value has these meanings:

Value	Meaning
Less than zero	This instance precedes `other` in the sort order.
Zero	This instance occurs in the same position in the sort order as `other`.

Value	Meaning
Greater than zero	This instance follows `other` in the sort order.

Equals(CodePoint)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(CodePoint other)
```

Parameters

`other` CodePoint

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetArabicJoiningClass(in CodePoint)

Gets the ArabicJoiningClass for the given codepoint.

```
public static ArabicJoiningClass GetArabicJoiningClass(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

ArabicJoiningClass

The ArabicJoiningClass.

GetBidiClass(in CodePoint)

Gets the BidiClass for the given codepoint.

```
public static BidiClass GetBidiClass(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

BidiClass

The BidiClass.

GetCanonicalCombiningClass(CodePoint)

Gets the canonical combining class of the given code point.

```
public static int GetCanonicalCombiningClass(CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The code point to evaluate.

Returns

int

The canonical combining class.

GetCodePointCount(ReadOnlySpan<char>)

Returns the number of codepoints in a given string buffer.

```
public static int GetCodePointCount(ReadOnlySpan<char> source)
```

Parameters

`source` ReadOnlySpan<char>

The source buffer to parse.

Returns

int

The int count.

GetEastAsianWidthClass(in CodePoint)

Gets the EastAsianWidthClass for the given codepoint.

```
public static EastAsianWidthClass GetEastAsianWidthClass(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

EastAsianWidthClass

The EastAsianWidthClass.

Remarks

This returns the Unicode East_Asian_Width property value from UAX #11. It does not resolve context-sensitive display-cell width; for example, Ambiguous may resolve as narrow or wide depending on language, script, source encoding, font, or explicit markup.

GetEmojiProperties(in CodePoint)

Gets the EmojiProperties for the given codepoint.

```
public static EmojiProperties GetEmojiProperties(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

EmojiProperties

The EmojiProperties.

GetGeneralCategory(in CodePoint)

Gets the UnicodeCategory for the given codepoint.

```
public static UnicodeCategory GetGeneralCategory(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

UnicodeCategory

The UnicodeCategory.

GetGraphemeClusterClass(in CodePoint)

Gets the GraphemeClusterClass for the given codepoint.

```
public static GraphemeClusterClass GetGraphemeClusterClass(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

GraphemeClusterClass

The GraphemeClusterClass.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

GetIndicConjunctBreakClass(in CodePoint)

Gets the IndicConjunctBreakClass for the given codepoint.

```
public static IndicConjunctBreakClass GetIndicConjunctBreakClass(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

IndicConjunctBreakClass

The IndicConjunctBreakClass.

GetIndicPositionalCategory(in CodePoint)

Gets the IndicPositionalCategory for the given codepoint.

```
public static IndicPositionalCategory GetIndicPositionalCategory(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

IndicPositionalCategory

The IndicPositionalCategory.

GetIndicSyllabicCategory(in CodePoint)

Gets the IndicSyllabicCategory for the given codepoint.

```
public static IndicSyllabicCategory GetIndicSyllabicCategory(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

IndicSyllabicCategory

The IndicSyllabicCategory.

GetLineBreakClass(in CodePoint)

Gets the LineBreakClass for the given codepoint.

```
public static LineBreakClass GetLineBreakClass(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

LineBreakClass

The LineBreakClass.

GetMarkOrderingClass(CodePoint)

Gets the class that orders a mark against the marks around it.

This is the canonical combining class with the classes of several scripts renumbered, so that sorting by it leaves the marks of those scripts in the order they are drawn rather than the order the standard assigns. Hebrew, Arabic, Syriac, Telugu, Thai, Lao and Tibetan all order differently from their assigned classes.

```
public static int GetMarkOrderingClass(CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The code point to evaluate.

Returns

int

The ordering class.

GetScriptClass(in CodePoint)

Gets the ScriptClass for the given codepoint.

```
public static ScriptClass GetScriptClass(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

ScriptClass

The ScriptClass.

GetVerticalOrientationType(in CodePoint)

Gets the VerticalOrientationType for the given codepoint.

```
public static VerticalOrientationType GetVerticalOrientationType(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

VerticalOrientationType

The VerticalOrientationType.

GetWordBreakClass(in CodePoint)

Gets the WordBreakClass for the given codepoint.

```
public static WordBreakClass GetWordBreakClass(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint.

Returns

WordBreakClass

The WordBreakClass.

IsControl(in CodePoint)

Gets a value indicating whether the given codepoint is a control character.

```
public static bool IsControl(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a control character; otherwise, false

IsDigit(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as a decimal digit.

```
public static bool IsDigit(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a decimal digit; otherwise, false

IsLetter(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as a letter.

```
public static bool IsLetter(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a letter; otherwise, false

IsLetterOrDigit(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as a letter or decimal digit.

```
public static bool IsLetterOrDigit(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a letter or decimal digit; otherwise, false

IsLower(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as a lowercase letter.

```
public static bool IsLower(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a lowercase letter; otherwise, false

IsMark(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as a mark.

```
public static bool IsMark(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a symbol; otherwise, false

IsNewLine(in CodePoint)

Gets a value indicating whether the given codepoint is a new line indicator.

```
public static bool IsNewLine(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a new line indicator; otherwise, false

IsNonBreakingSpace(in CodePoint)

Gets a value indicating whether the given codepoint is a non-breaking space.

```
public static bool IsNonBreakingSpace(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a non-breaking space character; otherwise, false

IsNumber(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as a number.

```
public static bool IsNumber(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a number; otherwise, false

IsPunctuation(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as punctuation.

```
public static bool IsPunctuation(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is punctuation; otherwise, false

IsSeparator(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as a separator.

```
public static bool IsSeparator(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a separator; otherwise, false

IsSymbol(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as a symbol.

```
public static bool IsSymbol(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a symbol; otherwise, false

IsTabulation(in CodePoint)

Gets a value indicating whether the given codepoint is a tabulation indicator.

```
public static bool IsTabulation(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a tabulation indicator; otherwise, false

IsUpper(in CodePoint)

Returns a value that indicates whether the specified codepoint is categorized as an uppercase letter.

```
public static bool IsUpper(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a uppercase letter; otherwise, false

IsValid(int)

Returns true if `value` is a valid Unicode code point, i.e., is in [U+0000..U+10FFFF], inclusive.

```
public static bool IsValid(int value)
```

Parameters

`value` int

The value to evaluate.

Returns

bool

true if `value` represents a valid codepoint; otherwise, false

IsValid(uint)

Returns true if `value` is a valid Unicode code point, i.e., is in [U+0000..U+10FFFF], inclusive.

```
public static bool IsValid(uint value)
```

Parameters

`value` uint

The value to evaluate.

Returns

bool

true if `value` represents a valid codepoint; otherwise, false

IsVariationSelector(in CodePoint)

Gets a value indicating whether the given codepoint is a variation selector.
https://en.wikipedia.org/wiki/Variation_Selectors_%28Unicode_block%29

```
public static bool IsVariationSelector(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a variation selector character; otherwise, false

IsWhiteSpace(in CodePoint)

Gets a value indicating whether the given codepoint is white space.

```
public static bool IsWhiteSpace(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a whitespace character; otherwise, false

IsZeroWidthJoiner(in CodePoint)

Gets a value indicating whether the given codepoint is a zero-width-joiner.

```
public static bool IsZeroWidthJoiner(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a zero-width-joiner character; otherwise, false

IsZeroWidthNonJoiner(in CodePoint)

Gets a value indicating whether the given codepoint is a zero-width-non-joiner.

```
public static bool IsZeroWidthNonJoiner(in CodePoint codePoint)
```

Parameters

`codePoint` CodePoint

The codepoint to evaluate.

Returns

bool

true if `codePoint` is a zero-width-non-joiner character; otherwise, false

ToString()

Returns the fully qualified type name of this instance.

```
public override string ToString()
```

Returns

string

The fully qualified type name.

TryCompose(CodePoint, CodePoint, out CodePoint)

Tries to join the given pair of code points into the single code point they are canonically equivalent to.

```
public static bool TryCompose(CodePoint first, CodePoint second, out CodePoint composed)
```

Parameters

`first` CodePoint

The first code point.

`second` CodePoint

The second code point.

`composed` CodePoint

When this method returns, contains the joined code point.

Returns

bool

true if the pair joins.

TryDecompose(CodePoint, out CodePoint, out CodePoint)

Tries to take the given code point apart into the pair of code points it is canonically equivalent to.

```
public static bool TryDecompose(CodePoint codePoint, out CodePoint first, out
CodePoint second)
```

Parameters

`codePoint` CodePoint

The code point to take apart.

`first` CodePoint

When this method returns, contains the first code point.

`second` CodePoint

When this method returns, contains the second code point, or the default when the code point stands for a single other one.

Returns

bool

true if the code point comes apart.

TryGetBidiMirror(in CodePoint, out CodePoint)

Gets the codepoint representing the bidi mirror for this instance.
http://www.unicode.org/reports/tr44/#Bidi_Mirrored

```
public static bool TryGetBidiMirror(in CodePoint codePoint, out CodePoint mirror)
```

Parameters

`codePoint` CodePoint

The code point to be mapped.

`mirror` CodePoint

When this method returns, contains the codepoint representing the bidi mirror for this instance; otherwise, the default value for the type of the `codePoint` parameter. This parameter is passed uninitialized. .

Returns

bool

true if this instance has a mirror; otherwise, false

TryGetVerticalMirror(in CodePoint, out CodePoint)

Gets the codepoint representing the vertical mirror for this instance.
https://www.unicode.org/reports/tr50/#vertical_alternates

```
public static bool TryGetVerticalMirror(in CodePoint codePoint, out CodePoint mirror)
```

Parameters

`codePoint` CodePoint

The code point to be mapped.

`mirror` CodePoint

When this method returns, contains the codepoint representing the vertical mirror for this instance; otherwise, the default value for the type of the `codePoint` parameter. This parameter is passed uninitialized. .

Returns

bool

true if this instance has a mirror; otherwise, false

Operators

operator ==(CodePoint, CodePoint)

```
public static bool operator ==(CodePoint left, CodePoint right)
```

Parameters

`left` CodePoint

`right` CodePoint

Returns

bool

explicit operator CodePoint(char)

```
public static explicit operator CodePoint(char ch)
```

Parameters

`ch` char

Returns

CodePoint

explicit operator CodePoint(int)

```
public static explicit operator CodePoint(int value)
```

Parameters

`value` int

Returns

CodePoint

explicit operator CodePoint(uint)

```
public static explicit operator CodePoint(uint value)
```

Parameters

`value` uint

Returns

CodePoint

operator >(CodePoint, CodePoint)

```
public static bool operator >(CodePoint left, CodePoint right)
```

Parameters

`left` CodePoint

`right` CodePoint

Returns

bool

operator >=(CodePoint, CodePoint)

```
public static bool operator >=(CodePoint left, CodePoint right)
```

Parameters

`left` CodePoint

`right` CodePoint

Returns

bool

operator !=(CodePoint, CodePoint)

```
public static bool operator !=(CodePoint left, CodePoint right)
```

Parameters

`left` CodePoint

`right` CodePoint

Returns

bool

operator <(CodePoint, CodePoint)

```
public static bool operator <(CodePoint left, CodePoint right)
```

Parameters

`left` CodePoint

`right` CodePoint

Returns

bool

operator <=(CodePoint, CodePoint)

```
public static bool operator <=(CodePoint left, CodePoint right)
```

Parameters

`left` CodePoint

`right` CodePoint

Returns

bool

Enum EastAsianWidthClass

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode East_Asian_Width property values. https://www.unicode.org/reports/tr11/

```
public enum EastAsianWidthClass
```

Fields

`Ambiguous = 1`

Ambiguous (A): characters that can be either wide or narrow depending on context.

Ambiguous characters need extra information, such as language, script, font, source encoding, or explicit markup, before they can be resolved to a display width. In East Asian legacy contexts they may be treated as wide; otherwise UAX #11 recommends treating them as narrow by default.

`Fullwidth = 2`

Fullwidth (F): explicitly encoded fullwidth compatibility characters.

Fullwidth characters have a compatibility decomposition of type `<wide>` to another Unicode character that is implicitly narrow. They exist to preserve round-tripping with mixed-width East Asian legacy encodings.

`Halfwidth = 3`

Halfwidth (H): explicitly encoded halfwidth compatibility characters.

Halfwidth characters have a compatibility decomposition of type `<narrow>` to another Unicode character that is implicitly wide, with the special case of U+20A9 WON SIGN. They are distinct from ordinary narrow characters because they can still behave like East Asian compatibility forms for font selection and some punctuation behavior.

`Narrow = 4`

Narrow (Na): characters that are always narrow and have explicit wide or fullwidth counterparts.

Narrow characters are implicitly narrow in East Asian typography and legacy character sets. ASCII is the common example: the ordinary ASCII code points are Narrow, while their compatibility forms are Fullwidth.

```
Neutral = 0
```

Neutral (N): characters that are not East Asian for this property.

Neutral characters are not normally found in legacy East Asian character sets or traditional East Asian typography. UAX #11 recommends treating them like narrow characters for practical resolved-width decisions, but the property value itself is distinct from Narrow.

```
Wide = 5
```

Wide (W): characters that are always wide in East Asian typography.

Wide characters behave like ideographs for East Asian layout. This includes many Han, Kana, Hangul, and emoji-presentation characters that are not encoded as explicit Fullwidth compatibility forms.

Remarks

East_Asian_Width is a Unicode character property used when interoperating with East Asian legacy encodings and typography. The property has six default values, but any operation that needs a display-cell width must still resolve those values to a context-specific narrow or wide result.

Enum EmojiProperties

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Binary Unicode emoji properties from UTS #51.

```
[Flags]
public enum EmojiProperties : byte
```

Fields

`Emoji = 1`

The scalar has the Emoji property.

`EmojiComponent = 16`

The scalar is used as a component in emoji sequences.

`EmojiKeycapSequenceBase = 128`

The scalar can start an emoji keycap sequence.

`EmojiModifier = 4`

The scalar is an emoji modifier.

`EmojiModifierBase = 8`

The scalar can be followed by an emoji modifier.

`EmojiPresentation = 2`

The scalar defaults to emoji presentation.

`EmojiPresentationSequenceBase = 64`

The scalar can be followed by U+FE0F to request emoji presentation.

`None = 0`

No emoji properties.

```
TextPresentationSequenceBase = 32
```

The scalar can be followed by U+FE0E to request text presentation.

Struct GraphemeCluster

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Represents a Unicode grapheme cluster and metadata derived while enumerating it.

```
public readonly ref struct GraphemeCluster
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

GraphemeCluster(ReadOnlySpan<char>, int, int, int, GraphemeClusterFlags, CodePoint)

Initializes a new instance of the GraphemeCluster struct.

```
public GraphemeCluster(ReadOnlySpan<char> span, int utf16Offset, int codePointCount, int
terminalCellWidth, GraphemeClusterFlags flags, CodePoint firstCodePoint)
```

Parameters

`span` ReadOnlySpan<char>

The UTF-16 span containing the grapheme cluster.

`utf16Offset` int

The UTF-16 offset of the cluster in the original source.

`codePointCount` int

The number of Unicode scalar values in the cluster.

`terminalCellWidth` int

The policy-resolved terminal cell width of the cluster.

`flags` GraphemeClusterFlags

The cluster flags derived while scanning the cluster.

`firstCodePoint` CodePoint

The first code point in the cluster.

Properties

CodePointCount

Gets the number of Unicode scalar values in the cluster.

```
public int CodePointCount { get; }
```

Property Value

int

FirstCodePoint

Gets the first code point in the cluster.

```
public CodePoint FirstCodePoint { get; }
```

Property Value

CodePoint

Flags

Gets the cluster flags derived while scanning the cluster.

```
public GraphemeClusterFlags Flags { get; }
```

Property Value

GraphemeClusterFlags

Span

Gets the UTF-16 span containing the grapheme cluster.

```
public ReadOnlySpan<char> Span { get; }
```

Property Value

ReadOnlySpan<char>

TerminalCellWidth

Gets the policy-resolved terminal cell width of the cluster.

```
public int TerminalCellWidth { get; }
```

Property Value

int

Utf16Length

Gets the UTF-16 length of the cluster.

```
public int Utf16Length { get; }
```

Property Value

int

Utf16Offset

Gets the UTF-16 offset of the cluster in the original source.

```
public int Utf16Offset { get; }
```

Property Value

int

Enum GraphemeClusterClass

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Grapheme_Cluster_Break property values and local rule sentinels.
https://www.unicode.org/reports/tr29/#Grapheme_Cluster_Break_Property_Values

```
public enum GraphemeClusterClass
```

Fields

`Any = 0`

Rule sentinel that matches any code point.

This is not a Unicode property value; it represents the "any" operand in UAX #29 boundary rules.

`CarriageReturn = 1`

U+000D CARRIAGE RETURN (CR).

`Control = 3`

Controls, separators, formats, and default-ignorable unassigned code points that form hard grapheme cluster boundaries.

This class excludes CR, LF, U+200C ZERO WIDTH NON-JOINER, U+200D ZERO WIDTH JOINER, and prepended concatenation marks because those participate in more specific UAX #29 rules.

`Extend = 4`

Extending code points that remain in the same extended grapheme cluster as the preceding base.

This includes Grapheme_Extend code points, emoji modifiers, U+200C ZERO WIDTH NON-JOINER, and a small number of spacing marks needed for canonical equivalence.

`ExtendedPictographic = 13`

Extended pictographic code points used by GB11 emoji ZWJ sequence handling.

This is not itself a Grapheme_Cluster_Break property value; UAX #29 uses it when matching emoji ZWJ sequences.

`HangulLead = 8`

Hangul leading consonant Jamo (Hangul_Syllable_Type = L).

`HangulLeadVowel = 11`

Hangul LV syllables.

`HangulLeadVowelTail = 12`

Hangul LVT syllables.

`HangulTail = 10`

Hangul trailing consonant Jamo (Hangul_Syllable_Type = T).

`HangulVowel = 9`

Hangul vowel Jamo (Hangul_Syllable_Type = V).

`LineFeed = 2`

U+000A LINE FEED (LF).

`Other = 255`

Other.

This is the Unicode `Other` / `XX` fallback for code points without an explicit grapheme cluster break class.

`Prepend = 6`

Code points that prepend to the following grapheme cluster.

This includes Indic_Syllabic_Category values Consonant_Preceding_Repha and Consonant_Prefixed, plus Prepended_Concatenation_Mark code points.

`RegionalIndicator = 5`

Regional indicator symbols used to build flag emoji pairs.

`SpacingMark = 7`

Spacing marks that extend the previous grapheme cluster.

This includes spacing marks whose Grapheme_Cluster_Break value is not Extend, plus U+0E33 THAI CHARACTER SARA AM and U+0EB3 LAO VOWEL SIGN AM.

`ZeroWidthJoiner = 14`

U+200D ZERO WIDTH JOINER.

Remarks

UAX #29 uses these classes in ordered boundary rules to determine extended grapheme clusters. Some members are rule sentinels rather than Unicode property values exposed by the standard.

Enum GraphemeClusterFlags

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Flags describing properties of a Unicode grapheme cluster.

```
[Flags]
public enum GraphemeClusterFlags : byte
```

Fields

`AllZeroWidth = 4`

All scalars in the cluster are zero-width for terminal measurement.

`ContainsAmbiguous = 2`

At least one scalar in the cluster has East Asian Width Ambiguous.

`ContainsControl = 8`

The cluster contains a C0 or C1 control scalar.

`ContainsEmoji = 16`

The cluster contains an emoji-like scalar or sequence.

`ContainsVariationSelector = 64`

The cluster contains a variation selector.

`ContainsWide = 1`

At least one scalar in the cluster resolved to two terminal cells before any whole-cluster emoji override was applied.

`ContainsZwjSequence = 32`

The cluster contains a zero-width joiner sequence.

`IsSingleCodePoint = 128`

The cluster contains exactly one code point.

```
None = 0
```

No flags.

Enum IndicConjunctBreakClass

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Indic_Conjunct_Break property values used by UAX #29 GB9c.

```
public enum IndicConjunctBreakClass
```

Fields

`Consonant = 1`

Consonant: a code point that can start or continue an Indic conjunct sequence.

In GB9c this is the stable anchor on either side of the linker sequence.

`Extend = 2`

Extend: a combining or extending code point that is transparent inside an Indic conjunct sequence.

Extend code points can appear between the consonant and linker without ending the conjunct sequence.

`Linker = 3`

Linker: a code point, such as a virama, that connects a following consonant.

A linker keeps the following consonant in the same extended grapheme cluster when the surrounding GB9c pattern matches.

`None = 0`

No Indic conjunct break behavior. This is the default for code points that do not participate in GB9c.

Remarks

UAX #29 GB9c uses this property to keep matching consonant-linker-consonant sequences inside one extended grapheme cluster.

https://www.unicode.org/reports/tr29/#Grapheme_Cluster_Boundary_Rules

Enum IndicPositionalCategory

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Indic_Positional_Category property values.
https://www.unicode.org/reports/tr44/#Indic_Positional_Category

```
public enum IndicPositionalCategory
```

Fields

`Bottom = 0`

Bottom.

`BottomAndLeft = 1`

Bottom_And_Left.

`BottomAndRight = 2`

Bottom_And_Right.

`Left = 3`

Left.

`LeftAndRight = 4`

Left_And_Right.

`NA = 255`

Not applicable.

This is the Unicode fallback for code points without an explicit Indic_Positional_Category.

`Overstruck = 6`

Overstruck.

`Right = 7`

Right.

`Top = 8`

Top.

`TopAndBottom = 9`

Top_And_Bottom.

`TopAndBottomAndLeft = 10`

Top_And_Bottom_And_Left.

`TopAndBottomAndRight = 11`

Top_And_Bottom_And_Right.

`TopAndLeft = 12`

Top_And_Left.

`TopAndLeftAndRight = 13`

Top_And_Left_And_Right.

`TopAndRight = 14`

Top_And_Right.

`VisualOrderLeft = 15`

Visual_Order_Left.

Remarks

These values describe the notional slot of dependent vowels, visible viramas, and related signs around an Indic syllable core. The property supplements IndicSyllabicCategory for shaping and segmentation, but it is not a prescriptive font-design or final glyph-placement property. NA is the default for code points whose syllabic role does not have a positional slot.

Enum IndicSyllabicCategory

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Indic_Syllabic_Category property values.
https://www.unicode.org/reports/tr44/#Indic_Syllabic_Category

```
public enum IndicSyllabicCategory
```

Fields

`Avagraha = 0`

Avagraha.

`Bindu = 1`

Bindu.

`BrahmiJoiningNumber = 2`

Brahmi_Joining_Number.

`CantillationMark = 3`

Cantillation_Mark.

`Consonant = 4`

Consonant

`ConsonantDead = 5`

Consonant_Dead

`ConsonantFinal = 6`

Consonant_Final

`ConsonantHeadLetter = 7`

Consonant_Head_Letter

`ConsonantInitialPostfixed = 8`

Consonant_Initial_Postfixed

`ConsonantKiller = 9`

Consonant_Killer

`ConsonantMedial = 10`

Consonant_Medial

`ConsonantPlaceholder = 11`

Consonant_Placeholder

`ConsonantPrecedingRepha = 12`

Consonant_Preceding_Repha

`ConsonantPrefixed = 13`

Consonant_Prefixed

`ConsonantSubjoined = 14`

Consonant_Subjoined

`ConsonantSucceedingRepha = 15`

Consonant_Succeeding_Repha

`ConsonantWithStacker = 16`

Consonant_With_Stacker

`GeminationMark = 17`

Gemination_Mark

`InvisibleStacker = 18`

Invisible_Stacker

`Joiner = 19`

Joiner

`ModifyingLetter = 20`

Modifying_Letter

`NonJoiner = 21`

Non_Joiner

`Nukta = 22`

Nukta

`Number = 23`

Number

`NumberJoiner = 24`

Number_Joiner

`Other = 255`

Other.

This is the Unicode fallback for code points without an explicit Indic_Syllabic_Category.

`PureKiller = 26`

Pure_Killer

`RegisterShifter = 27`

Register_Shifter

`ReorderingKiller = 28`

Reordering_Killer

`SyllableModifier = 29`

Syllable_Modifier

`ToneLetter = 30`

Tone_Letter

`ToneMark = 31`

Tone_Mark

`Virama = 32`

Virama

`Visarga = 33`

Visarga

`Vowel = 34`

Vowel

`VowelDependent = 35`

Vowel_Dependent

`VowelIndependent = 36`

Vowel_Independent

Remarks

These values describe subtypes relevant to Indic syllable, or aksara, construction and segmentation.

Enum LineBreakClass

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Line_Break property values. https://www.unicode.org/reports/tr14/#Table1

```
public enum LineBreakClass : uint
```

Fields

`Aksara = 43`

Aksara (AK): consonants that form orthographic syllables in Brahmic scripts.

`AksaraPrebase = 44`

Aksara pre-base (AP): pre-base signs, such as repha, that form Brahmic orthographic syllables.

`AksaraStart = 45`

Aksara start (AS): independent vowels and related starters for Brahmic orthographic syllables.

`Alphabetic = 12`

Alphabetic (AL): letters and ordinary symbols that use alphabetic line breaking behavior.

`Ambiguous = 33`

Ambiguous alphabetic or ideographic (AI): resolved by LB1 before rule evaluation.

`BreakAfter = 17`

Break after (BA): characters that generally provide a line break opportunity after themselves.

`BreakBefore = 18`

Break before (BB): characters that generally provide a line break opportunity before themselves.

`BreakBeforeAndAfter = 19`

Break before and after (B2): characters that allow breaks on either side, but not between two B2 characters.

`BreakSymbols = 7`

Symbols allowing break after (SY): characters that prevent a line break before and allow one after.

`CarriageReturn = 36`

Carriage return (CR): causes a line break after itself except in a CR LF sequence.

`CloseParenthesis = 2`

Close parenthesis (CP): closing bracket and parenthesis characters that prohibit a line break before them.

`ClosePunctuation = 1`

Close punctuation (CL): characters that prohibit a line break before them.

`CombiningMark = 21`

Combining mark (CM): combining marks and related controls that stay with the preceding character.

`ComplexContext = 39`

Complex context dependent (SA): requires language-specific analysis for line break opportunities.

`ConditionalJapaneseStarter = 35`

Conditional Japanese starter (CJ): small Kana and related characters resolved by tailoring.

`ContingentBreak = 32`

Contingent break (CB): break opportunity determined by additional layout information.

`EmojiBase = 29`

Emoji base (EB): emoji characters that must not break from a following emoji modifier.

`EmojiModifier = 30`

Emoji modifier (EM): emoji modifiers that must not break from a preceding emoji base.

`Exclamation = 6`

Exclamation or interrogation (EX): sentence punctuation that prohibits a line break before itself.

`Glue = 4`

Non-breaking glue (GL): characters that prohibit line breaks before and after themselves.

`HangulLeadJamo = 25`

Hangul L Jamo (JL): leading consonant Jamo used to form Korean syllable blocks.

`HangulLeadVowelSyllable = 23`

Hangul LV syllable (H2): part of the Hangul sequence classes used to form Korean syllable blocks.

`HangulLeadVowelTailSyllable = 24`

Hangul LVT syllable (H3): part of the Hangul sequence classes used to form Korean syllable blocks.

`HangulTailJamo = 27`

Hangul T Jamo (JT): trailing consonant Jamo used to form Korean syllable blocks.

`HangulVowelJamo = 26`

Hangul V Jamo (JV): vowel Jamo used to form Korean syllable blocks.

`HebrewLetter = 13`

Hebrew letter (HL): Hebrew characters with special hyphen and solidus behavior.

`Hyphen = 16`

Hyphen (HY): hyphen-minus and similar characters that allow breaks after except in numeric context.

`Ideographic = 14`

Ideographic (ID): ideographic characters that generally allow breaks before or after.

`InfixNumeric = 8`

Infix numeric separator (IS): characters that suppress breaks inside numeric expressions.

`Inseparable = 15`

Inseparable (IN): characters, such as leaders, that allow only indirect line breaks between pairs.

`LineFeed = 37`

Line feed (LF): causes a line break after itself.

`MandatoryBreak = 34`

Mandatory break (BK): characters that cause a line break after themselves.

`NextLine = 38`

Next line (NL): causes a line break after itself.

`Nonstarter = 5`

Nonstarter (NS): characters that allow only indirect line breaks before themselves.

`Numeric = 11`

Numeric (NU): digits and related characters that form numeric expressions.

`OpenPunctuation = 0`

Open punctuation (OP): characters that prohibit a line break after them.

`PostfixNumeric = 10`

Postfix numeric (PO): characters that stay with a preceding numeric expression.

`PrefixNumeric = 9`

Prefix numeric (PR): characters that stay with a following numeric expression.

`Quotation = 3`

Quotation (QU): quotation marks that can behave as opening punctuation, closing punctuation, or both.

`RegionalIndicator = 28`

Regional indicator (RI): symbols paired for flag sequences.

`Space = 41`

Space (SP): enables indirect line breaks.

`Surrogate = 40`

Surrogate (SG): surrogate code points, which do not occur in well-formed Unicode scalar text.

`UnambiguousHyphen = 46`

Unambiguous hyphen (HH): hyphen characters with unambiguous break-after behavior except word-initially.

`Unknown = 255`

Unknown (XX): code points without an explicit line break class.

`Virama = 48`

Virama (VI): conjoining viramas that form Brahmic orthographic syllables.

`ViramaFinal = 47`

Virama final (VF): final consonant viramas that form Brahmic orthographic syllables.

`WordJoiner = 22`

Word joiner (WJ): characters that prohibit line breaks before and after themselves.

`ZeroWidthJoiner = 31`

Zero width joiner (ZWJ): joiner control that prohibits breaks inside joiner sequences.

`ZeroWidthSpace = 20`

Zero width space (ZW): an explicit opportunity for a line break.

Class MemoryExtensions

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Contains extension methods for memory types.

```
public static class MemoryExtensions
```

Inheritance

object ← MemoryExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

EnumerateCodePoints(ReadOnlySpan<char>)

Returns an enumeration of CodePoint from the provided span.

```
public static SpanCodePointEnumerator EnumerateCodePoints(this ReadOnlySpan<char> span)
```

Parameters

`span` ReadOnlySpan<char>

The read-only span of char elements representing the text to enumerate.

Returns

SpanCodePointEnumerator

The SpanCodePointEnumerator.

Remarks

Invalid UTF-16 sequences will be represented in the enumeration by ReplacementChar.

EnumerateCodePoints(Span<char>)

Returns an enumeration of CodePoint from the provided span.

```
public static SpanCodePointEnumerator EnumerateCodePoints(this Span<char> span)
```

Parameters

`span` Span<char>

The span of char elements representing the text to enumerate.

Returns

SpanCodePointEnumerator

The SpanCodePointEnumerator.

Remarks

Invalid UTF-16 sequences will be represented in the enumeration by ReplacementChar.

EnumerateGraphemes(ReadOnlySpan<char>)

Returns an enumeration of grapheme clusters from the provided span.

```
public static SpanGraphemeEnumerator EnumerateGraphemes(this ReadOnlySpan<char> span)
```

Parameters

`span` ReadOnlySpan<char>

The read-only span of char elements representing the text to enumerate.

Returns

SpanGraphemeEnumerator

The SpanGraphemeEnumerator.

Remarks

Invalid UTF-16 sequences are treated as ReplacementChar while determining grapheme boundaries.

EnumerateGraphemes(ReadOnlySpan<char>, TerminalWidthOptions)

Returns an enumeration of grapheme clusters from the provided span with terminal-width metadata.

```
public static SpanGraphemeEnumerator EnumerateGraphemes(this ReadOnlySpan<char> span,
TerminalWidthOptions terminalWidthOptions)
```

Parameters

`span` ReadOnlySpan<char>

The read-only span of char elements representing the text to enumerate.

`terminalWidthOptions` TerminalWidthOptions

The terminal width options used to resolve TerminalCellWidth.

Returns

SpanGraphemeEnumerator

The SpanGraphemeEnumerator.

Remarks

Invalid UTF-16 sequences are treated as ReplacementChar while determining grapheme boundaries. Terminal width options affect only the width metadata on each returned cluster; they do not affect grapheme segmentation.

EnumerateGraphemes(Span<char>)

Returns an enumeration of grapheme clusters from the provided span.

```
public static SpanGraphemeEnumerator EnumerateGraphemes(this Span<char> span)
```

Parameters

`span` Span<char>

The span of char elements representing the text to enumerate.

Returns

SpanGraphemeEnumerator

The SpanGraphemeEnumerator.

Remarks

Invalid UTF-16 sequences are treated as ReplacementChar while determining grapheme boundaries.

EnumerateGraphemes(Span<char>, TerminalWidthOptions)

Returns an enumeration of grapheme clusters from the provided span with terminal-width metadata.

```
public static SpanGraphemeEnumerator EnumerateGraphemes(this Span<char> span,
TerminalWidthOptions terminalWidthOptions)
```

Parameters

`span` Span<char>

The span of char elements representing the text to enumerate.

`terminalWidthOptions` TerminalWidthOptions

The terminal width options used to resolve TerminalCellWidth.

Returns

SpanGraphemeEnumerator

The SpanGraphemeEnumerator.

Remarks

Invalid UTF-16 sequences are treated as ReplacementChar while determining grapheme boundaries. Terminal width options affect only the width metadata on each returned cluster; they do not affect grapheme segmentation.

EnumerateWordSegments(ReadOnlySpan<char>)

Returns an enumeration of Unicode word-boundary segments from the provided span.

```
public static SpanWordEnumerator EnumerateWordSegments(this ReadOnlySpan<char> span)
```

Parameters

`span` ReadOnlySpan<char>

The read-only span of char elements representing the text to enumerate.

Returns

SpanWordEnumerator

The SpanWordEnumerator.

Remarks

Invalid UTF-16 sequences are treated as ReplacementChar while determining word boundaries.

EnumerateWordSegments(Span<char>)

Returns an enumeration of Unicode word-boundary segments from the provided span.

```
public static SpanWordEnumerator EnumerateWordSegments(this Span<char> span)
```

Parameters

`span` Span<char>

The span of char elements representing the text to enumerate.

Returns

SpanWordEnumerator

The SpanWordEnumerator.

Remarks

Invalid UTF-16 sequences are treated as ReplacementChar while determining word boundaries.

GetCodePointCount(ReadOnlySpan<char>)

Returns the number of code points in the provided span.

```
public static int GetCodePointCount(this ReadOnlySpan<char> span)
```

Parameters

`span` ReadOnlySpan<char>

The read-only span of char elements representing the text to enumerate.

Returns

int

The number of code points.

GetCodePointCount(Span<char>)

Returns the number of code points in the provided span.

```
public static int GetCodePointCount(this Span<char> span)
```

Parameters

`span` Span<char>

The span of char elements representing the text to enumerate.

Returns

int

The number of code points.

GetCodePointCount(string)

Returns the number of code points in the provided text.

```
public static int GetCodePointCount(this string text)
```

Parameters

`text` string

The text to enumerate.

Returns

int

The number of code points.

GetGraphemeCount(ReadOnlySpan<char>)

Returns the number of grapheme clusters in the provided span.

```
public static int GetGraphemeCount(this ReadOnlySpan<char> span)
```

Parameters

`span` ReadOnlySpan<char>

The read-only span of char elements representing the text to enumerate.

Returns

int

The number of grapheme clusters.

GetGraphemeCount(Span<char>)

Returns the number of grapheme clusters in the provided span.

```
public static int GetGraphemeCount(this Span<char> span)
```

Parameters

`span` Span<char>

The span of char elements representing the text to enumerate.

Returns

int

The number of grapheme clusters.

GetGraphemeCount(string)

Returns the number of grapheme clusters in the provided text.

```
public static int GetGraphemeCount(this string text)
```

Parameters

`text` string

The text to enumerate.

Returns

int

The number of grapheme clusters.

GetTerminalCellWidth(ReadOnlySpan<char>)

Returns the terminal cell width of the provided span.

```
public static int GetTerminalCellWidth(this ReadOnlySpan<char> span)
```

Parameters

`span` ReadOnlySpan<char>

The read-only span of char elements representing the text to measure.

Returns

int

The terminal cell width, or `-1` when the configured control-character policy treats any grapheme cluster in the text as non-printable.

GetTerminalCellWidth(ReadOnlySpan<char>, TerminalWidthOptions)

Returns the terminal cell width of the provided span.

```
public static int GetTerminalCellWidth(this ReadOnlySpan<char> span,
TerminalWidthOptions terminalWidthOptions)
```

Parameters

`span` ReadOnlySpan<char>

The read-only span of char elements representing the text to measure.

`terminalWidthOptions` TerminalWidthOptions

The terminal width options to apply while measuring.

Returns

int

The terminal cell width, or `-1` when the configured control-character policy treats any grapheme cluster in the text as non-printable.

GetTerminalCellWidth(Span<char>)

Returns the terminal cell width of the provided span.

```
public static int GetTerminalCellWidth(this Span<char> span)
```

Parameters

`span` Span<char>

The span of char elements representing the text to measure.

Returns

int

The terminal cell width, or `-1` when the configured control-character policy treats any grapheme cluster in the text as non-printable.

GetTerminalCellWidth(Span<char>, TerminalWidthOptions)

Returns the terminal cell width of the provided span.

```
public static int GetTerminalCellWidth(this Span<char> span, TerminalWidthOptions
terminalWidthOptions)
```

Parameters

`span` Span<char>

The span of char elements representing the text to measure.

`terminalWidthOptions` TerminalWidthOptions

The terminal width options to apply while measuring.

Returns

int

The terminal cell width, or `-1` when the configured control-character policy treats any grapheme cluster in the text as non-printable.

GetTerminalCellWidth(string)

Returns the terminal cell width of the provided text.

```
public static int GetTerminalCellWidth(this string text)
```

Parameters

`text` string

The text to measure.

Returns

int

The terminal cell width, or `-1` when the configured control-character policy treats any grapheme cluster in the text as non-printable.

GetTerminalCellWidth(string, TerminalWidthOptions)

Returns the terminal cell width of the provided text.

```
public static int GetTerminalCellWidth(this string text, TerminalWidthOptions
terminalWidthOptions)
```

Parameters

`text` string

The text to measure.

`terminalWidthOptions` TerminalWidthOptions

The terminal width options to apply while measuring.

Returns

int

The terminal cell width, or `-1` when the configured control-character policy treats any grapheme cluster in the text as non-printable.

Enum ScriptClass

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Script values used when shaping text. https://www.unicode.org/reports/tr24/#Script

```
public enum ScriptClass
```

Fields

`Adlam = 3`

Shortcode: Adlm

`Ahom = 5`

Shortcode: Ahom

`AnatolianHieroglyphs = 55`

Shortcode: Hluw

`Arabic = 6`

Shortcode: Arab

`Armenian = 8`

Shortcode: Armn

`Avestan = 9`

Shortcode: Avst

`Balinese = 10`

Shortcode: Bali

`Bamum = 11`

Shortcode: Bamu

`BassaVah = 12`

Shortcode: Bass

`Batak = 13`

Shortcode: Batk

`Bengali = 14`

Shortcode: Beng

`BeriaErfe = 163`

Shortcode: Berf

`Bhaiksuki = 15`

Shortcode: Bhks

`Bopomofo = 16`

Shortcode: Bopo

`Brahmi = 17`

Shortcode: Brah

`Braille = 18`

Shortcode: Brai

`Buginese = 19`

Shortcode: Bugi

`Buhid = 20`

Shortcode: Buhd

`CanadianAboriginal = 22`

Shortcode: Cans

`Carian = 23`

Shortcode: Cari

`CaucasianAlbanian = 4`

Shortcode: Aghb

`Chakma = 21`

Shortcode: Cakm

`Cham = 24`

Shortcode: Cham

`Cherokee = 25`

Shortcode: Cher

`Chorasmian = 26`

Shortcode: Chrs

`Common = 1`

Common script. Shortcode: Zyyy.

`Coptic = 27`

Shortcode: Copt, Qaac

`Cuneiform = 159`

Shortcode: Xsux

`Cypriot = 29`

Shortcode: Cprt

`CyproMinoan = 28`

Shortcode: Cpmn

`Cyrillic = 30`

Shortcode: Cyrl

`Default = 999`

OpenType default script tag. Shortcode: DFLT.

Deseret = 34

Shortcode: Dsrt

Devanagari = 31

Shortcode: Deva

DivesAkuru = 32

Shortcode: Diak

Dogra = 33

Shortcode: Dogr

Duployan = 35

Shortcode: Dupl

EgyptianHieroglyphs = 36

Shortcode: Egyp

Elbasan = 37

Shortcode: Elba

Elymaic = 38

Shortcode: Elym

Ethiopic = 39

Shortcode: Ethi

Garay = 164

Shortcode: Gara

Georgian = 40

Shortcode: Geor

Glagolitic = 41

Shortcode: Glag

Gothic = 44

Shortcode: Goth

Grantha = 45

Shortcode: Gran

Greek = 46

Shortcode: Grek

Gujarati = 47

Shortcode: Gujr

GunjalaGondi = 42

Shortcode: Gong

Gurmukhi = 48

Shortcode: Guru

GurungKhema = 165

Shortcode: Gukh

Han = 50

Shortcode: Hani

Hangul = 49

Shortcode: Hang

HanifiRohingya = 119

Shortcode: Rohg

Hanunoo = 51

Shortcode: Hano

Hatran = 52

Shortcode: Hatr

```
Hebrew = 53
```

Shortcode: Hebr

```
Hiragana = 54
```

Shortcode: Hira

```
ImperialAramaic = 7
```

Shortcode: Armi

```
Inherited = 2
```

Inherited script. Shortcode: Zinh, Qaai.

```
InscriptionalPahlavi = 113
```

Shortcode: Phli

```
InscriptionalParthian = 117
```

Shortcode: Prti

```
Javanese = 61
```

Shortcode: Java

```
Kaithi = 69
```

Shortcode: Kthi

```
Kannada = 68
```

Shortcode: Knda

```
Katakana = 63
```

Shortcode: Kana

```
KatakanaOrHiragana = 58
```

Shortcode: Hrkt

```
Kawi = 166
```

Shortcode: Kawi

`KayahLi = 62`

Shortcode: Kali

`Kharoshthi = 64`

Shortcode: Khar

`KhitanSmallScript = 67`

Shortcode: Kits

`Khmer = 65`

Shortcode: Khmr

`Khojki = 66`

Shortcode: Khoj

`Khudawadi = 128`

Shortcode: Sind

`KiratRai = 167`

Shortcode: Krai

`Lao = 71`

Shortcode: Laoo

`Latin = 72`

Shortcode: Latn

`Lepcha = 73`

Shortcode: Lepc

`Limbu = 74`

Shortcode: Limb

`LinearA = 75`

Shortcode: Lina

`LinearB = 76`

Shortcode: Linb

`Lisu = 77`

Shortcode: Lisu

`Lycian = 78`

Shortcode: Lyci

`Lydian = 79`

Shortcode: Lydi

`Mahajani = 80`

Shortcode: Mahj

`Makasar = 81`

Shortcode: Maka

`Malayalam = 89`

Shortcode: Mlym

`Mandaic = 82`

Shortcode: Mand

`Manichaean = 83`

Shortcode: Mani

`Marchen = 84`

Shortcode: Marc

`MasaramGondi = 43`

Shortcode: Gonm

`Medefaidrin = 85`

Shortcode: Medf

`MeeteiMayek = 93`

Shortcode: Mtei

`MendeKikakui = 86`

Shortcode: Mend

`MeroiticCursive = 87`

Shortcode: Merc

`MeroiticHieroglyphs = 88`

Shortcode: Mero

`Miao = 116`

Shortcode: Plrd

`Modi = 90`

Shortcode: Modi

`Mongolian = 91`

Shortcode: Mong

`Mro = 92`

Shortcode: Mroo

`Multani = 94`

Shortcode: Mult

`Myanmar = 95`

Shortcode: Mymr

`MyanmarZawgyi = 1000`

Myanmar encoded using the legacy Zawgyi convention. Shortcode: Qaag.

`Nabataean = 98`

Shortcode: Nbat

NagMundari = 168

Shortcode: Nagm

Nandinagari = 96

Shortcode: Nand

NewTaiLue = 140

Shortcode: Talu

Newa = 99

Shortcode: Newa

Nko = 100

Shortcode: Nkoo

Nushu = 101

Shortcode: Nshu

NyiakengPuachueHmong = 57

Shortcode: Hmnp

Ogham = 102

Shortcode: Ogam

OlChiki = 103

Shortcode: Olck

OlOnal = 169

Shortcode: Onao

OldHungarian = 59

Shortcode: Hung

OldItalic = 60

Shortcode: Ital

`OldNorthArabian = 97`

Shortcode: Narb

`OldPermic = 111`

Shortcode: Perm

`OldPersian = 158`

Shortcode: Xpeo

`OldSogdian = 131`

Shortcode: Sogo

`OldSouthArabian = 122`

Shortcode: Sarb

`OldTurkic = 104`

Shortcode: Orkh

`OldUyghur = 108`

Shortcode: Ougr

`Oriya = 105`

Shortcode: Orya

`Osage = 106`

Shortcode: Osge

`Osmanya = 107`

Shortcode: Osma

`PahawhHmong = 56`

Shortcode: Hmng

`Palmyrene = 109`

Shortcode: Palm

PauCinHau = 110

Shortcode: Pauc

PhagsPa = 112

Shortcode: Phag

Phoenician = 115

Shortcode: Phnx

PsalterPahlavi = 114

Shortcode: Phlp

Rejang = 118

Shortcode: Rjng

Runic = 120

Shortcode: Runr

Samaritan = 121

Shortcode: Samr

Saurashtra = 123

Shortcode: Saur

Sharada = 126

Shortcode: Shrd

Shavian = 125

Shortcode: Shaw

Siddham = 127

Shortcode: Sidd

Sidetic = 170

Shortcode: Sidt

`SignWriting = 124`

Shortcode: Sgnw

`Sinhala = 129`

Shortcode: Sinh

`Sogdian = 130`

Shortcode: Sogd

`SoraSompeng = 132`

Shortcode: Sora

`Soyombo = 133`

Shortcode: Soyo

`Sundanese = 134`

Shortcode: Sund

`Sunuwar = 171`

Shortcode: Sunu

`SylotiNagri = 135`

Shortcode: Sylo

`Syriac = 136`

Shortcode: Syrc

`Tagalog = 146`

Shortcode: Tglg

`Tagbanwa = 137`

Shortcode: Tagb

`TaiLe = 139`

Shortcode: Tale

`TaiTham = 70`

Shortcode: Lana

`TaiViet = 143`

Shortcode: Tavt

`TaiYo = 172`

Shortcode: Tayo

`Takri = 138`

Shortcode: Takr

`Tamil = 141`

Shortcode: Taml

`Tangsa = 151`

Shortcode: Tnsa

`Tangut = 142`

Shortcode: Tang

`Telugu = 144`

Shortcode: Telu

`Thaana = 147`

Shortcode: Thaa

`Thai = 148`

Shortcode: Thai

`Tibetan = 149`

Shortcode: Tibt

`Tifinagh = 145`

Shortcode: Tfng

`Tirhuta = 150`

Shortcode: Tirh

`Todhri = 173`

Shortcode: Todr

`TolongSiki = 174`

Shortcode: Tols

`Toto = 152`

Shortcode: Toto

`TuluTigalari = 175`

Shortcode: Tutg

`Ugaritic = 153`

Shortcode: Ugar

`Unknown = 0`

Unknown script. Shortcode: Zzzz.

`Vai = 154`

Shortcode: Vaii

`Vithkuqi = 155`

Shortcode: Vith

`Wancho = 157`

Shortcode: Wcho

`WarangCiti = 156`

Shortcode: Wara

`Yezidi = 160`

Shortcode: Yezi

`Yi = 161`

Shortcode: Yiii

`ZanabazarSquare = 162`

Shortcode: Zanb

Remarks

Common and Inherited are Unicode script values. Default is an OpenType fallback tag rather than a Unicode Script property value. MyanmarZawgyi explicitly selects shaping for the legacy Zawgyi encoding and is not a Unicode Script property value.

Struct SpanCodePointEnumerator

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

An enumerator for retrieving CodePoint instances from a ReadOnlySpan<T>. Methods are pattern-matched by compiler to allow using foreach pattern.

```
public ref struct SpanCodePointEnumerator
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

SpanCodePointEnumerator(ReadOnlySpan<char>)

Initializes a new instance of the SpanCodePointEnumerator struct.

```
public SpanCodePointEnumerator(ReadOnlySpan<char> source)
```

Parameters

`source` ReadOnlySpan<char>

The buffer to read from.

Properties

Current

Gets the element in the collection at the current position of the enumerator.

```
public readonly CodePoint Current { get; }
```

Property Value

CodePoint

Methods

GetEnumerator()

Returns an enumerator that iterates through the collection.

```
public readonly SpanCodePointEnumerator GetEnumerator()
```

Returns

SpanCodePointEnumerator

An enumerator that iterates through the collection.

MoveNext()

Advances the enumerator to the next element of the collection.

```
public bool MoveNext()
```

Returns

bool

true if the enumerator was successfully advanced to the next element; false if the enumerator has passed the end of the collection.

Struct SpanGraphemeEnumerator

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

An enumerator for retrieving Grapheme instances from a ReadOnlySpan<T>.
Implements the Unicode Grapheme Cluster Algorithm. UAX:29
https://www.unicode.org/reports/tr29/
Supports the UAX #29 extended grapheme cluster rule for Indic conjunct sequences (GB9c)
using the IndicConjunctBreakClass property.
Methods are pattern-matched by compiler to allow using foreach pattern.

```
public ref struct SpanGraphemeEnumerator
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() ,
object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

SpanGraphemeEnumerator(ReadOnlySpan<char>)

Initializes a new instance of the SpanGraphemeEnumerator struct.

```
public SpanGraphemeEnumerator(ReadOnlySpan<char> source)
```

Parameters

`source` ReadOnlySpan<char>

The buffer to read from.

SpanGraphemeEnumerator(ReadOnlySpan<char>, TerminalWidthOptions)

Initializes a new instance of the SpanGraphemeEnumerator struct.

```
public SpanGraphemeEnumerator(ReadOnlySpan<char> source, TerminalWidthOptions
terminalWidthOptions)
```

Parameters

`source` ReadOnlySpan<char>

The buffer to read from.

`terminalWidthOptions` TerminalWidthOptions

The terminal width options to apply while enumerating.

Properties

Current

Gets the element in the collection at the current position of the enumerator.

```
public readonly GraphemeCluster Current { get; }
```

Property Value

GraphemeCluster

Methods

GetEnumerator()

Returns an enumerator that iterates through the collection.

```
public readonly SpanGraphemeEnumerator GetEnumerator()
```

Returns

SpanGraphemeEnumerator

An enumerator that iterates through the collection.

MoveNext()

Advances the enumerator to the next element of the collection.

```
public bool MoveNext()
```

Returns

bool

true if the enumerator was successfully advanced to the next element; false if the enumerator has passed the end of the collection.

Struct SpanWordEnumerator

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

An enumerator for retrieving word-boundary segments from a ReadOnlySpan<T>.
Implements the Unicode Word Boundary Algorithm. UAX #29
https://www.unicode.org/reports/tr29/
Methods are pattern-matched by compiler to allow using foreach pattern.

```
public ref struct SpanWordEnumerator
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() ,
object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

SpanWordEnumerator(ReadOnlySpan<char>)

Initializes a new instance of the SpanWordEnumerator struct.

```
public SpanWordEnumerator(ReadOnlySpan<char> source)
```

Parameters

`source` ReadOnlySpan<char>

The buffer to read from.

Properties

Current

Gets the element in the collection at the current position of the enumerator.

```
public readonly WordSegment Current { get; }
```

Property Value

WordSegment

Methods

GetEnumerator()

Returns an enumerator that iterates through the collection.

```
public readonly SpanWordEnumerator GetEnumerator()
```

Returns

SpanWordEnumerator

An enumerator that iterates through the collection.

MoveNext()

Advances the enumerator to the next element of the collection.

```
public bool MoveNext()
```

Returns

bool

true if the enumerator was successfully advanced to the next element; false if the enumerator has passed the end of the collection.

Enum TerminalAmbiguousWidth

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Defines how East Asian Width Ambiguous scalars resolve for terminal cell measurement.

```
public enum TerminalAmbiguousWidth
```

Fields

`Narrow = 0`

Resolve ambiguous scalars as one terminal cell.

`Wide = 1`

Resolve ambiguous scalars as two terminal cells.

Enum TerminalControlCharacterWidth

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Defines how C0 and C1 control scalars resolve for terminal cell measurement.

```
public enum TerminalControlCharacterWidth
```

Fields

`Narrow = 2`

Resolve controls as one terminal cell.

`NonPrintable = 0`

Resolve controls as non-printable.

`Zero = 1`

Resolve controls as zero terminal cells.

Enum TerminalEmojiWidth

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Defines how emoji clusters resolve for terminal cell measurement.

```
public enum TerminalEmojiWidth
```

Fields

`EastAsianWidth = 1`

Do not apply a whole-cluster emoji override; use East Asian Width-derived scalar widths.

`Wide = 0`

Resolve emoji clusters as two terminal cells.

Struct TerminalWidthOptions

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Defines policy used when resolving grapheme clusters to terminal cell widths.

```
public struct TerminalWidthOptions
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

AmbiguousWidth

Gets or sets the width used for East Asian Width Ambiguous scalars.

```
public TerminalAmbiguousWidth AmbiguousWidth { readonly get; set; }
```

Property Value

TerminalAmbiguousWidth

ControlCharacterWidth

Gets or sets the width policy used for C0 and C1 control scalars.

```
public TerminalControlCharacterWidth ControlCharacterWidth { readonly get; set; }
```

Property Value

TerminalControlCharacterWidth

Default

Gets the default terminal width policy.

```
public static TerminalWidthOptions Default { get; }
```

Property Value

TerminalWidthOptions

EmojiWidth

Gets or sets the width policy used for emoji clusters.

```
public TerminalEmojiWidth EmojiWidth { readonly get; set; }
```

Property Value

TerminalEmojiWidth

Enum VerticalOrientationType

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Vertical_Orientation property values. https://www.unicode.org/reports/tr50/#vo

```
public enum VerticalOrientationType
```

Fields

`Rotate = 1`

Rotated (R): displayed sideways, rotated 90 degrees clockwise from the code charts.

`TransformRotate = 3`

Transformed rotated (Tr): normally uses a vertical alternate glyph, falling back to rotated.

`TransformUpright = 2`

Transformed upright (Tu): normally uses a vertical alternate glyph, falling back to upright.

`Upright = 0`

Upright (U): displayed upright with the same orientation used in the code charts.

Remarks

These values are used when laying out text vertically. They describe the default orientation of the character's code chart glyph and whether a vertical alternate glyph should be used when the font supplies one.

Enum WordBreakClass

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Unicode Word_Break property values. https://www.unicode.org/reports/tr29/#Word_Boundaries

```
public enum WordBreakClass : uint
```

Fields

`ALetter = 9`

Alphabetic letters.

`CarriageReturn = 0`

U+000D CARRIAGE RETURN (CR).

`DoubleQuote = 11`

Double quote.

`Extend = 3`

Extending code points that are ignored by most word boundary rules.

`ExtendNumLet = 16`

Connector characters that extend letters, numbers, and Katakana.

`Format = 6`

Format characters that are ignored by most word boundary rules.

`HebrewLetter = 8`

Hebrew letters.

`Katakana = 7`

Katakana characters.

`LineFeed = 1`

U+000A LINE FEED (LF).

`MidLetter = 13`

Mid-letter punctuation.

`MidNum = 14`

Mid-number punctuation.

`MidNumLet = 12`

Mid-letter and mid-number punctuation.

`Newline = 2`

Newline characters other than CR and LF.

`Numeric = 15`

Numeric characters.

`Other = 255`

Other.

`RegionalIndicator = 5`

Regional indicator symbols used to build flag emoji pairs.

`SingleQuote = 10`

Single quote.

`WSegSpace = 17`

Horizontal whitespace segmented as word-segmentation space.

`ZeroWidthJoiner = 4`

U+200D ZERO WIDTH JOINER.

Struct WordSegment

Namespace: SixLabors.Fonts.Unicode

Assembly: SixLabors.Fonts.dll

Represents a segment between two Unicode word boundaries.

```
public readonly ref struct WordSegment
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

WordSegment(ReadOnlySpan<char>, int, int, int)

Initializes a new instance of the WordSegment struct.

```
public WordSegment(ReadOnlySpan<char> span, int utf16Offset, int codePointOffset,
int codePointCount)
```

Parameters

`span` ReadOnlySpan<char>

The UTF-16 span containing the word-boundary segment.

`utf16Offset` int

The UTF-16 offset of the segment in the original source.

`codePointOffset` int

The code point offset of the segment in the original source.

`codePointCount` int

The number of Unicode scalar values in the segment.

Properties

CodePointCount

Gets the number of Unicode scalar values in the segment.

```
public int CodePointCount { get; }
```

Property Value

int

CodePointOffset

Gets the code point offset of the segment in the original source.

```
public int CodePointOffset { get; }
```

Property Value

int

Span

Gets the UTF-16 span containing the word-boundary segment.

```
public ReadOnlySpan<char> Span { get; }
```

Property Value

ReadOnlySpan<char>

Utf16Length

Gets the UTF-16 length of the segment.

```
public int Utf16Length { get; }
```

Property Value

int

Utf16Offset

Gets the UTF-16 offset of the segment in the original source.

```
public int Utf16Offset { get; }
```

Property Value

int

Namespace SixLabors.Fonts.WellKnownIds

Enums

KnownNameIds

Provides enumeration of common name ids https://docs.microsoft.com/en-us/typography/opentype/spec/name#name-ids

Enum KnownNameIds

Namespace: SixLabors.Fonts.WellKnownIds

Assembly: SixLabors.Fonts.dll

Provides enumeration of common name ids https://docs.microsoft.com/en-us/typography/opentype/spec/name#name-ids

```
public enum KnownNameIds : ushort
```

Fields

`CopyrightNotice = 0`

The copyright notice

`Description = 10`

Description; description of the typeface. Can contain revision information, usage recommendations, history, features, etc.

`Designer = 9`

Designer; name of the designer of the typeface.

`DesignerUrl = 12`

URL Designer; URL of typeface designer (with protocol, e.g., http://, ftp://).

`FontFamilyName = 1`

The font family name; Up to four fonts can share the Font Family name, forming a font style linking group (regular, italic, bold, bold italic — as defined by OS/2.fsSelection bit settings).

`FontSubfamilyName = 2`

The font subfamily name; The Font Subfamily name distinguishes the font in a group with the same Font Family name (name ID 1). This is assumed to address style (italic, oblique) and weight (light, bold, black, etc.). A font with no particular differences in weight or style (e.g. medium weight, not italic and fsSelection bit 6 set) should have the string "Regular" stored in this position.

`FullFontName = 4`

The full font name; a combination of strings 1 and 2, or a similar human-readable variant. If string 2 is "Regular", it is sometimes omitted from name ID 4.

`LicenseDescription = 13`

License Description; description of how the font may be legally used, or different example scenarios for licensed use. This field should be written in plain language, not legalese.

`LicenseInfoUrl = 14`

License Info URL; URL where additional licensing information can be found.

`Manufacturer = 8`

The manufacturer

`PostscriptName = 6`

Postscript name for the font; Name ID 6 specifies a string which is used to invoke a PostScript language font that corresponds to this OpenType font. When translated to ASCII, the name string must be no longer than 63 characters and restricted to the printable ASCII subset, codes 33 to 126, except for the 10 characters '[', ']', '(', ')', '{', '}', '<', '>', '/', '%'. In a CFF OpenType font, there is no requirement that this name be the same as the font name in the CFF’s Name INDEX. Thus, the same CFF may be shared among multiple font components in a Font Collection. See the 'name' table section of Recommendations for OpenType fonts "" for additional information.

`SampleText = 19`

Sample text; This can be the font name, or any other text that the designer thinks is the best sample to display the font in.

`Trademark = 7`

Trademark; this is used to save any trademark notice/information for this font. Such information should be based on legal advice. This is distinctly separate from the copyright.

`TypographicFamilyName = 16`

Typographic Family name: The typographic family grouping doesn't impose any constraints on the number of faces within it, in contrast with the 4-style family grouping (ID 1), which is present both for historical reasons and to express style linking groups. If name ID 16 is absent,

then name ID 1 is considered to be the typographic family name. (In earlier versions of the specification, name ID 16 was known as "Preferred Family".)

`TypographicSubfamilyName = 17`

Typographic Subfamily name: This allows font designers to specify a subfamily name within the typographic family grouping. This string must be unique within a particular typographic family. If it is absent, then name ID 2 is considered to be the typographic subfamily name. (In earlier versions of the specification, name ID 17 was known as "Preferred Subfamily".)

`UniqueFontID = 3`

The unique font identifier

`VendorUrl = 11`

URL Vendor; URL of font vendor (with protocol, e.g., http://, ftp://). If a unique serial number is embedded in the URL, it can be used to register the font.

`Version = 5`

Version string. Should begin with the syntax 'Version <number>.<number>' (upper case, lower case, or mixed, with a space between "Version" and the number). The string must contain a version number of the following form: one or more digits (0-9) of value less than 65,535, followed by a period, followed by one or more digits of value less than 65,535. Any character other than a digit will terminate the minor number. A character such as ";" is helpful to separate different pieces of version information. The first such match in the string can be used by installation software to compare font versions. Note that some installers may require the string to start with "Version ", followed by a version number as above.

`WwsFamilyName = 21`

WWS Family name: Used to provide a family grouping for faces that differ only in weight, width, or slope.

`WwsSubfamilyName = 22`

WWS Subfamily name: Used to describe weight, width, or slope within a WWS family grouping.